INTELLIGENT CLOUD INCIDENT AND CHANGE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SYED FAIYAZ AHAMED

(241401113)

in partial fulfilment of the End Semester Examination for the course

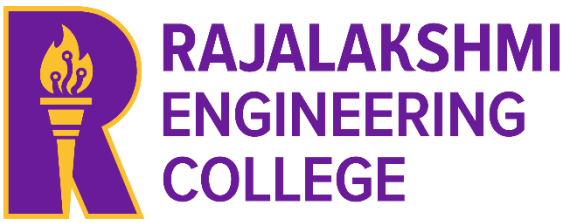
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**INTELLIGENT CLOUD INCIDENT AND CHANGE MANAGEMENT SYSTEM**" is the bonafide work of **SYED FAIYAZ AHAMED (241401113)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R. SOWMIA
HEAD OF THE DEPARTMENT
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

ALPHIN EZHIL MANUEL M L
COURSE FACULTY
Assistant Professor (SG)
Department of Computer Science and Business Systems
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **SYED FAIYAZ AHAMED (241401113)**, hereby declare that the project report titled **"INTELLIGENT CLOUD INCIDENT AND CHANGE MANAGEMENT SYSTEM"**, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 03 November 2026

SIGNATURE OF THE STUDENT
SYED FAIYAZ AHAMED
241401113

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, **Mr. S. Meganathan**, **B.E., F.I.E.,** Chairperson, **Dr. Thangam Meganathan, Ph.D.,** and Vice-Chairman, **Mr. Abhay Shankar Meganathan, B.E., M.S.,** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S. N. Murugesan, M.E., Ph.D.,** Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. K. R. Sowmia, Ph.D.,** Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **Alphin Ezhil Manuel M L**, **Assistant Professor (SG)**, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

SYED FAIYAZ AHAMED

ABSTRACT

Organizations that depend on cloud infrastructure require efficient IT service management to resolve incidents and implement infrastructure changes safely. In many organizations, incidents are reported through emails and telephone calls, tracked using spreadsheets and assigned manually, while change requests are reviewed through separate communication channels. These practices can cause delayed incident assignment, missed Service Level Agreement (SLA) deadlines, difficulties in tracking approvals and additional effort in preparing reports.

This project presents the Intelligent Cloud Incident & Change Management System, a centralized application developed using Zoho Creator, a low-code application development platform. It consists of four main forms: Staff Directory, Incident Management, Change Request and Change Approval History, supported by reports and a centralized dashboard. Deluge scripts and workflows automate incident priority calculation, SLA deadline generation, engineer assignment, status notifications and approval history creation. A scheduled workflow monitors overdue incidents and sends notifications. The Zoho Desk integration automatically creates a support ticket whenever an incident is submitted. Four application roles, namely Employee, IT Support Staff, Cloud Engineer and Approving Manager, are configured with separate permission profiles to control access to application components.

The application was developed and tested within the Zoho Creator environment. Functional and integration testing covered incident submission, priority calculation, engineer assignment, SLA notifications, approval history creation and Zoho Desk ticket generation. Identified configuration and scripting errors were corrected during testing. Role-based permissions and restricted module access provide basic security controls. Although the application has not been deployed to a separate production environment and formal performance measurements were not conducted, it demonstrates the potential to reduce repetitive manual activities, centralize service records and improve coordination. The project highlights the practical use of low-code development for IT service management.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; IT Incident Management; Change Management.*

PROJECT AT A GLANCE

Attribute	Details
Project Title	Intelligent Cloud Incident & Change Management System
Application Domain	Cloud Computing – IT Service Management
Student Name & Register No.	Syed Faiyaz Ahamed – 241401113
Zoho Creator Edition	Trial Edition
Application Name (Link Name)	Cloud Incident & Change Management System
Published / Live URL	https://creatorapp.zoho.in/syedfaiyazahamed.2024.csb716/cloud-incident-change-management-system
Demo Credentials (per role)	Administrator, Employee, IT Support Staff, Cloud Engineer and Approving Manager logins are provided to the examiner during the demonstration (passwords are not printed in this report).
Number of Forms / Subforms	4 / 0
Number of Reports (by type)	List: 6 | Kanban: 0 | Calendar: 0 | Pivot/Chart: [Verify] | Others: 0
Number of Pages / Dashboards	1 dashboard
Number of Workflows	Form: 8 | Schedule: 1 | Approval: 0 | Blueprint: 0 | Batch: 0
Deluge Functions / Custom Actions	0 standalone functions; Deluge scripts are embedded in workflows
Integrations	Zoho Desk – Automatic incident ticket creation through OAuth connection
Roles & Profiles Defined	Employee, IT Support Staff, Cloud Engineer and Approving Manager; Administrator access
Environments Used	Development and testing in Zoho Creator
Date of Production Deployment	Not applicable – separate production deployment not performed

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES xi
LIST OF ABBREVIATIONS xiii
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 3
1.3 Objectives 4
1.4 Scope of the Project 4
1.5 Justification for a Low-Code Approach 6
1.6 Organisation of the Report 8
LITERATURE SURVEY AND SYSTEM STUDY 9
2.1 Study of the Existing System 9
2.2 Limitations of the Existing System 11
2.3 Low-Code versus Traditional Development 12
2.4 Literature Survey 13
2.5 Emerging Trends in Low-Code Development 15
2.6 Proposed System 17
REQUIREMENT ANALYSIS 20
3.1 Stakeholders and User Roles 20
3.2 Functional Requirements 20
3.3 Non-Functional Requirements 21
3.4 Hardware and Software Requirements 22
3.5 Use Case Model 23
SYSTEM DESIGN 26
4.1 System Architecture 26
4.2 Data Model 27
4.3 Form Design 29
4.4 Process and Workflow Design 33
4.5 User Interface and Navigation Design 36
4.6 Access Control Design 37
IMPLEMENTATION 38
5.1 Application Setup 38
5.2 Forms and Data Capture 39

5.3 Reports 43
5.4 Pages and Dashboards 47
5.5 Deluge Scripting 49
5.6 Workflows and Automation 58
5.7 Integrations 60
5.8 User Management and Role-Based Access 63
5.9 AI and Analytics Features 65
5.10 Pro-Code Extension 66
5.11 Mobile and Portal Access 67
TESTING AND DEBUGGING 68
6.1 Testing Strategy 68
6.2 Test Cases 69
6.3 Debugging Deluge Scripts 72
6.4 Defect Log 73
6.5 User Acceptance Testing 74
DEPLOYMENT, SECURITY AND GOVERNANCE 75
7.1 Environments and Deployment Strategy 75
7.2 Performance Optimisation 77
7.3 Security Features 78
7.4 Governance and Administration 80
RESULTS AND DISCUSSION 81
8.1 Application Output 81
8.2 Achievement of Objectives 83
8.3 Comparison with the Existing System 84
8.4 Limitations of the Low-Code Approach 86
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 87
CONCLUSION AND FUTURE ENHANCEMENTS 89
9.1 Conclusion 89
9.2 Future Enhancements 89
9.3 Learning Outcomes and Career Relevance 90
REFERENCES 91
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 93
APPENDIX B – COURSE OUTCOME MAPPING 95
APPENDIX C – COMPLETE DELUGE SOURCE CODE 96
APPENDIX D – USER MANUAL 103
D.1 Accessing the Application 103
D.2 Instructions for Emloyee 103

D.3 Instructions for IT Support Staff 104
D.4 Instructions for Cloud Engineer 1045
D.5 Instructions for Approving Manager 1045
APPENDIX E – EVALUATION RUBRIC 106
APPENDIX F – VIVA-VOCE RECORD SHEET 108

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 12
Table 2.2: Summary of Literature Survey 14
Table 2.3: Comparison of Existing System Limitations and Proposed Solutions 18
Table 3.1: Stakeholders and User Roles 20
Table 3.2: Functional Requirements 20
Table 3.3: Non-Functional Requirements 21
Table 3.4: Hardware and Software Requirements 22
Table 3.5: Use Case Description – Submit Incident 24
Table 3.6: Use Case Description – Approve / Reject Change 24
Table 3.7: Use Case Description – Review and Assign Incident 25
Table 4.1: Entities and Relationships 28
Table 4.2: Field Specification – Staff Directory Form 29
Table 4.3: Field Specification – Incident Management Form 30
Table 4.4: Field Specification – Change Request Form 31
Table 4.5: Field Specification – Change Approval History Form 32
Table 4.6: Role–Permission Matrix 37
Table 5.1: Reports Implemented 43
Table 5.2: Deluge Scripts Implemented 49
Table 5.3: Workflows Implemented 58
Table 5.4: Integrations Implemented 60
Table 6.1: Test Cases 69
Table 6.2: Defect Log 73
Table 6.3: User Acceptance Feedback 74
Table 7.1: Deployment Checklist 76
Table 7.2: Performance Optimisation Results 77
Table 7.3: Security Controls Implemented 79
Table 8.1: Mapping of Objectives to Outcomes 83
Table 8.2: Existing System versus Proposed Application 84
Table A.1: Feature Coverage Checklist 93
Table B.1: Mapping of Course Outcomes to Report Chapters 95
Table E.1: Evaluation Rubric for the Individual Project 106
Table E.2: Performance Level Descriptors 107
Table F.1: Viva-Voce Questions and Responses 108
Table F.2: Marks Summary 108

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 11
Figure 3.1: Use Case Diagram of the Application 23
Figure 4.1: System Architecture of the Application 26
Figure 4.2: Entity–Relationship Diagram 28
Figure 4.3: Process Flow for Incident Management 34
Figure 4.4: Process Flow for Change Request Management 35
Figure 4.5: Navigation Structure of Cloud Incident & Change Management System 36
Figure 4.6: Wireframe of Cloud Incident & Change Management System 36
Figure 5.1: Application Home and Navigation Sections 38
Figure 5.2: Staff Directory Form in the Form Builder 39
Figure 5.3: Staff Directory Form in Live Mode 40
Figure 5.4: Incident Management Form in the Form Builder 40
Figure 5.5: Incident Management Form in Live Mode 41
Figure 5.6: Change Request Form in the Form Builder 41
Figure 5.7: Change Request Form in Live Mode 42
Figure 5.8: Change Approval History Form in the Form Builder 42
Figure 5.9: Change Approval History Form in Live Mode 43
Figure 5.10: Staff Directory Report 44
Figure 5.11: All Incidents Report 45
Figure 5.12: Assigned Incidents Report 45
Figure 5.13: All Change Requests Report 46
Figure 5.10: Pending Change Approval Report 46
Figure 5.10: Change Approval History Report 47
Figure 5.11(a): Incident & Change Management Dashboard 48
Figure 5.11(b): Incident & Change Management Dashboard 48
Figure 5.12: Form Workflow Configuration – Incident Management Form 59
Figure 5.13: Form Workflow Configuration – Change Request Form 59
Figure 5.14: Schedule Workflow Configuration – Incident Management Form 60
Figure 5.15: Accessing Microservices in Zoho Creator 61
Figure 5.16: Zoho Desk Connection Configuration 62
Figure 5.17: Successful Zoho Desk Ticket Creation After Incident Submission 62
Figure 5.18: Application Roles Configuration 64
Figure 5.19: Role-Based Permission Profiles Configuration 64
Figure 5.20: User Role and Permission Assignment 65
Figure 5.21: List of Users Added to the Application 65

Figure 5.22: AI-Assisted Dashboard and Analytics in Zoho Creator 66
Figure 6.1: Deluge Debug Output Showing Successful Zoho Desk Integration 73
Figure 7.1: Environments and Application Publishing in Zoho Creator 76
Figure 7.2: Audit Trail and Application Administration in Zoho Creator 80
Figure 8.1: Scenario Step 1 – Incident Submission 82
Figure 8.2: Scenario Step 2 – Automatic Priority Calculation and Engineer Assignment 82
Figure 8.3: Scenario Step 3 – Zoho Desk Ticket Creation 83

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SSO	Single Sign-On
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Cloud computing has become an essential part of modern business operations, enabling organizations to access computing resources such as servers, storage, databases, networks and software applications through the internet. Many organizations depend on cloud-based infrastructure to maintain their daily operations, store important information and provide uninterrupted services to customers. As the usage of cloud services increases, managing technical issues and infrastructure changes has become an important responsibility for IT departments.

The project, **Intelligent Cloud Incident & Change Management System**, is designed for a medium-sized IT services organization that manages cloud infrastructure and provides technical support to its employees and business operations. For this project, a hypothetical organization named CloudTech Solutions is considered. The organization consists of approximately 200 employees, including system administrators, cloud engineers, IT support executives, department managers and general employees. Its IT department is responsible for maintaining cloud servers, applications, databases, network connectivity and other essential technology resources.

In a typical organization, IT-related problems may occur due to server downtime, network failures, application errors, database connectivity issues, configuration problems and unexpected service interruptions. These problems are generally reported by employees to the IT support team through emails, telephone calls or direct communication. The support team collects the issue details, identifies its priority, assigns it to the appropriate technical staff and works towards resolving it. Once the issue is resolved, the employee is informed and the incident is closed.

Similarly, organizations frequently require changes to their existing IT infrastructure to improve performance, introduce new services, update software, modify configurations or upgrade cloud resources. Such modifications must be carefully planned and reviewed before implementation because an incorrect or unauthorized change may affect system availability and business operations. The change management process generally involves submitting a change request, reviewing its purpose and potential risks,

obtaining approval from authorized personnel, implementing the change and verifying its outcome.

However, in many organizations, these processes are handled using separate communication channels, spreadsheets and manual records. Incident details may be scattered across emails, making it difficult to identify pending issues and monitor their progress. Manual assignment of incidents can also result in delays, while the absence of a structured change approval process may increase the possibility of unauthorized modifications. Furthermore, tracking service-level agreement (SLA) deadlines and preparing performance reports can become time-consuming.

The major stakeholders involved in these processes include:

1. **Employees:** Report technical problems and monitor the status of their submitted incidents.
2. **IT Support Staff:** Receive incidents, investigate problems, assign priorities and provide technical solutions.
3. **Cloud Engineers:** Handle complex cloud infrastructure issues and implement approved technical changes.
4. **Department Managers:** Review and approve relevant change requests based on business requirements.**System Administrators:** Manage application access, user roles, system configurations and overall administration.

To address these challenges, the proposed application provides a centralized platform for managing IT incidents and infrastructure changes. It enables employees to submit incident reports, allows IT personnel to track and resolve issues, and provides a structured workflow for submitting and approving change requests. The system also supports automated notifications, priority-based tracking, SLA monitoring and performance reporting.

The application is developed using **Zoho Creator**, a low-code application development platform that provides tools for creating forms, reports, dashboards, workflows and automated business processes. Its capabilities can be used to reduce manual work and improve coordination between different stakeholders. AI-assisted features can also be incorporated to support incident categorization and provide relevant resolution suggestions.

Thus, the Intelligent Cloud Incident & Change Management System aims to improve the efficiency, transparency and reliability of IT service operations by bringing incident management and change management activities into a single integrated application.

1.2 Problem Statement

Organizations that depend on cloud-based infrastructure often face difficulties in managing IT incidents and infrastructure changes due to the use of manual processes and disconnected communication channels. Incident reporting, assignment, tracking and resolution are frequently handled through emails, spreadsheets and direct communication, resulting in delays and a lack of coordination among employees and IT support teams. Similarly, infrastructure change requests may experience delays due to manual approval procedures and insufficient tracking. The absence of a centralized system makes it difficult to monitor incident status, maintain accountability, track Service Level Agreement (SLA) deadlines and generate accurate performance reports. Therefore, a centralized and automated application is required to simplify incident management, streamline change approval processes and improve the overall efficiency of IT operations.

Major Pain Points:

- **Manual Incident Tracking:** Incident details are maintained through emails and spreadsheets, making it difficult to organize, update and retrieve records efficiently.
- **Delayed Incident Assignment and Resolution:** Manual communication and assignment processes cause delays in identifying responsible IT personnel and resolving technical issues.
- **Lack of Structured Change Approval:** Infrastructure changes may face delays due to manual approval procedures and insufficient tracking of approval status.
- **No Real-Time Monitoring:** Limited visibility into pending incidents, overdue requests, priorities and SLA deadlines makes operational monitoring difficult.
- **Lack of Centralized Reporting and Access Control:** Manual report preparation and inadequate role-based access make it difficult to maintain data security, accountability and performance transparency.

1.3 Objectives

The main objectives of the Intelligent Cloud Incident & Change Management System are:

- To design a structured and normalized data model with appropriate forms for managing users, IT incidents, change requests, approvals and resolution records.
- To automate incident assignment, priority updates, status tracking, change request approvals and SLA notifications using Deluge scripting and Zoho Creator workflows.
- To develop role-based dashboards and reports for employees, IT support staff, cloud engineers and administrators to monitor incidents, change requests and operational performance.
- To implement a structured change management process that enables users to submit change requests and authorized personnel to review, approve or reject them before implementation.
- To integrate Zoho Creator with suitable Zoho services and AI-assisted capabilities to support automated notifications, incident categorization and resolution recommendations.
- To deploy the application securely with role-based access control, data validation and appropriate governance measures to ensure reliable and controlled system usage.

1.4 Scope of the Project

The scope of the Intelligent Cloud Incident & Change Management System is to develop a centralized, web-based application using Zoho Creator for managing IT incidents and infrastructure change requests within an organization. The application focuses on improving incident reporting, tracking, resolution, change approvals and operational monitoring through automated workflows and role-based access.

A. In-Scope

The following functionalities are included in the proposed application:

1. User Roles and Access Management

- Administrator: Manages users, application settings, permissions and overall system operations.

- IT Support Staff: Handles reported incidents, assigns priorities and updates resolution status.
- Cloud Engineers: Investigate technical issues and implement approved infrastructure changes.
- Employees: Submit incidents, raise change requests where authorized and track their requests.
- Approving Managers: Review, approve or reject change requests.

2. Application Modules

- Incident Management: Registration, categorization, prioritization, assignment and resolution of IT incidents.
- Change Management: Submission, review, approval, rejection and tracking of infrastructure change requests.
- SLA Management: Monitoring incident resolution deadlines and identifying overdue incidents.
- Notification Management: Automated notifications for incident assignments, status changes, pending approvals and SLA alerts.
- Dashboard and Reports: Displaying incident statistics, pending requests, resolution performance and change request summaries.
- AI-Assisted Support: Providing intelligent incident categorization and suggested resolutions using available AI capabilities.

3. Platform and Accessibility

- Web-based application developed using Zoho Creator.
- Access through supported web browsers on desktop and mobile devices.
- Role-based access to forms, reports and dashboards.
- Centralized data storage and management within the Zoho Creator environment.

B. Out-of-Scope

The following functionalities are not included in the current project implementation:

- Direct monitoring and automatic detection of failures in live cloud servers.

- Actual provisioning, configuration or deployment of cloud infrastructure.
- Integration with every third-party cloud provider such as AWS, Microsoft Azure and Google Cloud.
- Advanced cybersecurity monitoring, threat detection and penetration testing.
- Full-scale enterprise IT service management and asset discovery.
- Native Android and iOS application development outside the Zoho Creator platform.
- Automated execution of infrastructure changes without human approval.
- Advanced predictive analytics using large-scale historical incident datasets.

The project primarily focuses on automating and simplifying incident and change management activities rather than directly controlling cloud infrastructure. The proposed system provides an organized environment for managing IT service operations, improving coordination and maintaining proper records of incidents and infrastructure changes.

1.5 Justification for a Low-Code Approach

The Intelligent Cloud Incident & Change Management System requires multiple functionalities, including data management, incident tracking, approval workflows, automated notifications, role-based access and interactive dashboards. Developing these features using traditional programming would require considerable time, coding effort and infrastructure maintenance. Therefore, a low-code development approach using Zoho Creator is considered suitable for this project.

Zoho Creator provides a visual development environment with drag-and-drop tools, pre-built components and customizable workflows. It allows developers to create applications with minimal manual coding while providing the flexibility to implement advanced business logic using Deluge scripting[1].

The following factors justify the selection of Zoho Creator for this project:

1. Faster Application Development

Zoho Creator provides ready-made forms, reports, pages and workflow components that reduce the time required to develop the application. Incident registration forms,

change request forms and dashboards can be created using its visual interface without developing every component from scratch.

2. Easy Maintenance and Citizen Developer Support

The low-code environment allows developers and business users with basic technical knowledge to maintain and modify the application. Forms, fields, workflows and reports can be updated easily as organizational requirements change. Deluge scripting can be used when additional customization is required.

3. Built-in Cloud Hosting

Zoho Creator provides a cloud-based environment for hosting and managing applications. It reduces the need for separate server configuration, infrastructure setup and maintenance. This allows the development team to focus more on application functionality and business requirements.

4. Web and Mobile Accessibility

Applications developed using Zoho Creator can be accessed through web browsers and supported mobile devices. This allows employees and IT support staff to report incidents, update request statuses and access relevant information from different locations[1].

5. Security and Access Control

Zoho Creator provides security features such as role-based access control, user permissions, data encryption and audit-related capabilities. These features support controlled access to incident records, change requests and administrative functions, helping maintain data confidentiality and accountability[2].

6. Cost-Effective Development

The low-code approach reduces the amount of manual programming and infrastructure management required for application development. Its reusable components and built-in services can reduce development and maintenance effort compared with building an equivalent application entirely from scratch. Zoho Creator also provides different pricing options based on organizational requirements.

7. Integration and Extensibility

Zoho Creator supports integration with other Zoho applications and external services through built-in connectors and APIs. This provides opportunities to extend the application with additional notification services, analytics tools and AI-assisted functionalities as future requirements arise[1].

Based on these advantages, Zoho Creator is an appropriate platform for developing the proposed application. Its low-code capabilities, workflow automation, cloud hosting, mobile accessibility and security features make it suitable for building and maintaining an efficient incident and change management solution with reduced development complexity.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In many organizations, IT incident management and infrastructure change management are carried out using manual methods and separate communication channels such as emails, telephone calls, spreadsheets and direct communication. These methods are used to report technical issues, assign responsibilities, monitor progress and obtain approvals for infrastructure changes. However, the absence of a centralized application makes it difficult to maintain consistent records and monitor the overall progress of IT operations.

The existing incident management process generally follows a sequence of manual activities. When an employee encounters an IT-related problem, such as server downtime, network connectivity failure or application malfunction, the issue is reported to the IT support team through email or telephone. The support team records the incident details, identifies its severity and assigns it to the appropriate technical personnel. The assigned staff investigate the issue, identify the cause and perform the necessary corrective actions. Once the problem is resolved, the employee is informed and the incident is marked as closed. However, tracking pending incidents and monitoring resolution deadlines can be difficult when records are maintained separately.

Similarly, the existing change management process begins when a department or technical team identifies the need for an infrastructure modification. A change request is prepared manually and submitted to the concerned manager or administrator for approval. The request is reviewed based on its purpose, expected impact and associated risks. After approval, the technical team implements the change and verifies whether it has been completed successfully. If the request is rejected, the concerned team is informed and the request is closed. Since approvals and updates are handled through manual communication, the process may experience delays and a lack of transparency.

The major steps involved in the existing system are described below.

A. Existing Incident Management Process

1. **Incident Identification:** An employee identifies a technical issue affecting an IT service or application.
2. **Incident Reporting:** The employee reports the issue to the IT support team through email, telephone or direct communication.
3. **Incident Recording:** The support team manually records the incident details, including the description and reported date.
4. **Incident Assignment:** The incident is assigned to the appropriate technical personnel based on its nature and severity.
5. **Investigation and Resolution:** The assigned personnel investigate the issue and perform the necessary corrective actions.
6. **Incident Closure:** After successful resolution, the employee is informed and the incident is marked as closed.

B. Existing Change Management Process

1. **Change Identification:** A department or technical team identifies the requirement for modifying existing IT infrastructure.
2. **Change Request Submission:** A change request is prepared with details about the proposed modification and its purpose.
3. **Request Review:** The concerned manager or administrator examines the request and evaluates its possible impact and risks.
4. **Approval or Rejection:** The authorized personnel approve or reject the request based on the evaluation.
5. **Change Implementation:** Once approved, the technical team carries out the required infrastructure modification.
6. **Verification and Closure:** The implemented change is verified, documented and marked as completed.

The existing system depends heavily on manual coordination and documentation. This creates difficulties in maintaining centralized records, tracking incident status, monitoring service-level agreement (SLA) deadlines and ensuring timely approval of

change requests. These limitations highlight the need for an automated and centralized incident and change management application.

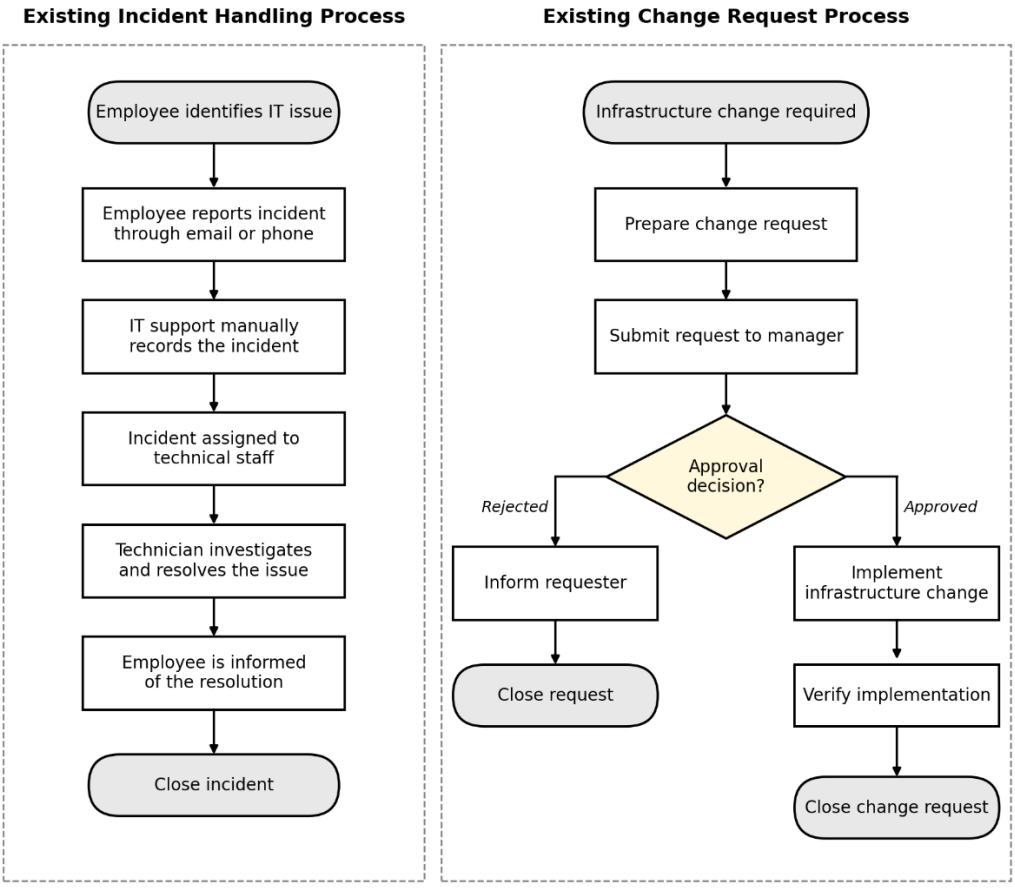


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

- **Lack of Centralized Data Management:** Incident details and change request records are maintained across different emails and spreadsheets, making it difficult to store, retrieve and manage information in a centralized manner.
- **Delayed Incident Assignment and Resolution:** Manual reporting and assignment procedures increase response time and delay the resolution of technical issues.
- **Inefficient Change Approval Process:** Change requests are submitted and reviewed through manual communication, resulting in delays and difficulties in tracking approval status.
- **Absence of Real-Time Monitoring:** The existing system does not provide a centralized view of ongoing incidents, pending requests, priorities and resolution progress.

- **Difficulty in SLA Monitoring:** Manual tracking of incident deadlines makes it difficult to identify overdue incidents and ensure timely resolution.
- **Lack of Automated Notifications:** Employees and technical staff may not receive timely updates regarding incident assignments, status changes, approvals and pending actions.
- **Limited Reporting and Performance Analysis:** Preparing reports manually consumes time and makes it difficult to evaluate incident resolution performance and overall IT service efficiency.
- **Inadequate Access Control:** Maintaining records through shared spreadsheets and emails makes it difficult to control access based on user roles and responsibilities.

2.3 Low-Code versus Traditional Development

The Intelligent Cloud Incident & Change Management System can be developed using either traditional programming or a low-code development platform. Traditional development requires developers to write code for application interfaces, database operations, business logic, authentication, workflows and reporting. In contrast, Zoho Creator provides a visual development environment with built-in components that simplify the development of these functionalities.

For the proposed application, traditional development would require the implementation of separate modules for incident registration, change request management, approval workflows, SLA monitoring, notifications and dashboards. Using Zoho Creator, these modules can be developed with forms, reports, workflows, Deluge scripting and built-in platform services.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Approximately 8–10 weeks, depending on complexity and development resources.	Approximately 2–3 weeks for a basic working application, depending on feature complexity.
Skills required	Requires programming knowledge of frontend, backend, databases, APIs and server-side technologies.	Requires basic application development knowledge, drag-and-drop tools and Deluge scripting.

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Hosting & infrastructure	Requires separate server setup, database configuration, deployment and infrastructure maintenance.	Provides a cloud-based application hosting environment through Zoho Creator.
Mobile application	Requires additional responsive development or separate mobile application development.	Supports access through web browsers and mobile devices using the Zoho Creator platform.
Security & access control	Requires manual implementation of authentication, authorization, permissions and security mechanisms.	Provides built-in user management, role-based permissions and application access controls.
Maintenance & change	Modifications may require changes to source code, testing and redeployment.	Forms, reports and workflows can be modified through the visual development environment with minimal coding.
Customisation limits	Provides greater flexibility and complete control over application architecture and functionality.	Supports customisation through Deluge scripting and integrations, but is subject to platform capabilities.
Cost	Generally involves higher initial development effort and additional hosting, infrastructure and maintenance expenses.	Reduces initial development effort and infrastructure management costs, with subscription charges depending on the selected plan.

2.4 Literature Survey

A literature survey is carried out to understand the existing research, technologies and applications related to low-code development and IT Service Management (ITSM). It helps identify the capabilities of existing platforms, their advantages and limitations, and the requirements that can be addressed through the proposed application.

The survey focuses on research related to low-code application development, workflow automation, incident management and change management. Five relevant research and

technical sources have been reviewed to identify their key findings and relevance to the Intelligent Cloud Incident & Change Management System.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	D. Di Ruscio et al., 2022 [3]	Low-Code Development and Model-Driven Engineering: Two Sides of the Same Coin?	Discusses low-code development platforms, visual modelling, reduced manual coding and their role in enabling faster application development.	Supports the selection of Zoho Creator as a low-code platform for developing incident management forms, workflows and dashboards.
2	D. Liu et al., 2024 [4]	What's Wrong With Low-Code Development Platforms? An Empirical Study of Low-Code Development Platform Bugs	Examines 974 confirmed bugs across four low-code platforms and highlights the importance of proper application design, testing and validation.	Emphasizes the need for systematic testing and validation of forms, workflows and application components in the proposed system.
3	Zoho Corporation [5]	Zoho Creator – Form Workflows Documentation	Explains how workflows can be triggered by form events and record operations to perform automated actions such as sending emails, updating fields and calling APIs.	Provides the technical foundation for automating incident assignment, status updates and change request notifications.
4	ServiceNow [6]	ServiceNow IT Service Management – Incident Management	Describes incident management activities such as incident reporting, investigation, resolution, escalation and monitoring through dashboards and reports.	Provides a reference for designing the incident registration, assignment, tracking and resolution modules.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
5	Atlassian [7]	Jira Service Management – Change Management	Explains structured change management practices involving request tracking, risk assessment, coordination and controlled infrastructure changes.	Provides a reference for implementing change request submission, approval, tracking and controlled implementation.

2.5 Emerging Trends in Low-Code Development

Low-code application development is evolving rapidly with the introduction of technologies that simplify application creation, improve automation and reduce development time. Emerging trends such as AI-assisted app building, citizen development, hyper-automation and composable applications influence how modern business applications are designed and developed.

1. AI-Assisted App Building

Artificial Intelligence (AI) is increasingly being used in low-code platforms to assist developers in creating applications. AI capabilities can help generate application components, suggest workflows, analyse data and provide recommendations. In the proposed system, AI-assisted features can support incident categorization and suggest possible resolutions based on incident descriptions. This can reduce manual effort and help IT support staff respond more efficiently. Such features will be included only to the extent supported and configured in the application.

2. Citizen Development

Citizen development allows users with limited programming knowledge to create and maintain applications using visual development tools. Platforms such as Zoho Creator provide drag-and-drop form builders, report creation tools and workflow configurations. This trend influenced the selection of Zoho Creator for the proposed system, as it enables the development of incident and change management modules with minimal conventional coding. Deluge scripting can be used wherever additional customization is required.

3. Hyper-Automation

Hyper-automation involves combining multiple technologies and automated workflows to reduce repetitive manual activities across business processes. In IT service management, automation can support incident assignment, priority updates, approval processes, status changes and notifications. This trend influenced the proposed system's workflow design, where repetitive incident and change management activities can be automated using Zoho Creator workflows and Deluge scripts.

4. Composable Applications

Composable application development involves building applications using independent and reusable components that can be combined to meet specific business requirements. These components may include forms, reports, dashboards, workflows and integrations. This approach influenced the modular design of the proposed system, where Incident Management, Change Management, SLA Management, Notifications and Reporting are organized as separate functional modules within one application. This makes the application easier to maintain and extend.

Application of Emerging Trends in the Proposed System

The emerging trends influenced the design of the Intelligent Cloud Incident & Change Management System in the following ways:

- AI-assisted capabilities provide opportunities for intelligent incident categorization and resolution recommendations.
- Citizen development supports faster application development through Zoho Creator's visual tools.
- Hyper-automation helps reduce manual work through predefined workflows and automated actions.
- Composable application design supports a modular structure, allowing individual features to be maintained and enhanced independently.

Emerging low-code trends provide opportunities to develop applications that are faster to build, easier to maintain and more adaptable to organizational requirements. By adopting relevant concepts from these trends, the proposed system uses Zoho Creator's visual development environment, modular components and workflow automation capabilities to improve IT incident and change management. Advanced AI capabilities

and integrations can be introduced based on platform support and implementation feasibility.

2.6 Proposed System

The proposed system, titled **Intelligent Cloud Incident & Change Management System**, is a centralized web-based application developed using Zoho Creator. It is designed to simplify and improve the management of IT incidents and infrastructure change requests in an organization. The system replaces manual processes involving emails, phone calls and spreadsheets with a structured application that provides centralized data storage, organized workflows, status tracking and reporting.

The application is designed for different organizational roles, including Employees, IT Support Staff, Cloud Engineers, Approving Managers and Administrators. Each role is provided with appropriate access based on its responsibilities. Employees can submit incidents and change requests, support staff can review and assign incidents, engineers can investigate and resolve issues, managers can review change requests, and administrators can manage application settings and access.

Key Components of the Proposed System

1. **Incident Management:** Provides forms to register incidents with details such as description, category, priority and affected service. Incidents can be assigned to responsible personnel and tracked through their status until resolution.
2. **Change Management:** Allows users to submit infrastructure change requests with details about the proposed modification, purpose and impact. Requests can be reviewed and approved or rejected by authorized managers before implementation.
3. **SLA Management:** Supports the monitoring of incident response and resolution deadlines. Appropriate deadline tracking and overdue alerts can be configured through workflows.
4. **Notification Management:** Uses workflow-based notifications to communicate important updates, including incident assignments, status changes and change request decisions.

5. **Dashboard and Reports:** Provides organized views of incidents, change requests, pending approvals and resolution status. These reports help authorized users monitor IT service activities and understand operational performance.
6. **AI-Assisted Support:** Provides scope for intelligent incident categorization and suggested resolutions using suitable AI capabilities, subject to successful integration and configuration.

How the Proposed System Overcomes the Existing Limitations

Table 2.3: Comparison of Existing System Limitations and Proposed Solutions

S.No	Limitations of Existing System (Section 2.2)	Proposed Solution
1	Lack of centralized data	Stores incident details, change requests and related records in a centralized Zoho Creator application.
2	Delayed assignment and resolution	Provides structured incident assignment and status tracking to improve coordination among support staff and engineers.
3	Inefficient change approval	Introduces a defined change request submission, review, approval and rejection process.
4	No real-time monitoring	Provides updated application reports and dashboards to view incident and change request status.
5	Difficult SLA monitoring	Supports deadline tracking and the configuration of overdue alerts through workflows.
6	No automated notifications	Enables workflow-based notifications for important assignments, updates and approval decisions.
7	Limited reporting	Provides centralized reports and dashboards for monitoring incidents, changes and pending activities.
8	Inadequate access control	Uses role-based permissions to restrict access according to user responsibilities.

The proposed system provides a structured and centralized approach to managing IT incidents and infrastructure changes. By using Zoho Creator forms, reports, dashboards,

workflows and Deluge scripting, it aims to reduce manual effort, improve coordination, provide better visibility and support controlled change management. Unlike the existing manual approach, the proposed application brings important IT service management activities together in a single platform. It manages the operational workflow but does not directly detect cloud infrastructure failures or perform infrastructure-level changes.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

The application supports different stakeholders involved in IT incident and change management. Each user is assigned a role with specific responsibilities and access permissions to ensure proper coordination and secure handling of records.

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Manages the overall application and user access.	Manage users, configure application settings, assign permissions and view all records.
Approving Manager	Supervises and authorizes infrastructure change requests.	Review change requests, assess risks, approve or reject requests and monitor dashboards.
IT Support Staff	Handles reported incidents and coordinates their resolution.	Review incidents, assign engineers, update priorities and monitor incident status.
Cloud Engineer	Investigates and resolves technical incidents and implements approved changes.	Analyse assigned incidents, update resolution details and carry out approved changes.
Employee / Portal User	Uses the application to report IT issues and request infrastructure changes.	Submit incidents and change requests, provide required details and track request status.

3.2 Functional Requirements

Functional requirements describe the operations and services that the Intelligent Cloud Incident & Change Management System must provide. They define how users interact with the application and how incidents and change requests are processed.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow employees and staff to register incidents by entering mandatory details such as description, category and affected service.	Employee, IT Support Staff	High
FR-02	The system shall automatically calculate the incident priority and SLA deadline based on the defined rules when an incident is submitted.	System, IT Support Staff	High

FR ID	Requirement	Role(s)	Priority
FR-03	The system shall assign incidents to the least-loaded available support engineer based on the defined assignment criteria.	System, IT Support Staff	High
FR-04	The system shall send notifications to the concerned users when an incident is assigned or its status is updated.	System, Employee, IT Support Staff, Cloud Engineer	Medium
FR-05	The system shall control incident status transitions through a defined Blueprint workflow.	IT Support Staff, Cloud Engineer	High
FR-06	The system shall allow users to submit change requests containing the change description, risk score, implementation plan and rollback plan.	Employee, Cloud Engineer	High
FR-07	The system shall route submitted change requests to the approving manager for review and allow approval or rejection.	Approving Manager	High
FR-08	The system shall display dashboards and reports showing incident status, pending change requests and other relevant operational information.	Administrator, Approving Manager, IT Support Staff	Medium
FR-09	The system shall restrict access to application records and functions according to the permissions assigned to each user role.	All Roles, Administrator	High
FR-10	The system shall allow authorized users to search, filter and view incident and change request records based on available fields and status.	Administrator, Approving Manager, IT Support Staff, Cloud Engineer	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	The application shall load forms, reports and dashboards within 5 seconds under normal testing conditions with up to 500 sample records.
NFR-02	Security	Users shall access only the data and functions permitted by their assigned roles.
NFR-03	Usability	The application shall provide a user-friendly interface accessible through supported web browsers and the Zoho Creator mobile application.

NFR ID	Category	Requirement
NFR-04	Availability	The application shall remain accessible during organizational working hours, except during scheduled maintenance or Zoho service interruptions.
NFR-05	Maintainability	Forms, reports and workflows shall be modifiable within Zoho Creator without requiring complete redevelopment of the application.
NFR-06	Auditability	Changes to important incident and change request records shall be traceable through available audit logs or record history features.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not used
Other Zoho services	Zoho Desk
Mobile	Zoho Creator mobile app (Android / iOS)
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

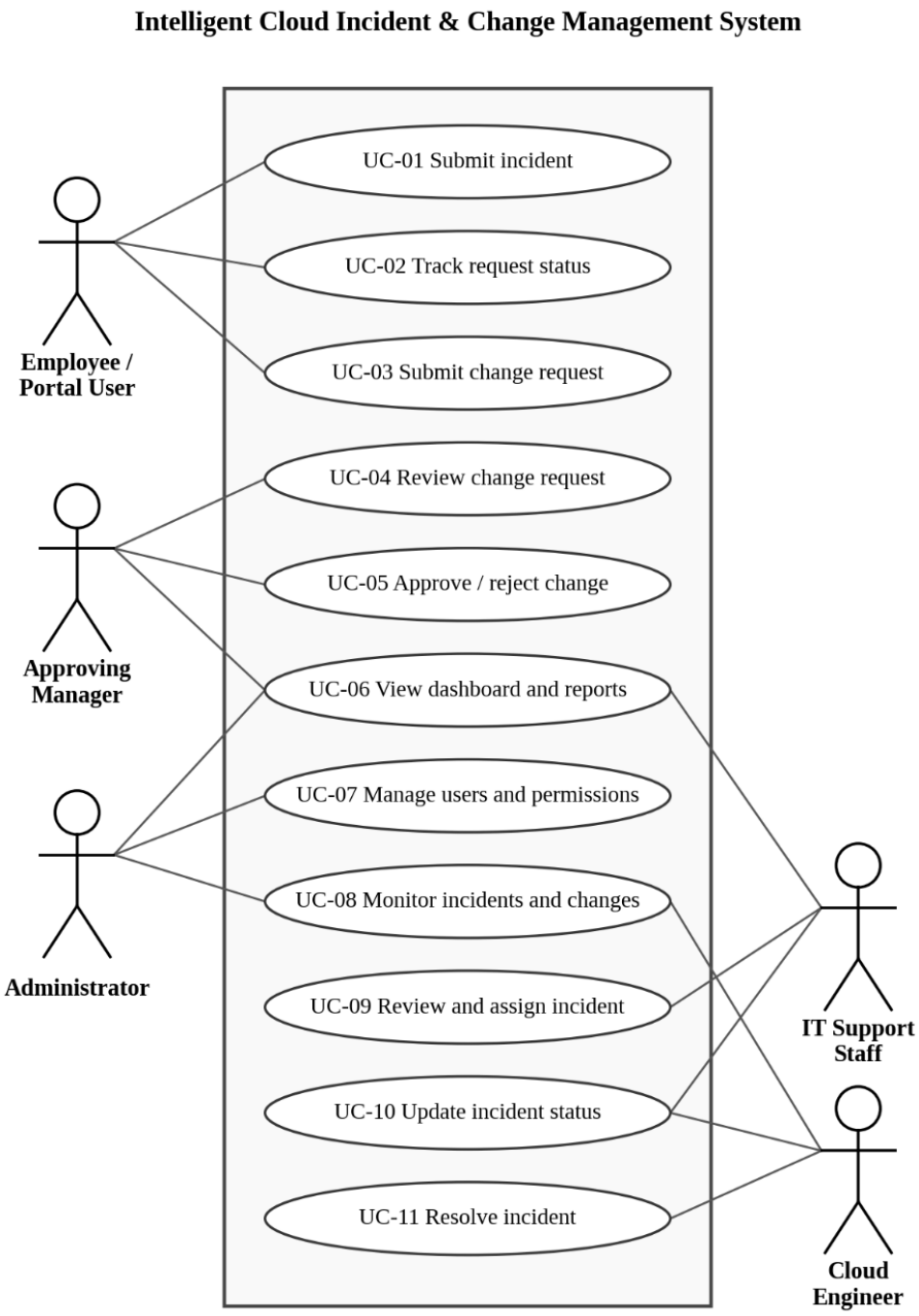


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Submit Incident

Use Case ID	UC-01
Use Case Name	Submit Incident
Actor(s)	Employee / Portal User
Pre-condition	The employee must have access to the application.
Main Flow	1. Employee opens the incident submission form. 2. Enters the incident description, category and affected service. 3. Submits the form. 4. System validates the mandatory fields. 5. System calculates priority and SLA deadline according to defined rules. 6. Incident details are saved.
Alternate Flow	If mandatory fields are missing or invalid, the system displays an error message and requests correction.
Post-condition	The incident is successfully registered and available for further processing.
Related FR	FR-01, FR-02

Table 3.6: Use Case Description – Approve / Reject Change

Use Case ID	UC-05
Use Case Name	Approve / Reject Change
Actor(s)	Approving Manager
Pre-condition	A change request must be submitted and available for managerial review.
Main Flow	1. Manager opens the pending change request. 2. Reviews the change description, risk score and implementation plan. 3. Evaluates the request. 4. Selects Approve or Reject. 5. System updates the decision and request status.
Alternate Flow	If the request requires additional information, the manager returns it for clarification instead of approving or rejecting it, if this option is configured.
Post-condition	The change request is updated with the manager's decision.
Related FR	FR-06, FR-07

Table 3.7: Use Case Description – Review and Assign Incident

Use Case ID	UC-09
Use Case Name	Review and Assign Incident
Actor(s)	IT Support Staff
Pre-condition	An incident must be registered and available for assignment.
Main Flow	1. IT Support Staff opens the incident list. 2. Selects a reported incident. 3. Reviews its description, category and priority. 4. Assigns the incident to an available Cloud Engineer. 5. System updates the assignment details.
Alternate Flow	If no suitable engineer is available, the incident remains pending assignment until an engineer can be assigned.
Post-condition	The incident is assigned to a Cloud Engineer for investigation and resolution.
Related FR	FR-03, FR-04

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The Intelligent Cloud Incident & Change Management System follows a layered architecture using Zoho Creator. The architecture consists of four main layers: User and Access Layer, Application Layer, Data Layer and Integration Layer. Each layer performs specific functions and communicates with the other layers to provide a centralized incident and change management solution.

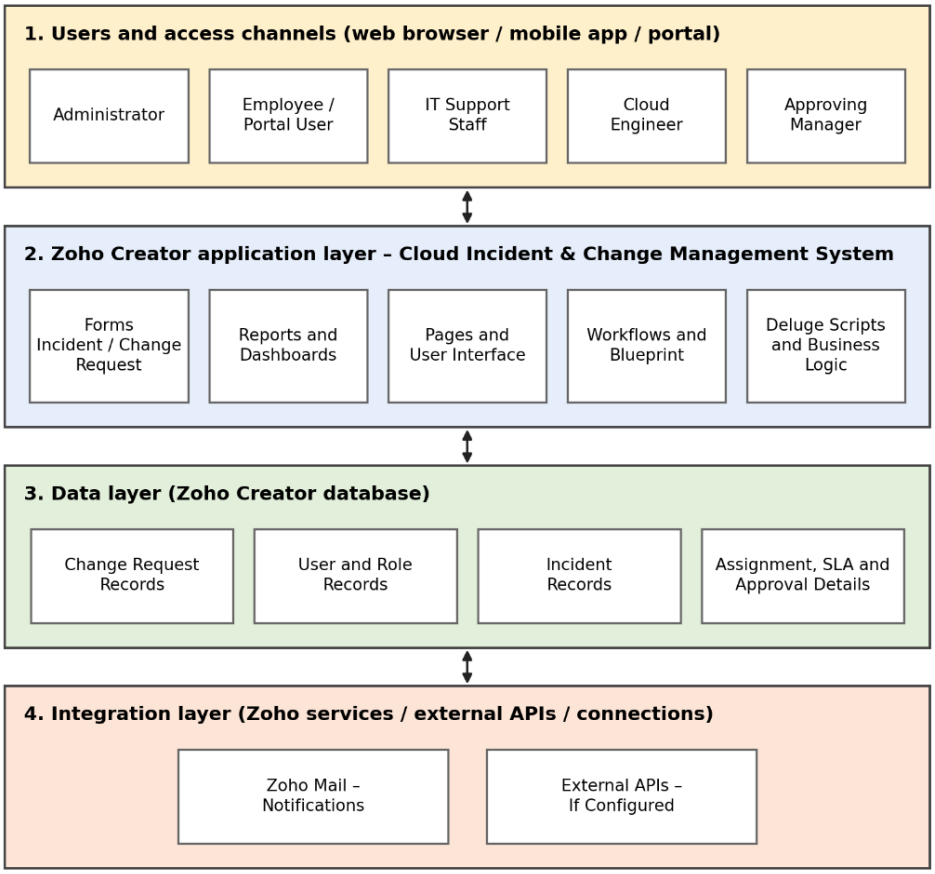


Figure 4.1: System Architecture of the Application

Layer 1 – User and Access Layer

This layer represents the users who interact with the application through supported web browsers, mobile applications or portal access. It includes Employees, IT Support Staff, Cloud Engineers, Approving Managers and Administrators. Each role receives access according to its responsibilities and assigned permissions.

Layer 2 – Application Layer – Zoho Creator

This is the main processing layer of the system. It contains the following components:

- **Forms:** Used to register incidents and submit change requests.
- **Reports and Dashboards:** Display incident details, change request status and operational information.
- **Pages:** Provide the application interface and organize information.
- **Workflows and Blueprint:** Manage automated actions and control incident or approval processes.
- **Deluge Scripts:** Implement custom business logic, validations and application-specific operations.

Layer 3 – Data Layer

The data layer uses Zoho Creator's built-in database to store and manage application records. It maintains user details, incident information, change requests, assignments, SLA details and approval records. The application retrieves and updates these records whenever users perform authorized operations.

Layer 4 – Integration Layer

This layer provides the facility to connect the application with other Zoho services and external systems through supported integrations, APIs and connections. For example, Zoho Mail can be used for email notifications. External API integrations can be added when required and configured.

The layered architecture separates user access, application processing, data storage and external integrations. This structure makes the system easier to manage, maintain and extend while providing a centralized platform for incident and change management.

4.2 Data Model

The data model defines how information is organized, stored and related in the Intelligent Cloud Incident & Change Management System. The proposed model contains three main entities: User, Incident and Change Request. These entities are connected through relationships implemented using Zoho Creator lookup fields. This structure helps maintain data consistency and avoids unnecessary duplication.

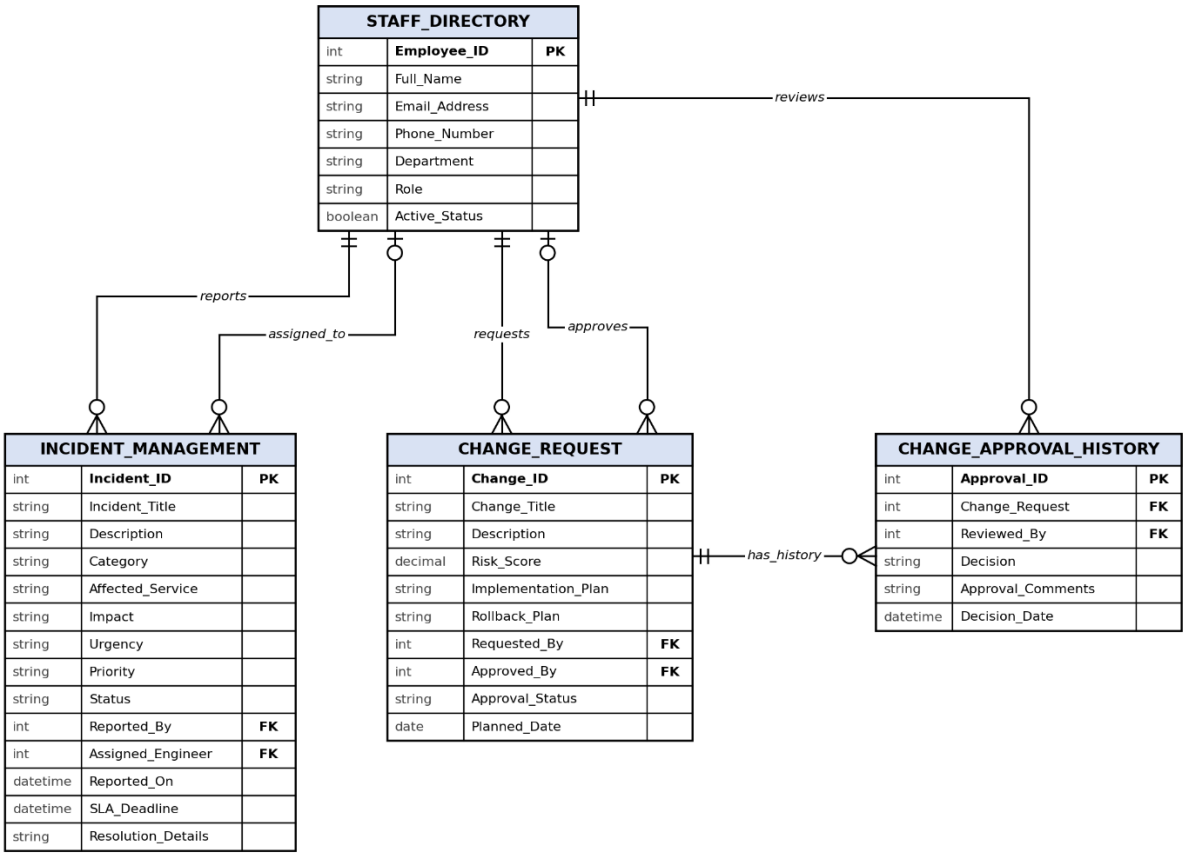


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Staff Directory	Employee_ID, Full_Name, Email_Address, Phone_Number, Department, Role, Active_Status	Incident Management, Change Request, Change Approval History	One-to-Many	Lookup fields
Incident Management	Incident_ID, Incident_Title, Description, Category, Priority, Status, SLA_Deadline	Staff Directory (Reporter and Assigned Engineer)	Many-to-One	Lookup fields
Change Request	Change_ID, Change_Title, Description, Risk_Score, Implementation_Plan, Rollback_Plan, Approval_Status	Staff Directory (Requester and Approver), Change Approval History	Many-to-One / One-to-Many	Lookup fields

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Change Approval History	Approval_ID, Change_Request, Reviewed_By, Decision, Approval_Comments, Decision_Date	Change Request, Staff Directory (Reviewer)	Many-to-One	Lookup fields

4.3 Form Design

The proposed Intelligent Cloud Incident & Change Management System consists of four major forms developed using Zoho Creator. Each form is designed to collect and maintain specific information related to employees, IT incidents, infrastructure change requests and approval activities.

The forms use appropriate field types, mandatory settings, validation rules and default values to maintain data accuracy and consistency. Lookup fields establish relationships between forms, while Auto Number fields generate unique record identifiers. Workflow automation is used to calculate incident priority, SLA deadlines and maintain approval history.

Table 4.2: Field Specification – Staff Directory Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Employee ID	Employee_ID	Auto Number	—	Start index: 1001	Unique employee identifier
Full Name	Full_Name	Single Line	Yes	Cannot be empty	Employee's full name
Email Address	Email_Address	Email	Yes	Valid email format; unique	Employee email address
Phone Number	Phone_Number	Phone	No	Valid phone number	Contact information
Department	Department	Drop Down	Yes	Cloud Engineering / Administration / IT Support / Other	Employee department

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Role	Role	Drop Down	Yes	Employee / IT Support Staff / Cloud Engineer / Approving Manager / Administrator	Defines employee role
Active Status	Active_Status	Decision Box	No	Default: Checked	Indicates employee availability

Table 4.3: Field Specification – Incident Management Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Incident ID	Incident_ID	Auto Number	—	Start index: 1001	Unique incident identifier
Incident Title	Incident_Title	Single Line	Yes	Cannot be empty	Short incident description
Description	Description	Multi Line	Yes	Cannot be empty	Detailed issue description
Category	Category	Drop Down	Yes	Network Issue / Server Failure / Application Error / Database Issue / Security Issue / Other	Incident classification
Affected Service	Affected_Service	Drop Down	Yes	Predefined service choices	Identifies affected service
Impact	Impact	Drop Down	Yes	High / Medium / Low	Measures business impact
Urgency	Urgency	Drop Down	Yes	High / Medium / Low	Indicates urgency of resolution
Priority	Priority	Drop Down	Yes	Calculated using Impact and Urgency	Incident priority
Status	Status	Drop Down	Yes	Default: Open	Tracks incident progress

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Reported By	Reported_By	Lookup	Yes	Lookup: Staff Directory	Identifies reporting employee
Assigned Engineer	Assigned_Engineer	Lookup	No	Lookup: Staff Directory; filtered by Cloud Engineer role	Responsible engineer
Reported On	Reported_On	Date-Time	Yes	Default: Current Date-Time	Incident reporting date and time
SLA Deadline	SLA_Deadline	Date-Time	No	Calculated based on priority	Expected resolution deadline
Resolution Details	Resolution_Details	Multi Line	No	Entered during resolution	Records corrective action
Attachment	Attachment	File Upload	No	Optional file upload	Supporting evidence

Table 4.4: Field Specification – Change Request Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Change ID	Change_ID	Auto Number	—	Start index: 1001	Unique change identifier
Change Title	Change_Title	Single Line	Yes	Cannot be empty	Title of proposed change
Description	Description	Multi Line	Yes	Cannot be empty	Details of the proposed change
Risk Score	Risk_Score	Decimal	Yes	Value between 1 and 10	Measures potential change risk
Implementation Plan	Implementation_Plan	Multi Line	Yes	Cannot be empty	Steps for implementing the change
Rollback Plan	Rollback_Plan	Multi Line	Yes	Cannot be empty	Recovery procedure if change fails

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Requested By	Approved_By	Lookup	Yes	Lookup: Staff Directory	Employee submitting the request
Approved By	Approval_Status	Lookup	No	Lookup: Staff Directory; filtered by Approving Manager role	Authorized approving manager
Approval Status	Approval_Status	Drop Down	Yes	Default: Pending; Pending / Approved / Rejected	Tracks approval decision
Planned Date	Planned_Date	Date	No	Valid planned date	Proposed implementation date

Table 4.5: Field Specification – Change Approval History Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Approval ID	Approval_ID	Auto Number	—	System generated	Unique approval identifier
Change Request	Change_Request	Lookup	Yes	Lookup: Change Request Form; Approval Status = Pending	Identifies the change request
Reviewed By	Reviewed_By	Lookup	Yes	Lookup: Staff Directory; Approving Manager role	Identifies the reviewing manager
Decision	Decision	Drop Down	Yes	Approved / Rejected	Records approval decision
Approval Comments	Approval_Comments	Multi Line	No	Optional comments	Reason for approval or rejection

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Decision Date	Decision_Date	Date-Time	No	Default: Current Date-Time	Date and time of decision

4.4 Process and Workflow Design

The application follows structured processes to manage IT incidents and infrastructure change requests. These processes ensure proper submission, classification, assignment, approval, tracking and resolution of requests.

Incident Management Process Flow

The incident management process begins when an employee reports an IT issue through the Incident Management Form. The incident is classified based on its impact and urgency, and an appropriate priority is determined. A Cloud Engineer is assigned to investigate the issue and perform the necessary corrective actions. The incident details and resolution progress are updated until the issue is resolved and closed.

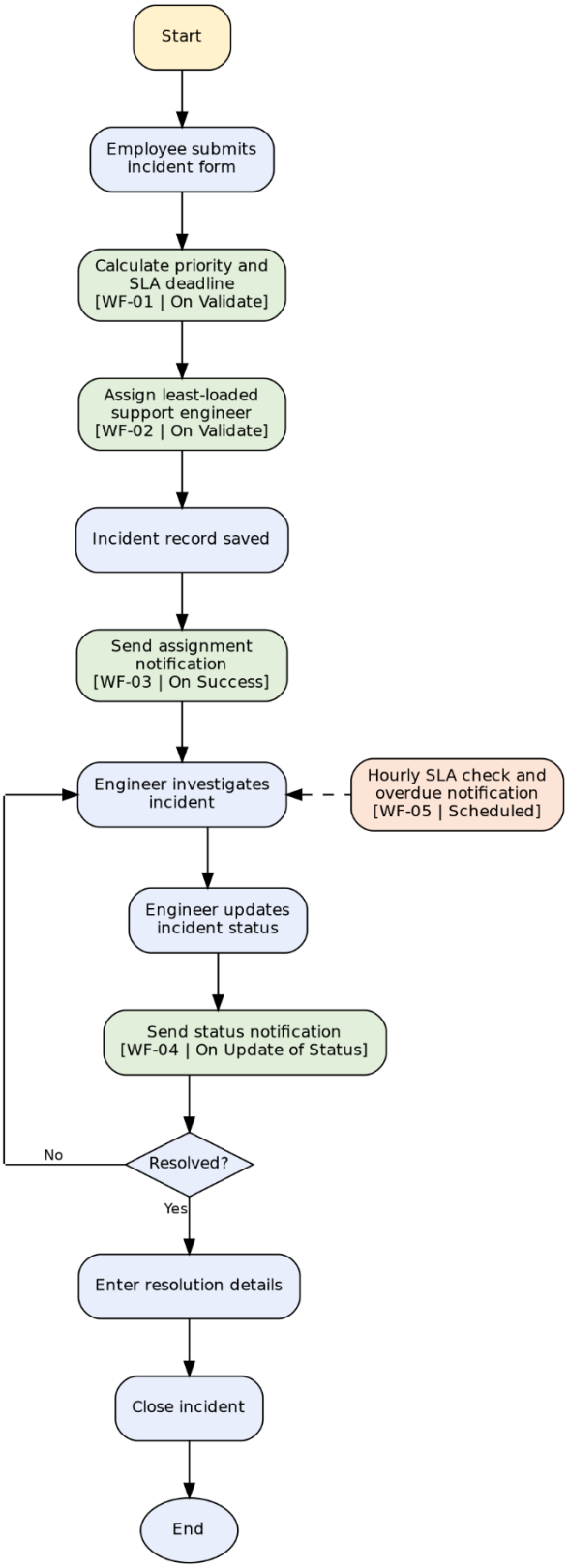


Figure 4.3: Process Flow for Incident Management

Change Request Management Process Flow

The change request management process begins when an employee submits an infrastructure change request with its description, risk score, implementation plan and rollback plan. The request is reviewed by the Approving Manager, who evaluates the associated risks and decides whether to approve or reject it. Approved changes proceed to implementation and verification, while rejected requests are returned to the requester. The approval decision is maintained for future reference.

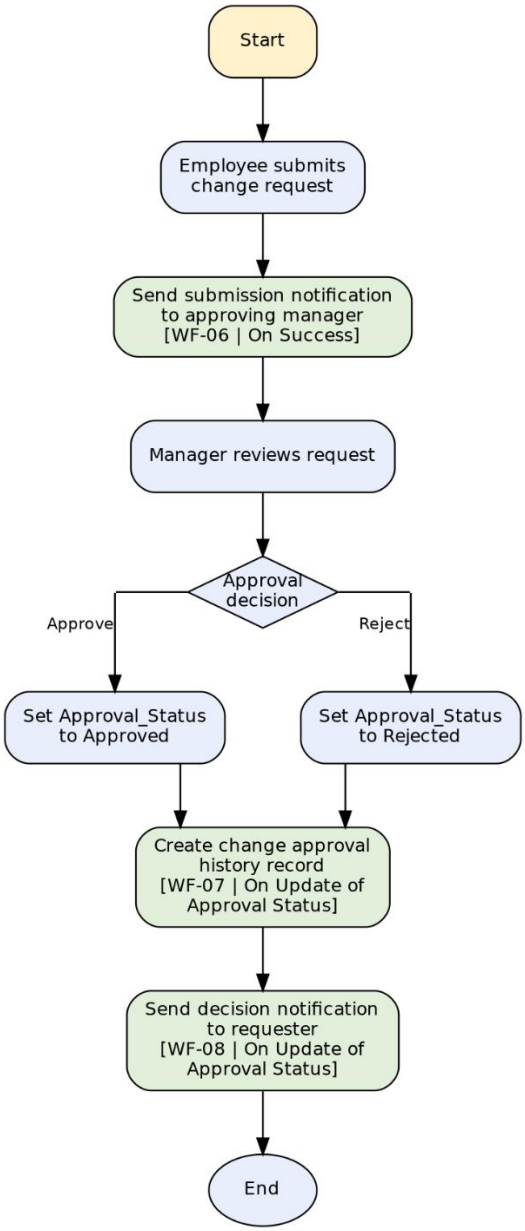


Figure 4.4: Process Flow for Change Request Management

4.5 User Interface and Navigation Design

The application provides a simple and user-friendly interface developed using Zoho Creator. The navigation menu organizes forms and reports into sections such as Staff Directory, Incident Management, Requests and Approval Histories.

Users can easily submit incidents, create change requests, view records and track approval decisions through the respective forms and reports. The dashboard provides a centralized view of incident statistics, SLA monitoring and change request activities using summary panels, charts and reports.

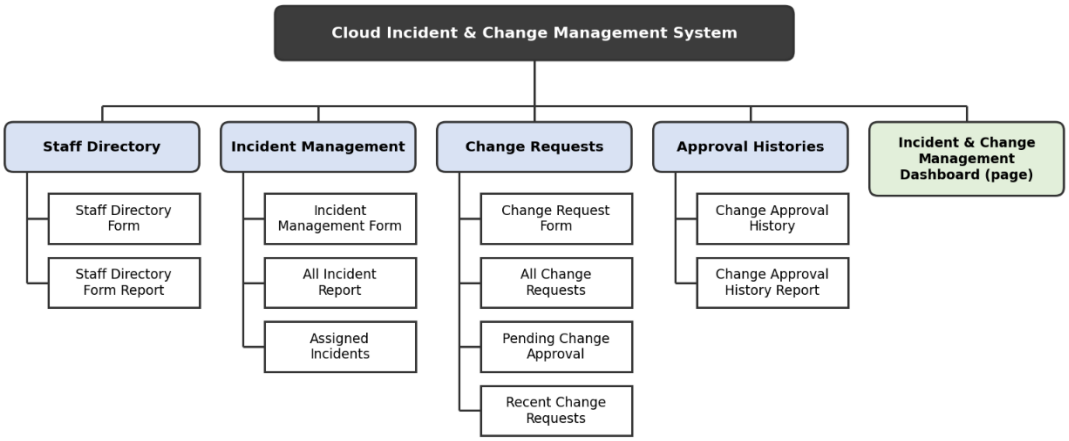


Figure 4.5: Navigation Structure of Cloud Incident & Change Management System

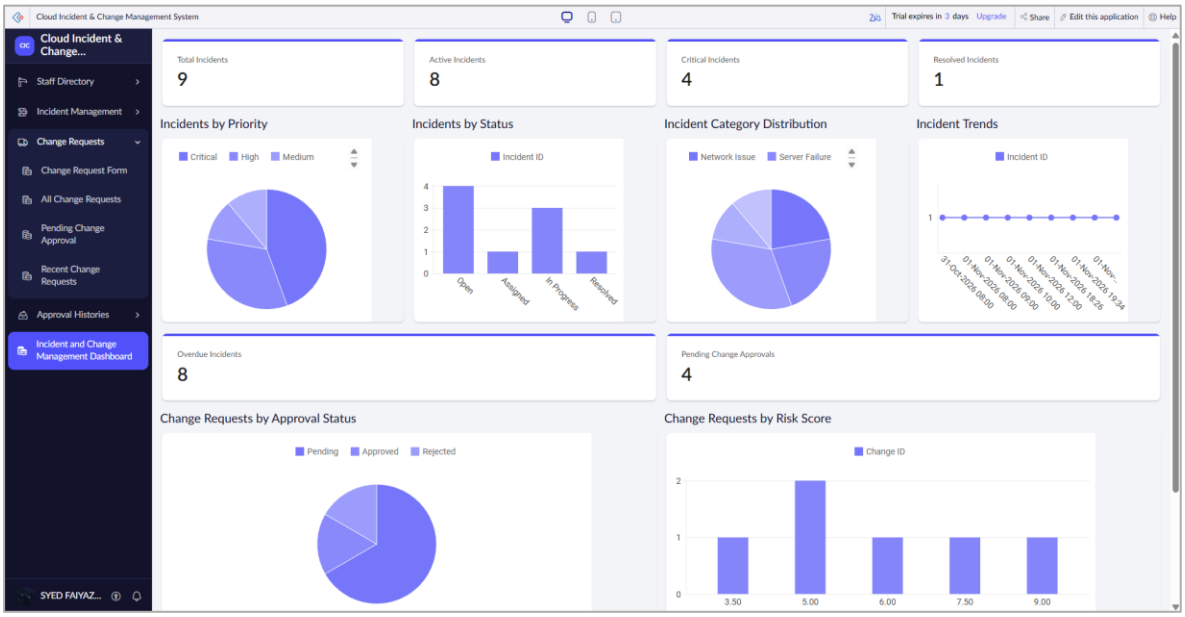


Figure 4.6: Wireframe of Cloud Incident & Change Management System

4.6 Access Control Design

The Intelligent Cloud Incident & Change Management System implements a Role-Based Access Control (RBAC) approach to manage user permissions and protect application data. Different users are provided with access based on their assigned roles and responsibilities. The application defines five user roles: Administrator, Approving Manager, IT Support Staff, Cloud Engineer and Employee/Portal User. Each role is assigned specific permissions to add, edit, view and delete records according to its operational requirements.

The Administrator manages the overall application, while the Approving Manager handles change request reviews and approval activities. IT Support Staff manage reported incidents, Cloud Engineers investigate and resolve assigned incidents, and Employees submit incidents and infrastructure change requests.

Table 4.6: Role–Permission Matrix

Component	Administrator	Approving Manager	IT Support Staff	Cloud Engineer	Employee / Portal User
Form: Staff Directory	A E V D	—	—	—	—
Form: Incident Management	A E V D	V	A E V	E V (assigned)	A V (own)
Form: Change Request	A E V D	E V	A V (own)	V (approved requests)	A V (own)
Form: Change Approval History	A E V D	V	—	—	—
Report: Staff Directory	V	—	—	—	—
Report: All Incidents	V	V	V	V (assigned)	V (own)
Report: Assigned Incidents	V	V	V	V (assigned)	—
Report: All Change Requests	V	V	V (own)	—	V (own)
Report: Pending Change Approvals	V	V	—	—	—
Report: Change Approval History	V	V	—	—	—
Page: Incident & Change Dashboard	V	V	V	V (operational data)	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application titled **Intelligent Cloud Incident & Change Management System** was created from scratch using Zoho Creator without using a predefined template. The application follows the data model designed in Chapter 4, with four main forms and their associated reports.

The application is configured in the Indian data centre (creatorapp.zoho.in) with the time zone set to **Asia/Kolkata**, date format set to **dd-MMM-yyyy** and time format set to **24-hour**. These settings ensure consistent date and time representation, particularly for incident reporting and SLA deadlines.

The navigation menu is organized into four main sections: **Staff Directory Form, Incident Management Form, Requests and Approval Histories**. Each section contains its respective forms and reports. A centralized dashboard was also created using Zia assistance to display incident statistics, SLA information and change request details.

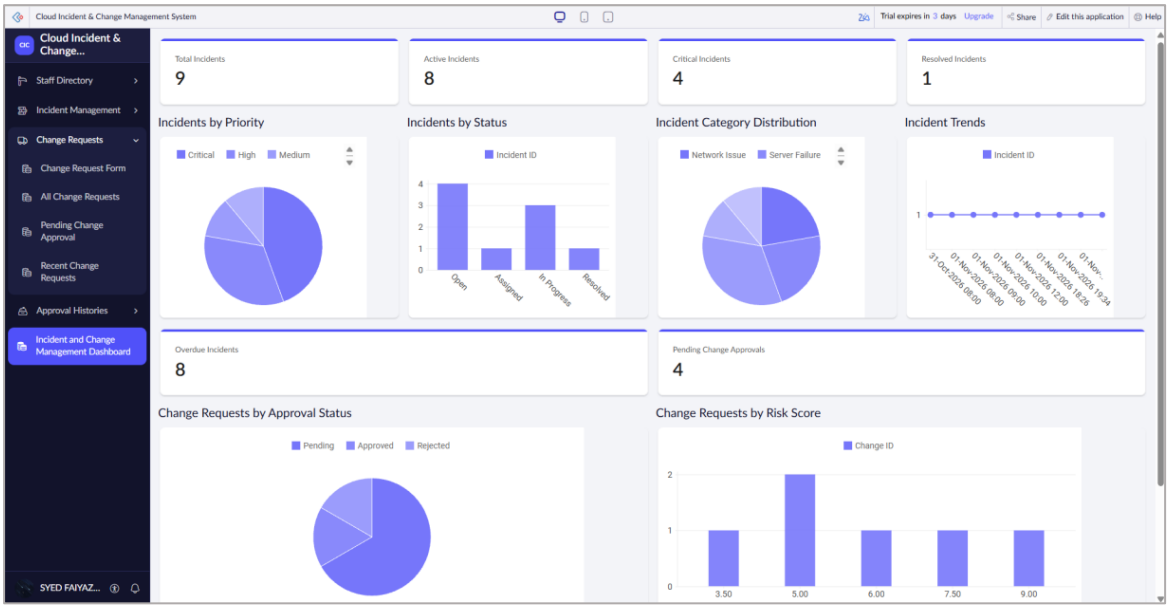


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Four forms are developed in Zoho Creator to collect and manage employee details, IT incidents, infrastructure change requests and approval records. Each form uses appropriate field types, mandatory settings and lookup relationships to ensure accurate data collection.

Staff Directory Form

The Staff Directory Form stores employee information, including Employee ID, Full Name, Email Address, Phone Number, Department, Role and Active Status. Auto Number generates employee identifiers, while mandatory fields ensure that essential employee details are provided.

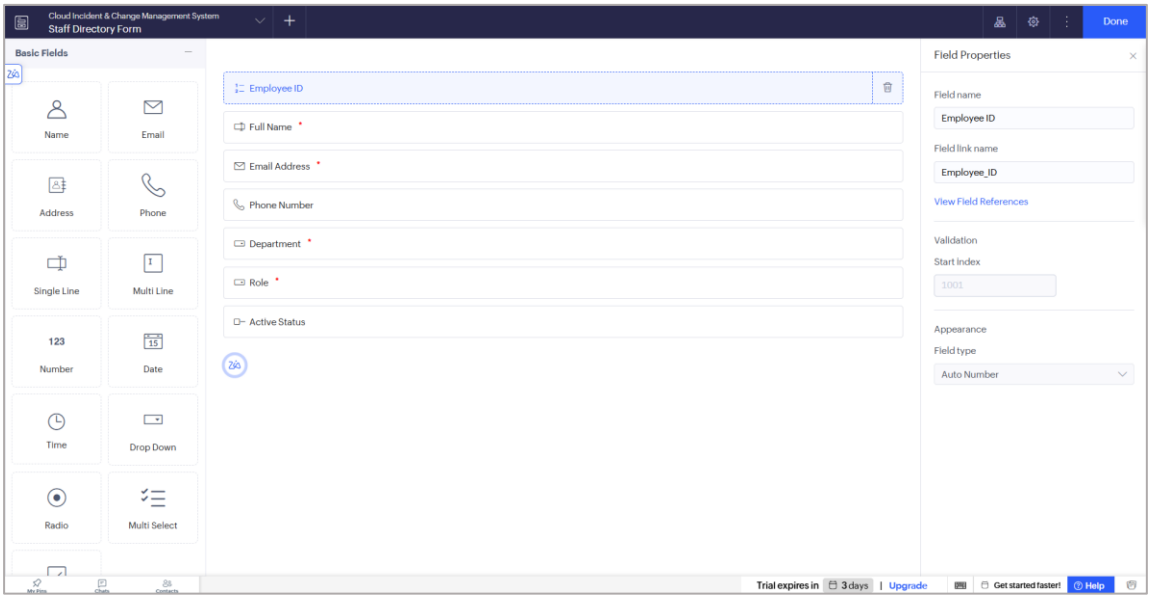


Figure 5.2: Staff Directory Form in the Form Builder

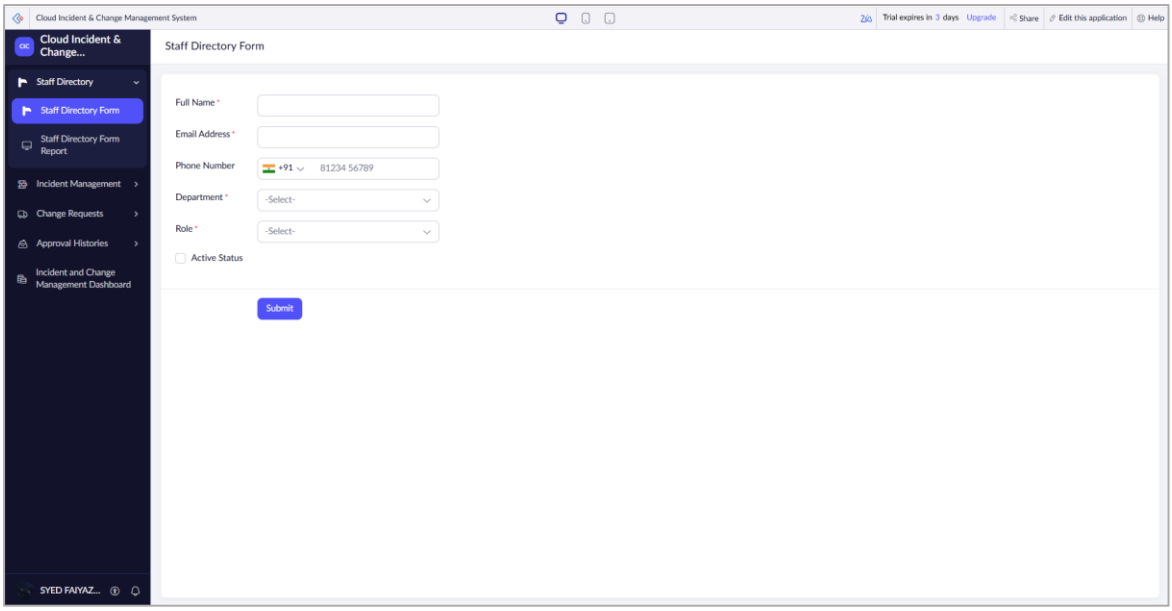


Figure 5.3: Staff Directory Form in Live Mode

Incident Management Form

The Incident Management Form captures incident details, classification and assignment information. It uses lookup-based employee selection for Reported By and Assigned Engineer. Priority and SLA Deadline are populated automatically through Deluge scripting based on Impact and Urgency.

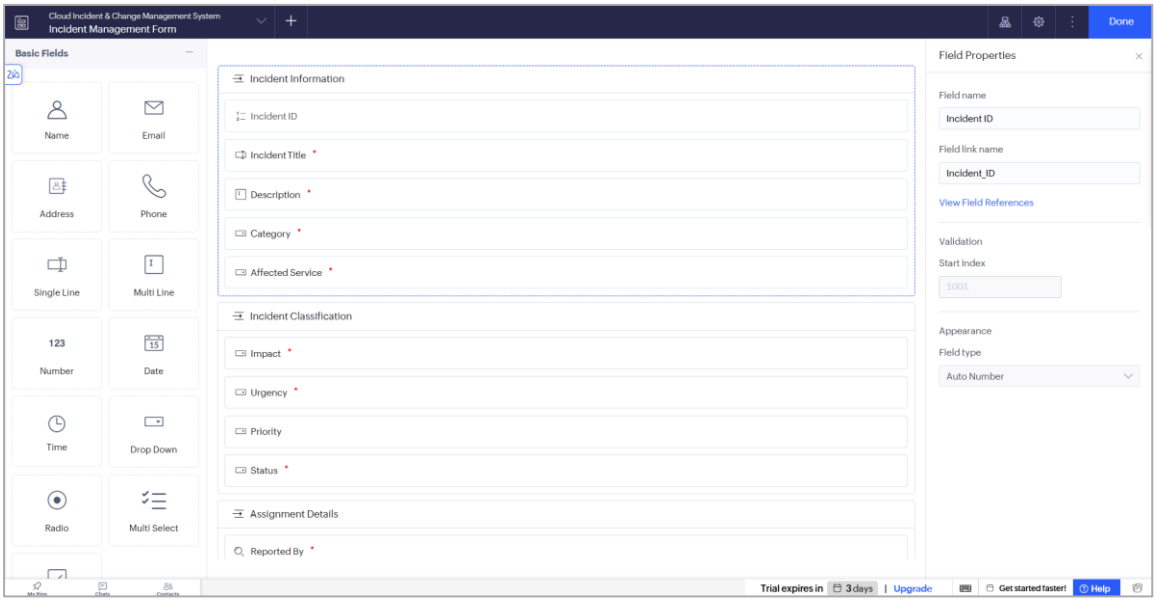


Figure 5.4: Incident Management Form in the Form Builder

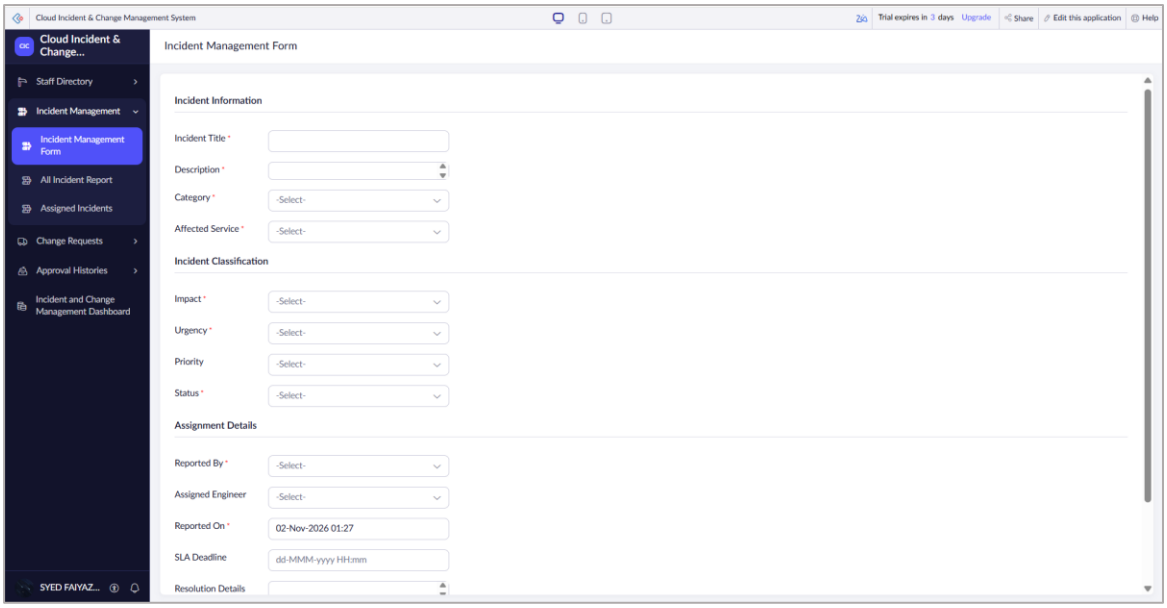


Figure 5.5: Incident Management Form in Live Mode

Change Request Form

The Change Request Form collects change details, risk score, implementation plan and rollback plan. Lookup fields connect the requester and approving manager to the Staff Directory. Approval Status is used to track the progress of each request.

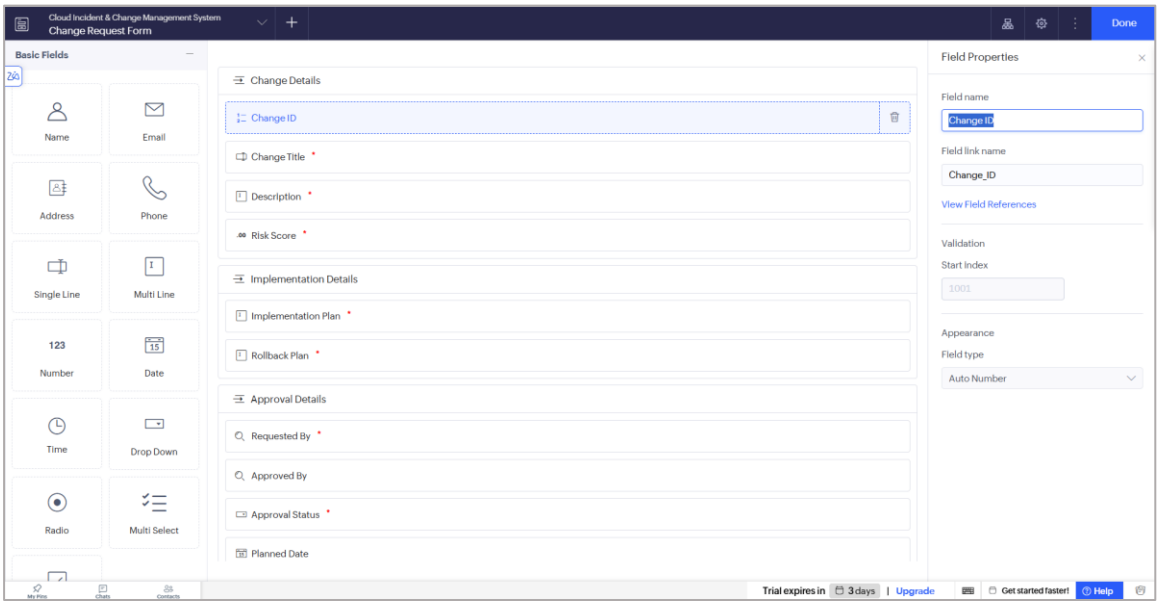


Figure 5.6: Change Request Form in the Form Builder

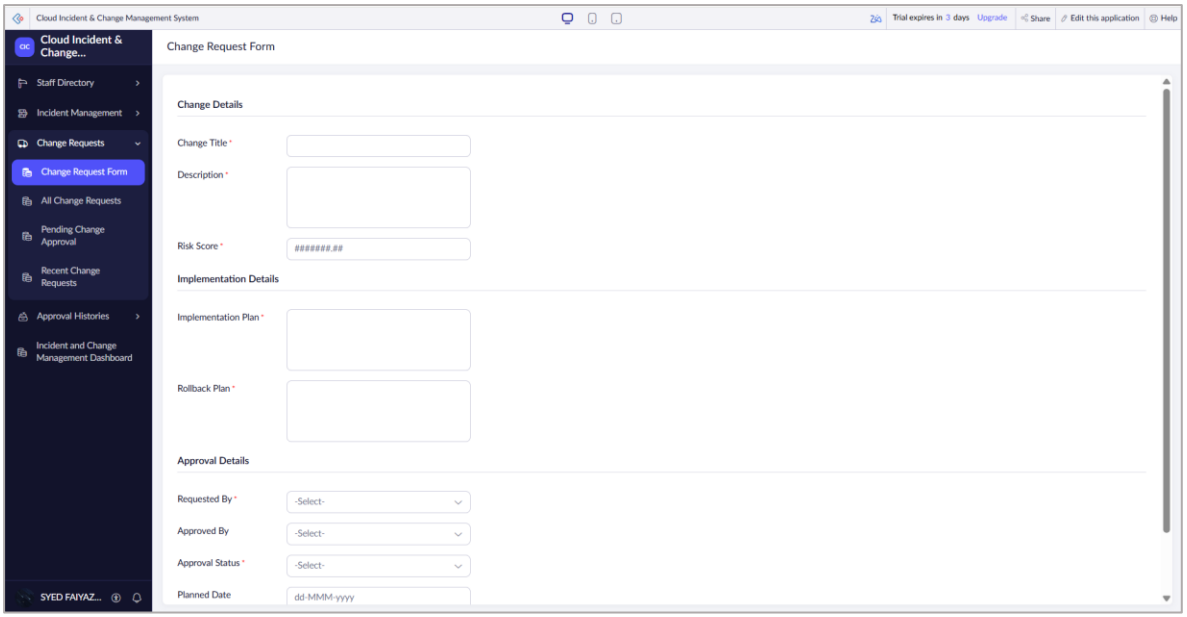


Figure 5.7: Change Request Form in Live Mode

Change Approval History Form

The Change Approval History Form maintains records of approval decisions. It links each decision to its corresponding change request and records the reviewing manager, decision, comments and decision date.

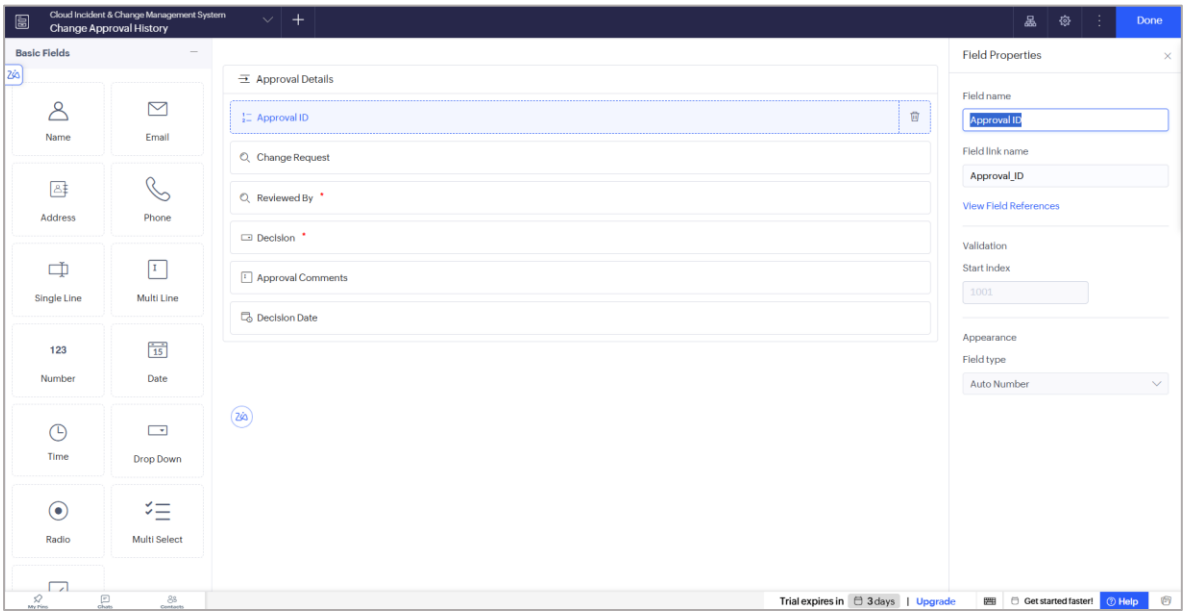


Figure 5.8: Change Approval History Form in the Form Builder

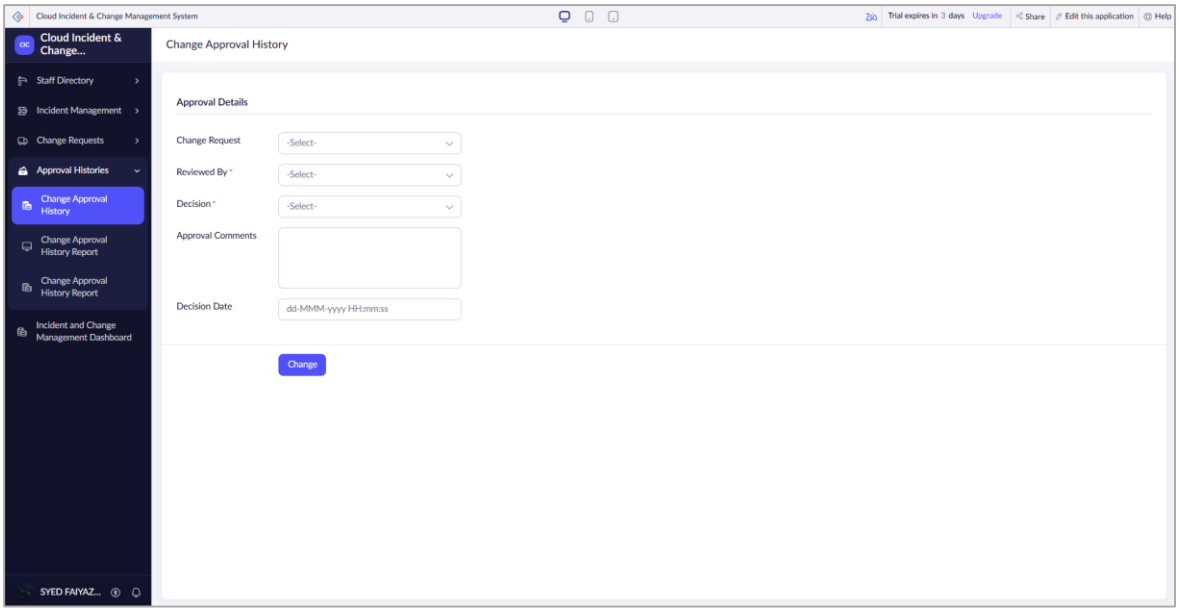


Figure 5.9: Change Approval History Form in Live Mode

All four forms use mandatory field validation where required. Lookup fields establish relationships between forms, while Auto Number fields generate unique identifiers. Form success messages are configured to confirm successful submission. Edit operations provide Update and Cancel options. Section headings organize related fields for easier data entry.

5.3 Reports

Reports are developed in Zoho Creator to display, organize and monitor records collected through the application forms. Six List Reports are configured to manage employee information, incidents, change requests and approval history. Filters and sorting options are applied wherever required to simplify record tracking and retrieval.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Staff Directory Form Report	List	Staff Directory Form	All records; no sorting	Edit, Duplicate, Delete	Administrator
All Incident Report	List	Incident Management Form	All records; Reported On – Descending	Edit, Duplicate, Delete	Administrator, IT Support Staff

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Assigned Incidents	List	Incident Management Form	Assigned Engineer is not empty; Status is not Resolved or Closed	Edit, Duplicate, Delete	Administrator, IT Support Staff, Cloud Engineer
All Change Requests	List	Change Request Form	All records; Planned Date – Descending	Edit, Duplicate, Delete	Administrator, Approving Manager
Pending Change Approval	List	Change Request Form	Approval Status = Pending; Planned Date – Ascending	Edit, Duplicate, Delete	Administrator, Approving Manager
Change Approval History Report	List	Change Approval History	All records; Decision Date – Descending	Edit, Duplicate, Delete	Administrator, Approving Manager

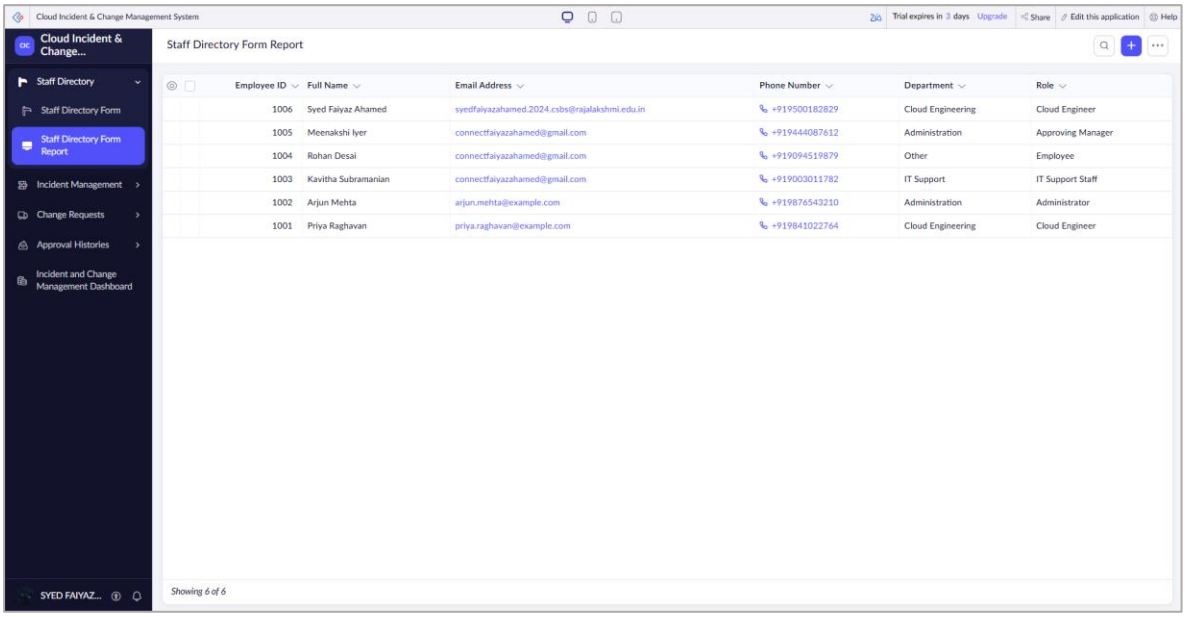
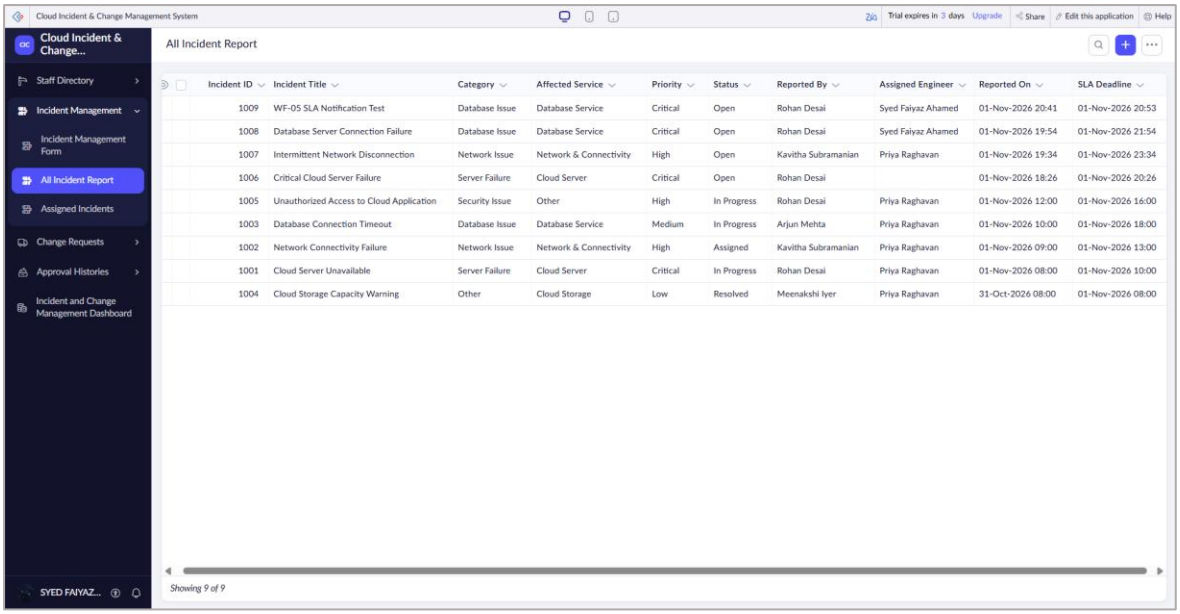
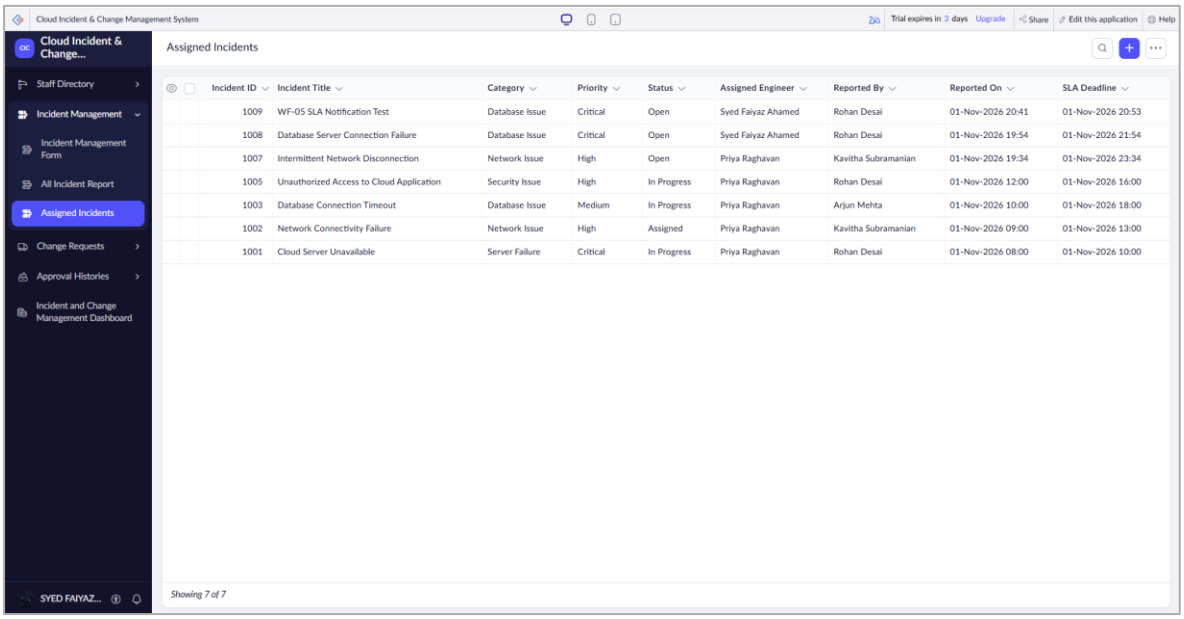


Figure 5.10: Staff Directory Report



Incident ID	Incident Title	Category	Affected Service	Priority	Status	Reported By	Assigned Engineer	Reported On	SLA Deadline
1009	WF-05 SLA Notification Test	Database Issue	Database Service	Critical	Open	Rohan Desai	Syed Faiyaz Ahamed	01-Nov-2026 20:41	01-Nov-2026 20:53
1008	Database Server Connection Failure	Database Issue	Database Service	Critical	Open	Rohan Desai	Syed Faiyaz Ahamed	01-Nov-2026 19:54	01-Nov-2026 21:54
1007	Intermittent Network Disconnection	Network Issue	Network & Connectivity	High	Open	Kavitha Subramanian	Priya Raghavan	01-Nov-2026 19:34	01-Nov-2026 23:34
1006	Critical Cloud Server Failure	Server Failure	Cloud Server	Critical	Open	Rohan Desai		01-Nov-2026 18:26	01-Nov-2026 20:26
1005	Unauthorized Access to Cloud Application	Security Issue	Other	High	In Progress	Rohan Desai	Priya Raghavan	01-Nov-2026 12:00	01-Nov-2026 16:00
1003	Database Connection Timeout	Database Issue	Database Service	Medium	In Progress	Arjun Mehta	Priya Raghavan	01-Nov-2026 10:00	01-Nov-2026 18:00
1002	Network Connectivity Failure	Network Issue	Network & Connectivity	High	Assigned	Kavitha Subramanian	Priya Raghavan	01-Nov-2026 09:00	01-Nov-2026 13:00
1001	Cloud Server Unavailable	Server Failure	Cloud Server	Critical	In Progress	Rohan Desai	Priya Raghavan	01-Nov-2026 08:00	01-Nov-2026 10:00
1004	Cloud Storage Capacity Warning	Other	Cloud Storage	Low	Resolved	Meenakshi Iyer	Priya Raghavan	31-Oct-2026 08:00	01-Nov-2026 08:00

Figure 5.11: All Incidents Report



Incident ID	Incident Title	Category	Priority	Status	Assigned Engineer	Reported By	Reported On	SLA Deadline
1009	WF-05 SLA Notification Test	Database Issue	Critical	Open	Syed Faiyaz Ahamed	Rohan Desai	01-Nov-2026 20:41	01-Nov-2026 20:53
1008	Database Server Connection Failure	Database Issue	Critical	Open	Syed Faiyaz Ahamed	Rohan Desai	01-Nov-2026 19:54	01-Nov-2026 21:54
1007	Intermittent Network Disconnection	Network Issue	High	Open	Priya Raghavan	Kavitha Subramanian	01-Nov-2026 19:34	01-Nov-2026 23:34
1005	Unauthorized Access to Cloud Application	Security Issue	High	In Progress	Priya Raghavan	Rohan Desai	01-Nov-2026 12:00	01-Nov-2026 16:00
1003	Database Connection Timeout	Database Issue	Medium	In Progress	Priya Raghavan	Arjun Mehta	01-Nov-2026 10:00	01-Nov-2026 18:00
1002	Network Connectivity Failure	Network Issue	High	Assigned	Priya Raghavan	Kavitha Subramanian	01-Nov-2026 09:00	01-Nov-2026 13:00
1001	Cloud Server Unavailable	Server Failure	Critical	In Progress	Priya Raghavan	Rohan Desai	01-Nov-2026 08:00	01-Nov-2026 10:00

Figure 5.12: Assigned Incidents Report

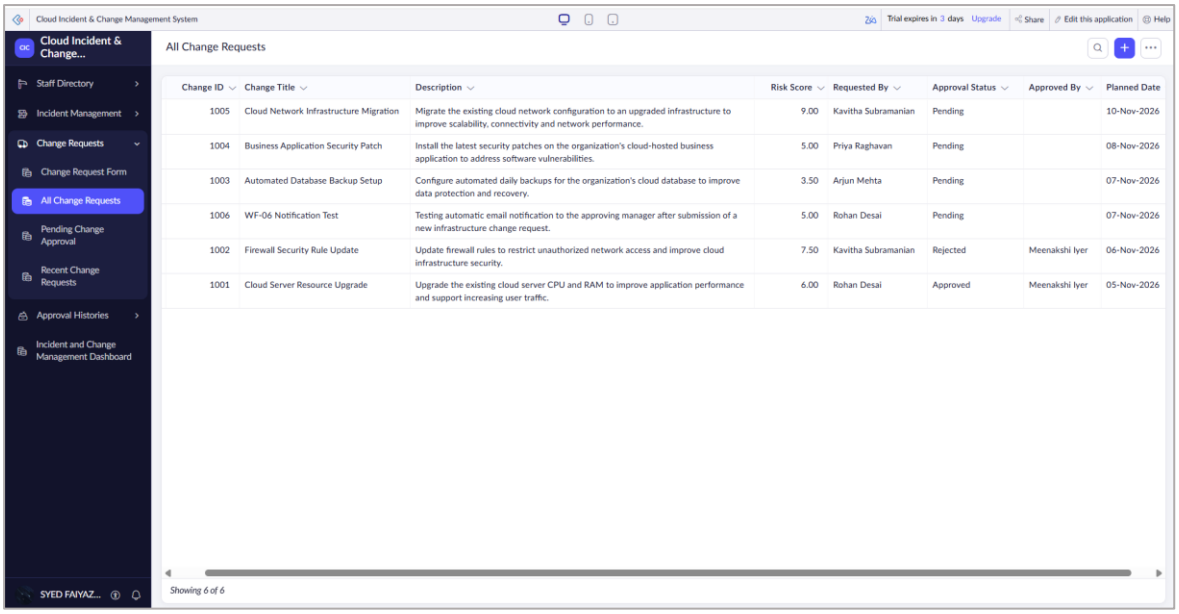


Figure 5.13: All Change Requests Report

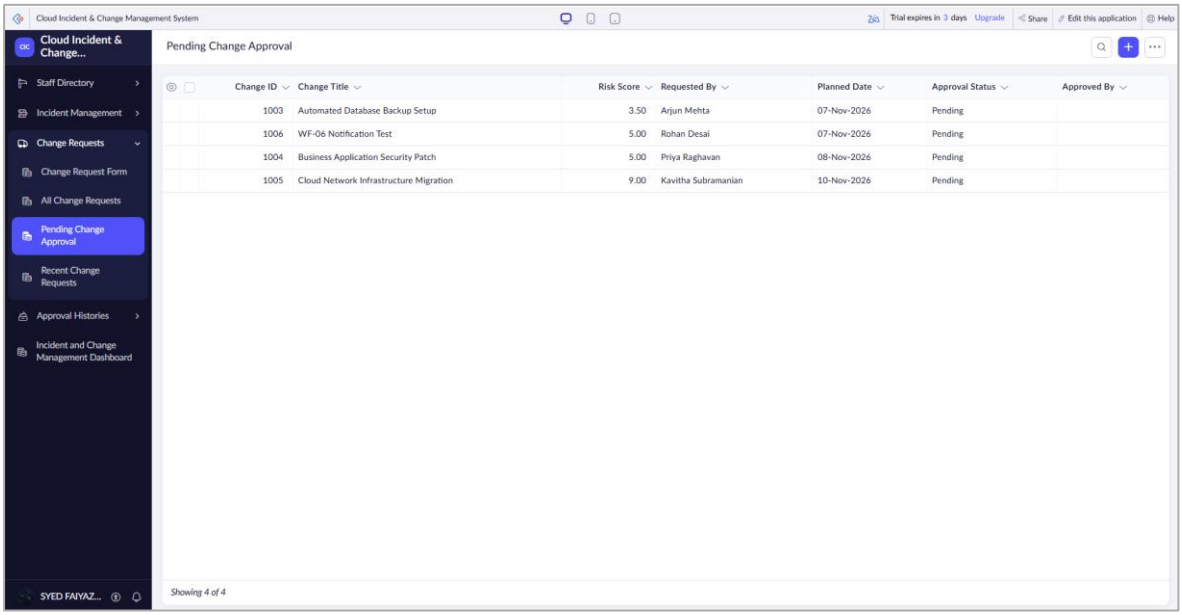


Figure 5.10: Pending Change Approval Report

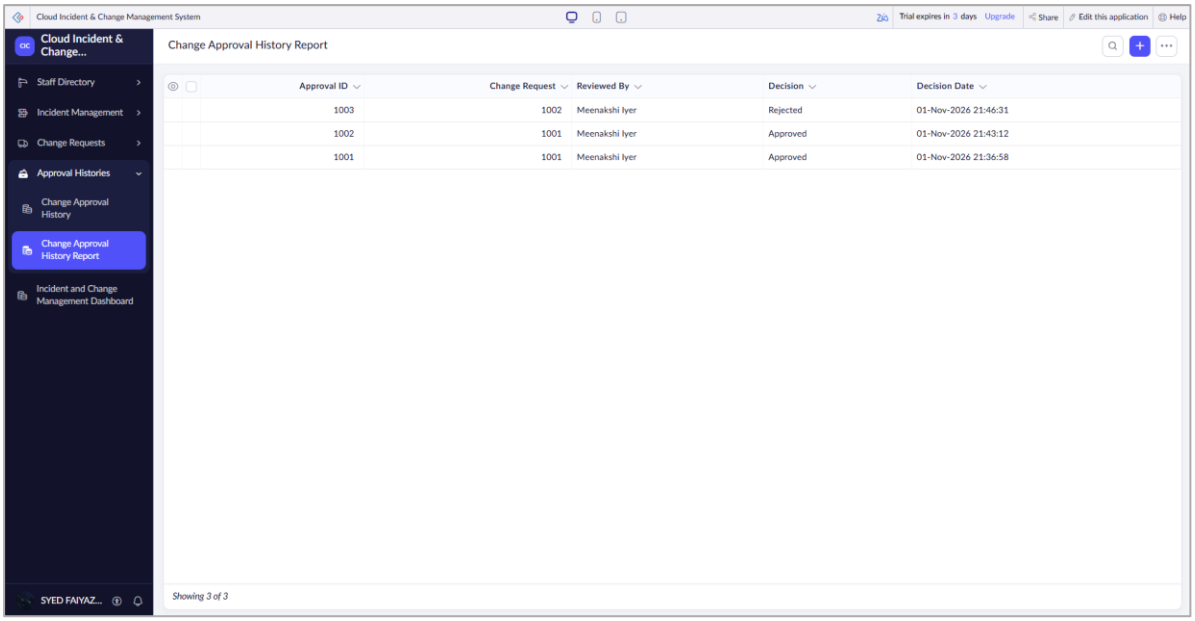


Figure 5.10: Change Approval History Report

5.4 Pages and Dashboards

An **Incident & Change Management Dashboard** is created using the Zoho Creator Page Builder to provide a centralized view of incident and change management activities. It helps users monitor important records, identify pending actions and understand the current status of IT services.

The dashboard is intended for Administrators, IT Support Staff, Cloud Engineers and Approving Managers. It includes summary panels to display key incident statistics, charts to analyse incident priorities, statuses and categories, and information related to SLA monitoring and change request approvals.

The dashboard provides a consolidated view of application activities, making it easier to monitor incidents, track pending change requests and support decision-making. Access permissions for individual roles can be configured separately through the application's permission settings.

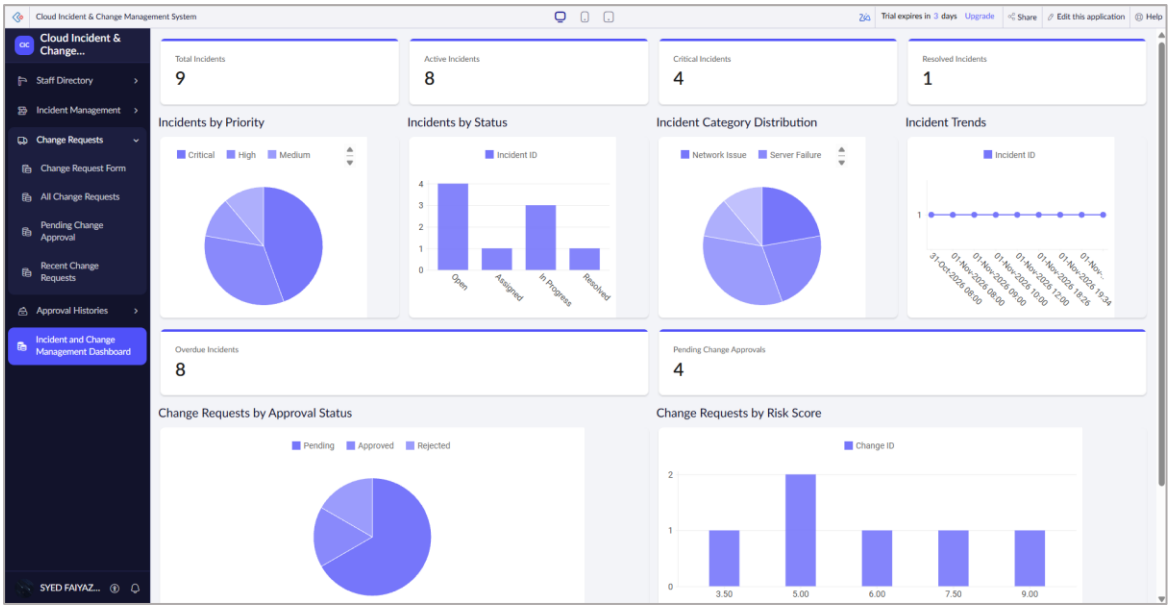


Figure 5.11(a): Incident & Change Management Dashboard

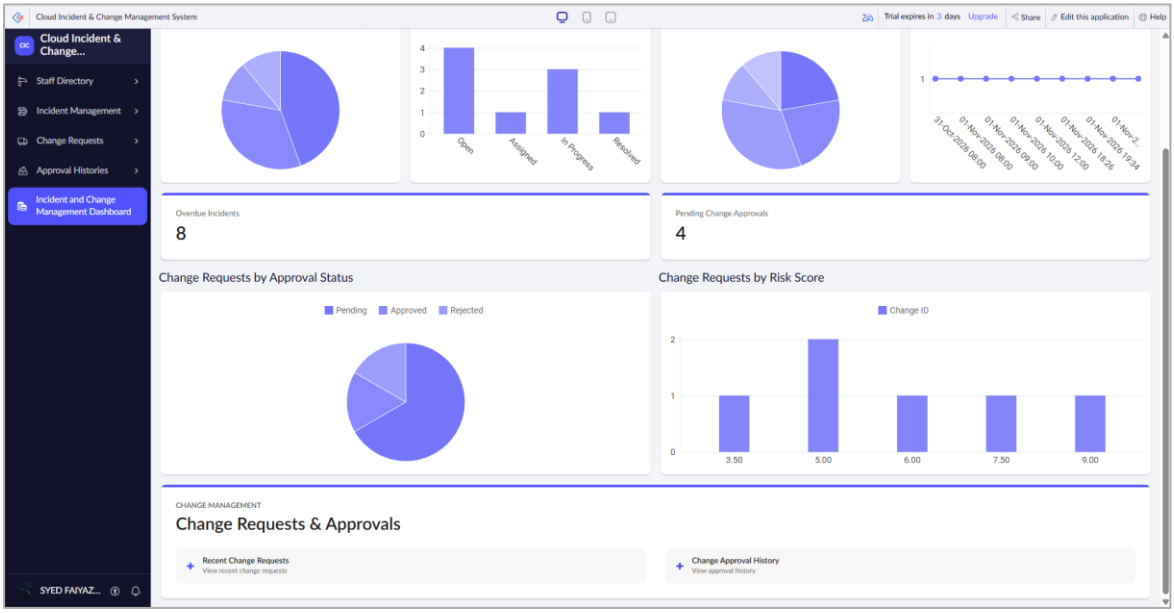


Figure 5.11(b): Incident & Change Management Dashboard

5.5 Deluge Scripting

Deluge scripting is used in the application to automate incident processing, engineer assignment, email notifications, SLA monitoring and change approval activities. The scripts are configured through form workflows and a scheduled workflow in Zoho Creator.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Incident Management Form – On Add, On Validate	Calculate incident priority and SLA deadline based on impact and urgency.	If-else, field assignment, date-time functions
DS-02	Incident Management Form – On Add, On Validate	Assign the incident to an active Cloud Engineer with the least number of unresolved incidents.	Record fetching, loops, count, conditional statements
DS-03	Incident Management Form – On Add, On Success	Send an email notification to the assigned engineer.	Record fetching, sendmail, field access
DS-04	Incident Management Form – On Edit, Status Update	Notify the reporter when the incident status changes.	Record fetching, sendmail, conditional logic
DS-05	Incident Management Form – Scheduled at SLA Deadline	Check overdue incidents and send notifications for incidents that remain unresolved or open.	Schedule workflow, criteria-based fetching, loops, date-time comparison
DS-06	Change Request Form – On Add, On Success	Notify active Approving Managers when a new pending change request is submitted.	Record fetching, for each loop, sendmail
DS-07	Change Request Form – On Edit, Approval Status Update	Create an approval history record when a change request is approved or rejected.	If-else, insert record, date-time function
DS-08	Change Request Form – On Edit, Approval Status Update	Notify the requester about the approval or rejection decision.	Record fetching, conditional statements, sendmail

Code 5.1: Automatic Incident Priority and SLA Calculation

```
// DS-01 | Form: Incident Management Form | Event: On Add, On Validate
// Purpose: Automatically calculate incident priority and SLA deadline
// based on the selected Impact and Urgency values.

if(input.Impact == "High" && input.Urgency == "High")
{
    input.Priority = "Critical";
    input.SLA_Deadline = input.Reported_On.addHour(2);
}
else if((input.Impact == "High" && input.Urgency == "Medium") ||
(input.Impact == "Medium" && input.Urgency == "High"))
{
    input.Priority = "High";
    input.SLA_Deadline = input.Reported_On.addHour(4);
}
else if((input.Impact == "High" && input.Urgency == "Low") ||
(input.Impact == "Medium" && input.Urgency == "Medium") ||
(input.Impact == "Low" && input.Urgency == "High"))
{
    input.Priority = "Medium";
    input.SLA_Deadline = input.Reported_On.addHour(8);
}
else
{
    input.Priority = "Low";
    input.SLA_Deadline = input.Reported_On.addHour(24);
}
```

Code 5.2: Automatic Engineer Assignment

```
// DS-02 | Form: Incident Management Form | Event: On Add, On Validate
// Purpose: Automatically assign the incident to the active Cloud
Engineer
// with the lowest number of unresolved incidents.
// Fetch all active Cloud Engineers

eligibleEngineers = Staff_Directory_Form
[
    Role == "Cloud Engineer" && Active_Status == true
];

// Initialize workload variables
selectedEngineerID = 0;
lowestLoad = 999999;

// Find the engineer with the lowest active incident count
for each engineer in eligibleEngineers
{
    currentLoad = Incident_Management_Form
    [
        Assigned_Engineer == engineer.ID &&
        Status != "Resolved" &&
        Status != "Closed"
    ].count();

    if(currentLoad < lowestLoad)
    {
        lowestLoad = currentLoad;
        selectedEngineerID = engineer.ID;
    }
}
```

```
// Assign the selected engineer
if(selectedEngineerID != 0)
{
    input.Assigned_Engineer = selectedEngineerID;
}
```

Code 5.3: Incident Assignment Notification

```
// DS-03 | Form: Incident Management Form | Event: On Add, On Success
// Purpose: Send an email notification to the assigned Cloud Engineer.
// Check whether an engineer is assigned

if(input.Assigned_Engineer != null)
{
    // Fetch the assigned engineer's staff record
    engineer = Staff_Directory_Form
    [ID == input.Assigned_Engineer];

    // Check whether the engineer's email exists
    if(engineer != null && engineer.Email_Address != null)
    {
        sendmail
        [
            from : zoho.adminuserid
            to : engineer.Email_Address
            subject : "New Incident Assigned - " + input.Incident_ID
            message : "Hello " + engineer.Full_Name + ",\n\n" +
                      "A new incident has been assigned to you.\n\n" +
                      "Incident ID: " + input.Incident_ID + "\n" +
                      "Incident Title: " + input.Incident_Title + "\n"
+
                      "Category: " + input.Category + "\n" +
                      "Affected Service: " + input.Affected_Service +
"\n" +
                      "Priority: " + input.Priority + "\n" +
                      "SLA Deadline: " + input.SLA_Deadline + "\n\n" +
                      "Please investigate and resolve the incident
within the specified SLA deadline.\n\n" +
                      "Regards,\nIT Incident Management System"
        ];
    }
}
```

Code 5.4: Incident Status Notification

```
// DS-04 | Form: Incident Management Form | Event: On Edit, Status
Update
// Purpose: Notify the incident reporter whenever the incident status
changes.
// Check whether a reporter is selected

if(input.Reported_By != null)
{
    // Fetch the reporter's email address and name
    reporterEmail = Staff_Directory_Form
    [ID == input.Reported_By].Email_Address;

    reporterName = Staff_Directory_Form
    [ID == input.Reported_By].Full_Name;

    // Check whether the reporter's email exists
```

```
    if(reporterEmail != null && reporterEmail != "")
    {
        emailBody = "Hello " + reporterName + ",\n\n";
        emailBody = emailBody + "The status of your reported incident
has been updated.\n\n";
        emailBody = emailBody + "INCIDENT DETAILS\n";
        emailBody = emailBody + "Incident ID: " + input.Incident_ID +
"\n";
        emailBody = emailBody + "Incident Title: " +
input.Incident_Title + "\n";
        emailBody = emailBody + "Category: " + input.Category + "\n";
        emailBody = emailBody + "Priority: " + input.Priority + "\n";
        emailBody = emailBody + "Current Status: " + input.Status +
"\n";
        emailBody = emailBody + "SLA Deadline: " + input.SLA_Deadline
+ "\n\n";
        emailBody = emailBody + "Please check the latest incident
updates.\n\n";
        emailBody = emailBody + "Regards,\nIT Incident Management
System";

        sendmail
        [
            from : zoho.adminuserid
            to : reporterEmail
            subject : "Incident Status Updated - " + input.Incident_ID
            message : emailBody
        ];
    }
}
```

Code 5.5: SLA Overdue Notification

```
// DS-05 | Form: Incident Management Form | Event: Scheduled Workflow
// Purpose: Identify unresolved incidents that have exceeded their SLA
deadline
// and send an email notification to the assigned Cloud Engineer.
// Fetch all unresolved, assigned incidents whose SLA deadline has
passed

overdueIncidents = Incident_Management_Form
[
    Status != "Resolved" &&
    Status != "Closed" &&
    Assigned_Engineer != null &&
    SLA_Deadline < zoho.currenttime
];

for each incident in overdueIncidents
{
    // Fetch the assigned engineer's staff record
    engineer = Staff_Directory_Form
    [ID == incident.Assigned_Engineer];

    // Check whether the engineer's email exists
    if(engineer != null && engineer.Email_Address != null &&
engineer.Email_Address != "")
    {
        emailBody = "Hello " + engineer.Full_Name + ",\n\n";
        emailBody = emailBody + "ALERT: INCIDENT SLA DEADLINE
EXCEEDED\n\n";
```

```
        emailBody = emailBody + "The following incident has exceeded
its SLA deadline and is still unresolved.\n\n";
        emailBody = emailBody + "INCIDENT DETAILS\n";
        emailBody = emailBody + "------------------------------\n";
        emailBody = emailBody + "Incident ID: " + incident.Incident_ID
+ "\n";
        emailBody = emailBody + "Incident Title: " +
incident.Incident_Title + "\n";
        emailBody = emailBody + "Category: " + incident.Category +
"\n";
        emailBody = emailBody + "Affected Service: " +
incident.Affected_Service + "\n";
        emailBody = emailBody + "Priority: " + incident.Priority +
"\n";
        emailBody = emailBody + "Current Status: " + incident.Status +
"\n";
        emailBody = emailBody + "SLA Deadline: " +
incident.SLA_Deadline + "\n\n";
        emailBody = emailBody + "ACTION REQUIRED\n";
        emailBody = emailBody + "Please investigate this incident
immediately and update its status.\n\n";
        emailBody = emailBody + "Regards,\n";
        emailBody = emailBody + "IT Incident Management System";

        sendmail
        [
            from : zoho.adminuserid
            to : engineer.Email_Address
            subject : "SLA Breached - Incident " +
incident.Incident_ID
            message : emailBody
        ];
    }
}
```

Code 5.6: Change Request Submission Notification

```
// DS-06 | Form: Change Request Form | Event: On Add, On Success
// Purpose: Notify all active Approving Managers when a new change
request is submitted.
// Fetch all active approving managers

managers = Staff_Directory_Form
[
    Role == "Approving Manager" &&
    Active_Status == true
];

// Fetch requester name
requesterName = "Not specified";

if(input.Requested_By != null)
{
    requesterName = Staff_Directory_Form
    [ID == input.Requested_By].Full_Name;
}

// Fetch change request details
changeID = input.Change_ID.toString();
changeTitle = input.Change_Title;
riskScore = input.Risk_Score.toString();
plannedDate = input.Planned_Date.toString();
```

```
// Send notification to active managers
for each manager in managers
{
    if(manager.Email_Address != null && manager.Email_Address != "")
    {
        // Prepare email message
        emailBody = "Hello " + manager.Full_Name + ",\n\n";
        emailBody = emailBody + "A new infrastructure change request
has been submitted and requires your review.\n\n";
        emailBody = emailBody + "CHANGE REQUEST DETAILS\n";
        emailBody = emailBody + "------------------------------\n";
        emailBody = emailBody + "Change ID: " + changeID + "\n";
        emailBody = emailBody + "Change Title: " + changeTitle + "\n";
        emailBody = emailBody + "Risk Score: " + riskScore + "\n";
        emailBody = emailBody + "Requested By: " + requesterName +
"\n";
        emailBody = emailBody + "Approval Status: " +
input.Approval_Status + "\n";
        emailBody = emailBody + "Planned Date: " + plannedDate +
"\n\n";
        emailBody = emailBody + "Please review the change request and
provide your approval decision.\n\n";
        emailBody = emailBody + "Regards,\n";
        emailBody = emailBody + "IT Change Management System";

        // Send email notification
        sendmail
        [
            from : zoho.adminuserid
            to : manager.Email_Address
            subject : "New Change Request - " + changeID
            message : emailBody
        ];
    }
}
```

Code 5.7: Change Approval History Automation

```
// DS-07 | Form: Change Request Form | Event: On Edit, Approval Status
Update
// Purpose: Automatically create an approval history record
// when a change request is approved or rejected.
// Check whether a final approval decision is made

if(input.Approval_Status == "Approved" || input.Approval_Status ==
"Rejected")
{
    // Check whether approving manager is selected
    if(input.Approved_By != null)
    {
        // Automatically create approval history record
        newApproval = insert into Change_Approval_History
        [
            Change_Request = input.ID
            Reviewed_By = input.Approved_By
            Decision = input.Approval_Status
            Decision_Date = zoho.currenttime
            Added_User = zoho.loginuser
        ];

        // Display created record ID in execution logs
        info "Approval History Created Successfully";
```

```
        info newApproval;
    }
    else
    {
        info "Please select an Approving Manager.";
    }
}
```

Code 5.8: Change Decision Notification

```
// DS-08 | Form: Change Request Form | Event: On Edit, Approval Status
Update
// Purpose: Notify the requester when a change request is approved or
rejected.
// Check whether a final decision has been made

if(input.Approval_Status == "Approved" || input.Approval_Status ==
"Rejected")
{
    // Fetch requester email from Staff Directory
    requesterEmail = Staff_Directory_Form
    [ID == input.Requested_By].Email_Address;

    // Fetch requester name
    requesterName = Staff_Directory_Form
    [ID == input.Requested_By].Full_Name;

    // Fetch approving manager name
    managerName = "Approving Manager";

    if(input.Approved_By != null)
    {
        managerName = Staff_Directory_Form
        [ID == input.Approved_By].Full_Name;
    }

    // Verify requester email
    if(requesterEmail != null && requesterEmail != "")
    {
        // Prepare email message
        emailBody = "Hello " + requesterName + ",\n\n";
        emailBody = emailBody + "Your infrastructure change request
has been reviewed.\n\n";
        emailBody = emailBody + "CHANGE REQUEST DETAILS\n";
        emailBody = emailBody + "------------------------------\n";
        emailBody = emailBody + "Change ID: " + input.Change_ID +
"\n";
        emailBody = emailBody + "Change Title: " + input.Change_Title
+ "\n";
        emailBody = emailBody + "Risk Score: " + input.Risk_Score +
"\n";
        emailBody = emailBody + "Decision: " + input.Approval_Status +
"\n";
        emailBody = emailBody + "Reviewed By: " + managerName + "\n";
        emailBody = emailBody + "Planned Date: " + input.Planned_Date
+ "\n\n";

        if(input.Approval_Status == "Approved")
        {
            emailBody = emailBody + "Your change request has been
APPROVED.\n";
```

```
            emailBody = emailBody + "You may proceed with the
implementation according to the approved plan.\n\n";
        }
        else
        {
            emailBody = emailBody + "Your change request has been
REJECTED.\n";
            emailBody = emailBody + "Please contact the approving
manager for further clarification.\n\n";
        }

        emailBody = emailBody + "Regards,\n";
        emailBody = emailBody + "IT Change Management System";

        // Send email to requester
        sendmail
        [
            from : zoho.adminuserid
            to : requesterEmail
            subject : "Change Request " + input.Approval_Status + " -
" + input.Change_ID
            message : emailBody
        ];
    }
}
```

DS-01 – Automatic Priority and SLA Calculation:

The script checks the impact and urgency values of an incident and assigns a priority level accordingly. Based on the calculated priority, it sets the SLA deadline to 2, 4, 8 or 24 hours from the reported time. As a result, each incident receives an appropriate priority and response deadline automatically.

DS-02 – Automatic Engineer Assignment:

The script retrieves active Cloud Engineers from the Staff Directory and counts their unresolved incidents. It identifies the engineer with the lowest workload and assigns the new incident to that engineer. This helps distribute incidents among available engineers.

DS-03 – Incident Assignment Notification:

After an incident is successfully submitted, the script retrieves the assigned engineer's email address and sends an assignment notification. During testing, the assigned engineer received an email containing the incident details.

DS-04 – Incident Status Notification:

Whenever the incident status is updated, the script retrieves the reporter's email address and sends a notification about the new status. During testing, an email notification was received after the incident status was changed.

DS-05 – SLA Overdue Notification:

The scheduled script checks incidents whose SLA deadlines have passed and whose statuses are neither Resolved nor Closed. It sends an email notification to the assigned engineer for overdue incidents. The scheduled notification was tested successfully.

DS-06 – Change Request Submission Notification:

When a new change request is submitted with Pending approval status, the script retrieves active Approving Managers and sends them an email notification. During testing, the approval notification was received by the concerned approvers.

DS-07 – Change Approval History Automation:

When a change request is approved or rejected, the script checks whether an approver has been selected. It then creates a record in the Change Approval History form containing the change request, reviewer, decision and decision date. During testing, the approval history record was successfully created and displayed in the corresponding report.

DS-08 – Change Decision Notification:

After an approval decision is updated, the script retrieves the requester's email address and sends a notification containing the change request ID and its decision. This keeps the requester informed about the approval outcome.

Overall, the implemented Deluge scripts automate important incident and change management activities. They reduce manual effort, improve communication, support SLA monitoring and maintain approval records within the application.

5.6 Workflows and Automation

Workflows are implemented in Zoho Creator to automate routine activities in incident and change management. They execute Deluge scripts based on form events and scheduled conditions, reducing manual effort and improving the consistency of application processes.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Automatic Priority and SLA Calculation	Form Workflow	Incident Form – On Add, On Validate	Based on the selected Impact and Urgency values.	Calculates incident priority and sets the SLA deadline.	DS-01
Automatic Engineer Assignment	Form Workflow	Incident Form – On Add, On Validate	Considers active Cloud Engineers and their unresolved incident count.	Assigns the incident to the engineer with the lowest workload.	DS-02
Incident Assignment Notification	Form Workflow	Incident Form – On Add, On Success	Uses the assigned engineer's details.	Sends an email notification to the assigned engineer.	DS-03
Incident Status Notification	Form Workflow	Incident Form – On Edit, Status Update	Executes when the incident status is updated.	Sends an email notification to the incident reporter.	DS-04
SLA Overdue Notification	Form Workflow	Form Schedule – Based on SLA Deadline	SLA deadline has passed and status is not Resolved or Closed.	Checks overdue incidents and sends email notifications to assigned engineers.	DS-05
Change Request Submission Notification	Form Workflow	Change Request Form – On Add, On Success	Approval Status is Pending.	Sends email notifications to active Approving Managers.	DS-06
Change Approval History Automation	Form Workflow	Change Request Form – On Edit, Approval Status Update	Approval Status is Approved or Rejected and an approver is selected.	Creates an approval history record with reviewer and decision details.	DS-07

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Change Decision Notification	Form Workflow	Change Request Form – On Edit, Approval Status Update	Approval Status is Approved or Rejected.	Sends the approval decision to the requester through email.	DS-08

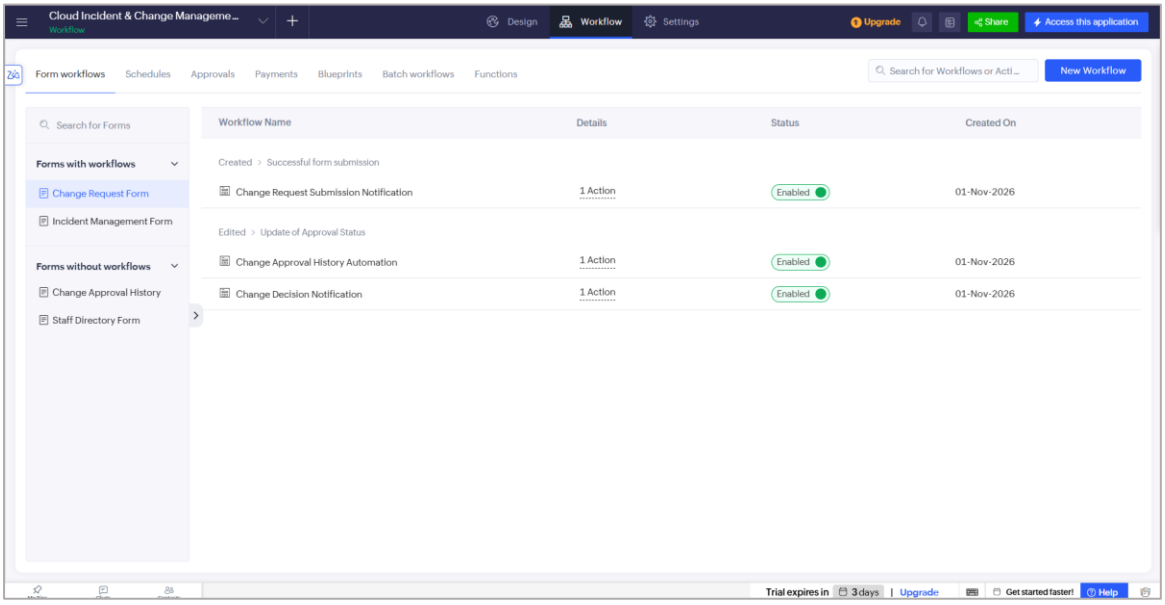


Figure 5.12: Form Workflow Configuration – Incident Management Form

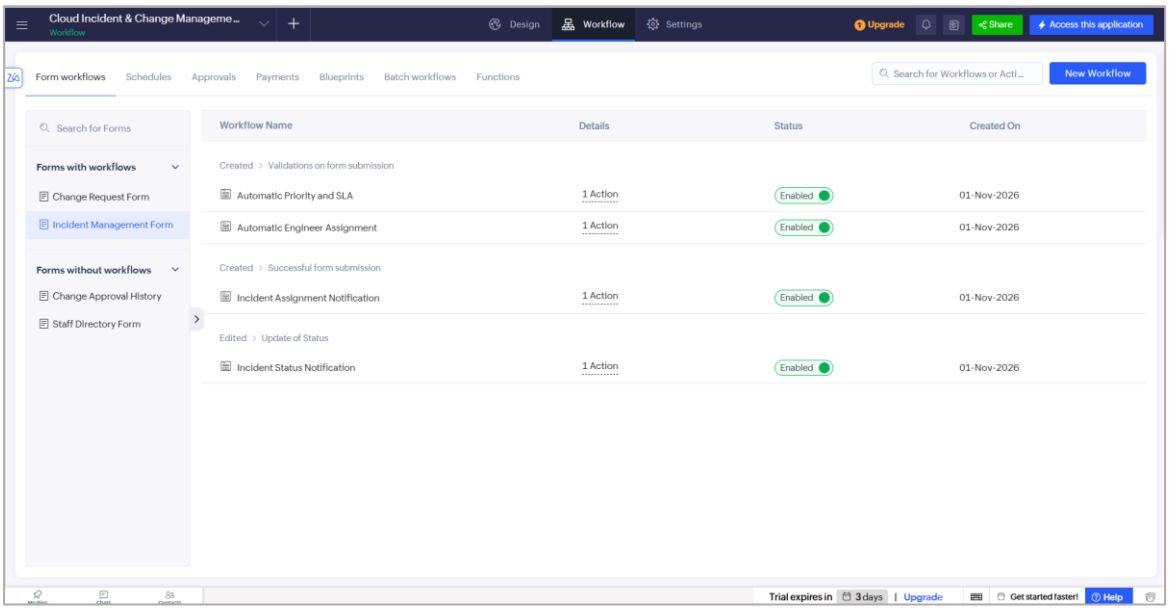


Figure 5.13: Form Workflow Configuration – Change Request Form

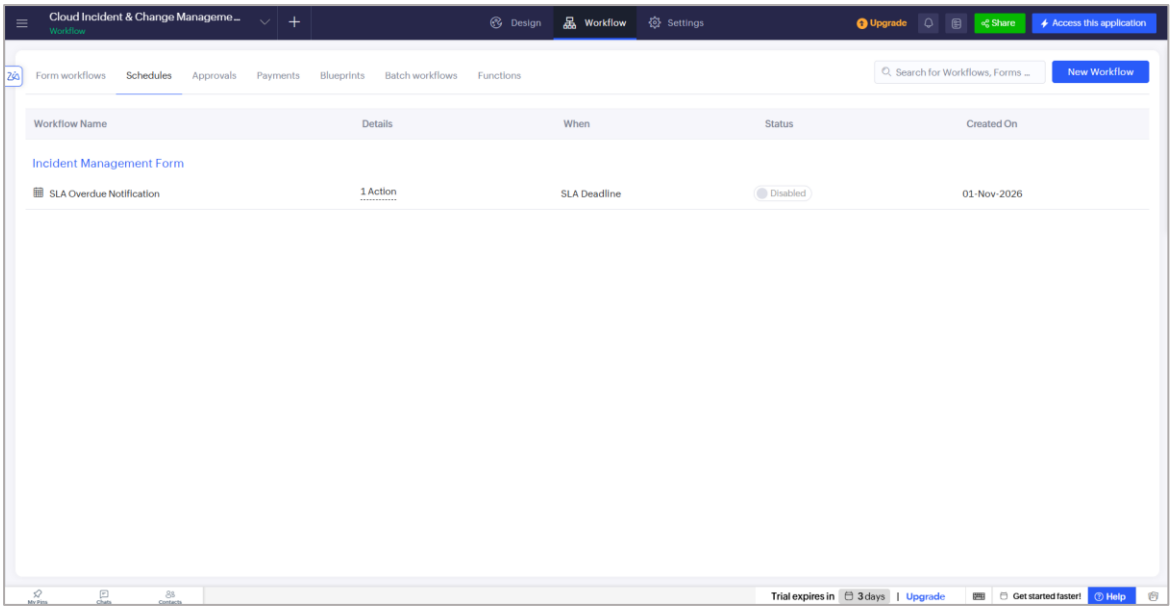


Figure 5.14: Schedule Workflow Configuration – Incident Management Form

5.7 Integrations

The application is integrated with Zoho Desk to automate the creation of support tickets from reported IT incidents. This integration reduces manual effort and enables centralized tracking of incidents through the Zoho Desk platform.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho Desk Integration	Zoho Desk – Tickets Module	Connection + `zoho.desk.create()`	OAuth 2.0	Transfers incident title and description along with department and contact details to create a support ticket automatically.

Code 5.9: Zoho Desk Ticket Creation Script

```
// WF-09: AUTOMATIC ZOHO DESK TICKET CREATION
// Zoho Desk Configuration
orgID = 60094239038;
// Replace with your Organization ID
departmentID = "282718000000010772";
// Replace with your Department ID
contactID = "282718000000364031";
// Replace with your created Contact ID
// Prepare ticket details
ticketData = Map();
ticketData.put("subject",input.Incident_Title);
ticketData.put("description",input.Description);
```

```
ticketData.put("departmentId",departmentID);
ticketData.put("contactId",contactID);
// Create ticket in Zoho Desk
deskResponse =
zoho.desk.create(orgID,"tickets",ticketData,"zoho_desk_connection");
// Check ticket creation response
if(deskResponse.containKey("id"))
{
      info "Zoho Desk Ticket Created Successfully";
      info "Ticket ID: " + deskResponse.get("id");
      info "Ticket Number: " + deskResponse.get("ticketNumber");
}
else
{
      info "Zoho Desk Ticket Creation Failed";
      info deskResponse;
}
```

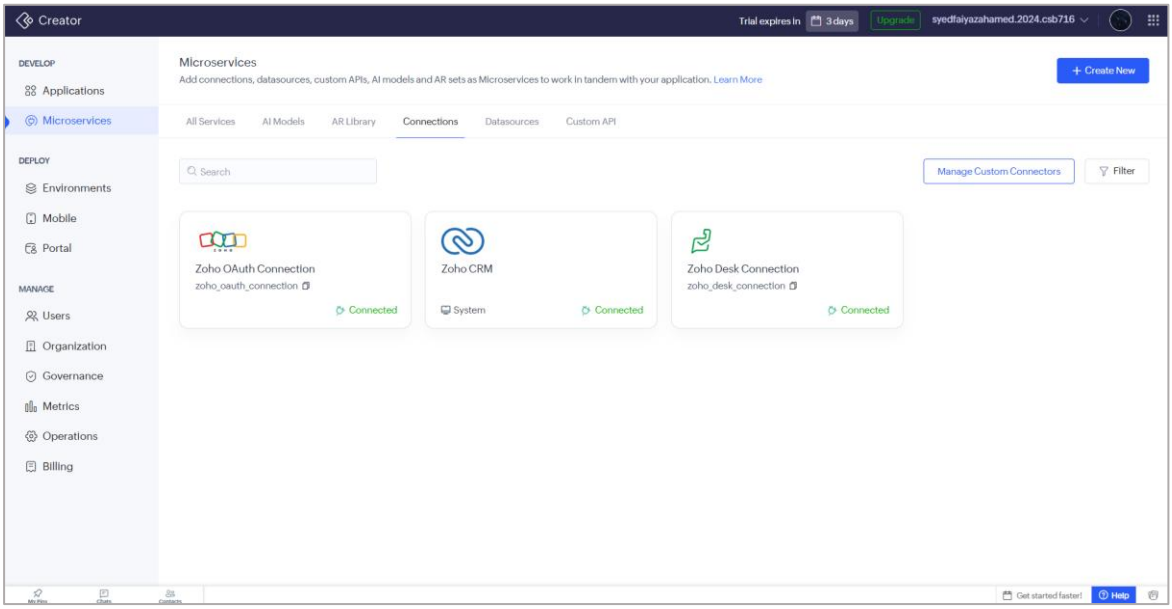


Figure 5.15: Accessing Microservices in Zoho Creator

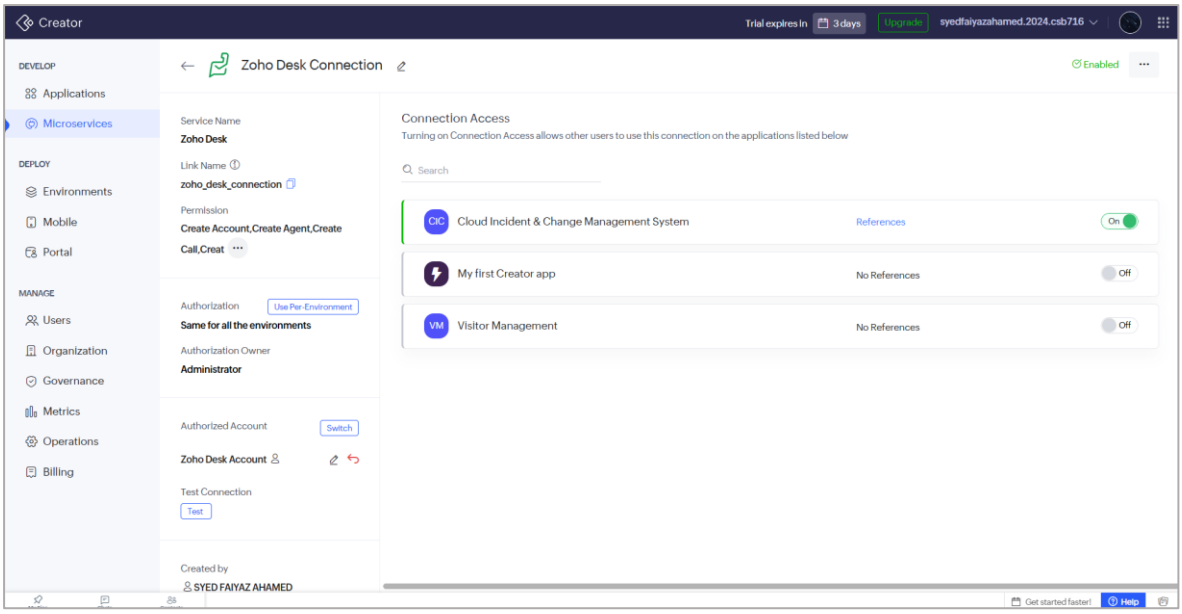


Figure 5.16: Zoho Desk Connection Configuration

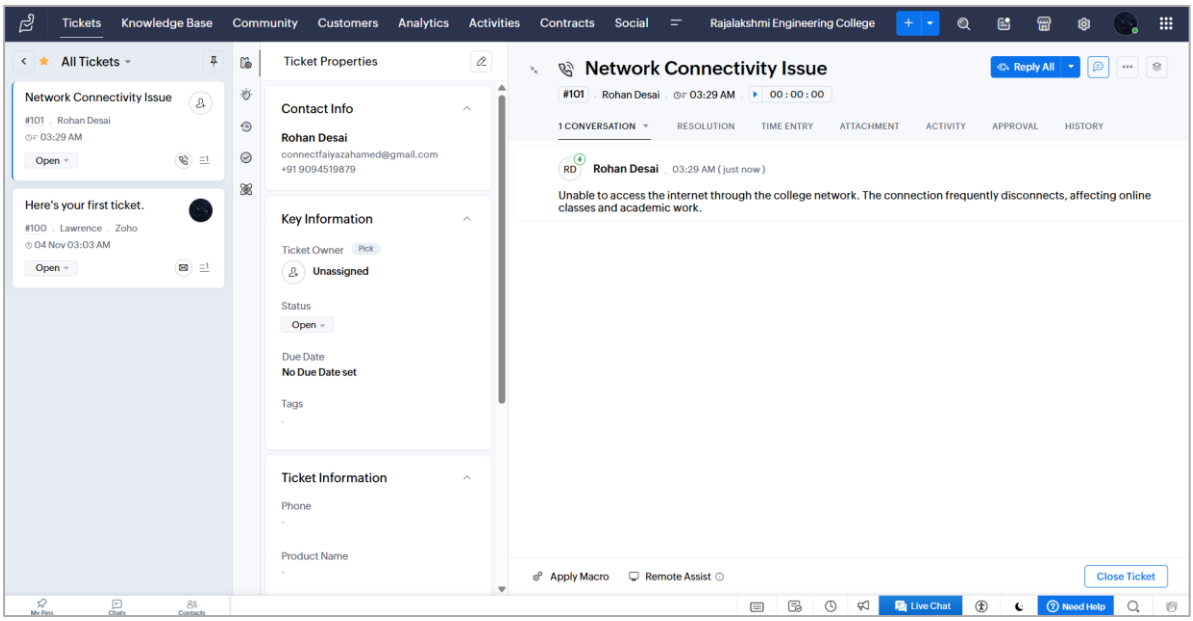


Figure 5.17: Successful Zoho Desk Ticket Creation After Incident Submission

5.8 User Management and Role-Based Access

The application implements Role-Based Access Control (RBAC) using the User Management and User Permissions features of Zoho Creator. This configuration ensures that users are assigned specific roles and permission profiles according to their responsibilities in the incident and change management process.

Four application roles were created: Employee, IT Support Staff, Cloud Engineer and Approving Manager. Each role was configured under the default CEO role. The Share Data with Peers option was kept disabled to avoid automatically sharing records among users at the same hierarchy level.

Separate permission profiles were created for each role. The Employee_Permission profile allows employees to access the Incident Management Form and Change Request Form for submitting requests. The IT_Support_Permission profile provides access to incident reports for monitoring and managing reported issues. The Cloud_Engineer_Permission profile allows engineers to view and update assigned incidents. The Approving_Manager_Permission profile provides access to change requests and approval history for reviewing and managing change decisions.

Users were added to the application through the Manage Users section using their email addresses. During user creation, the corresponding application role and permission profile were assigned. The application administrator retains overall control of application settings and user management.

The permission profiles restrict access to application components and control the viewing and editing of records based on user responsibilities. Delete permissions were disabled to prevent unauthorized removal of application records.

The application uses Zoho Creator's role and permission mechanisms to manage user access. Record-level sharing criteria and customer portal configuration were not separately implemented in this project.

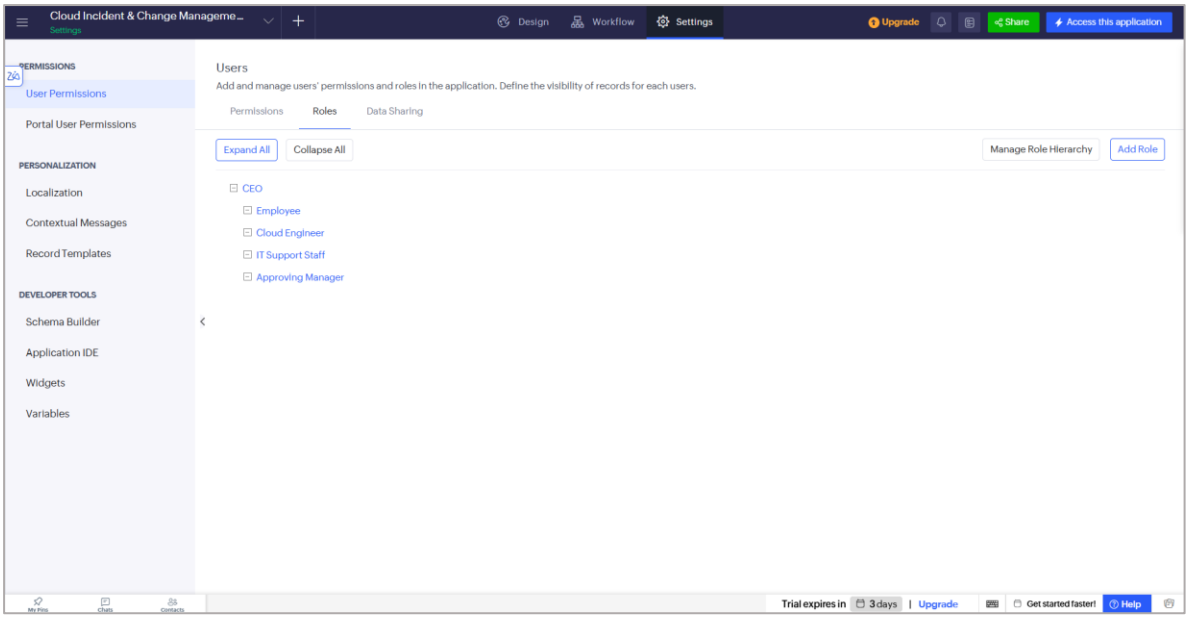


Figure 5.18: Application Roles Configuration

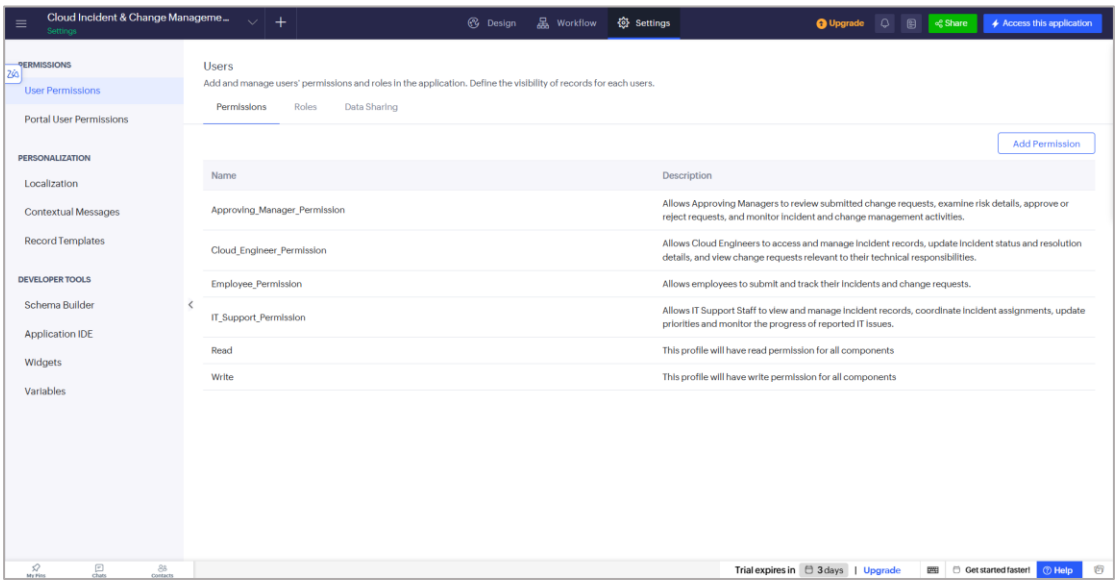


Figure 5.19: Role-Based Permission Profiles Configuration

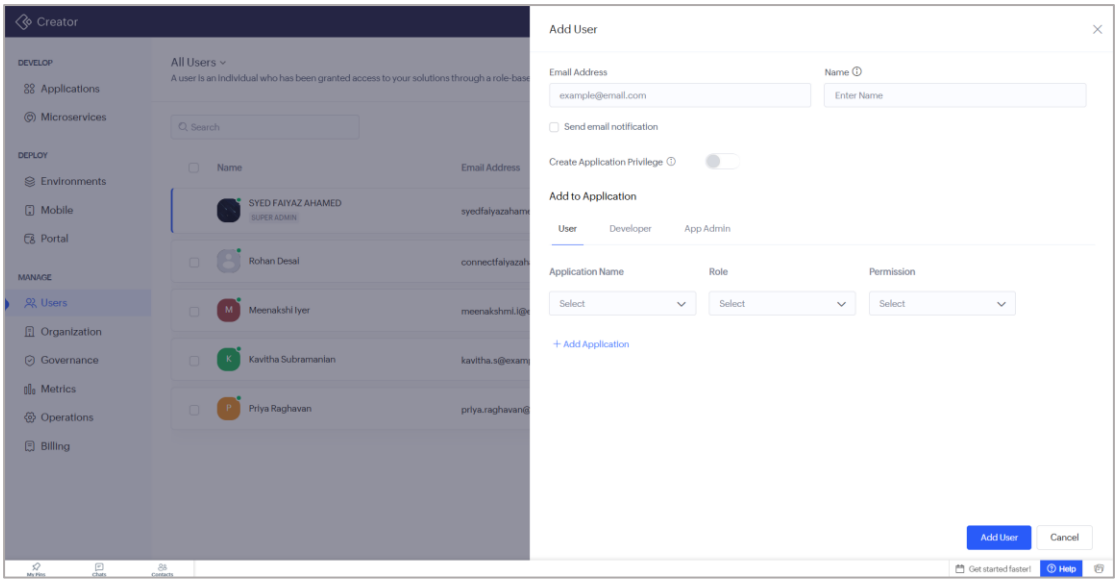


Figure 5.20: User Role and Permission Assignment

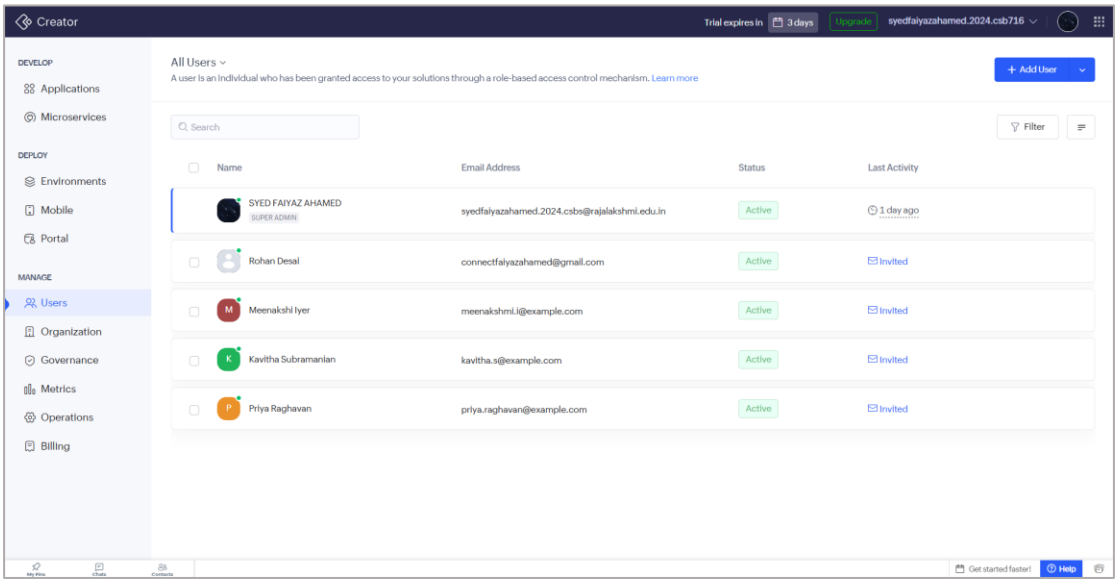


Figure 5.21: List of Users Added to the Application

5.9 AI and Analytics Features

The application uses the AI-assisted capabilities of Zoho Creator to simplify application development and improve the presentation of management information. Zia was used to assist in creating the application dashboard based on the project requirements. The generated dashboard layout and components were reviewed and adjusted to suit the incident and change management processes.

The dashboard provides a centralized view of application activities through reports, charts and summary panels. It helps administrators and other authorized users monitor

incident records, incident status, SLA-related information and change request activities. These visual summaries make it easier to understand the current status of IT support operations.

Zia-assisted features were used during the development process, while the application forms, Deluge scripts, business logic and external integration were configured and verified according to the project requirements. The generated components were checked by reviewing their structure, testing form submissions and verifying the execution of automated workflows.

Advanced AI features such as prediction models, OCR, sentiment analysis, keyword extraction and object detection were not separately implemented in this project. The analytics functionality is based on the reports, charts and dashboard components available in Zoho Creator.

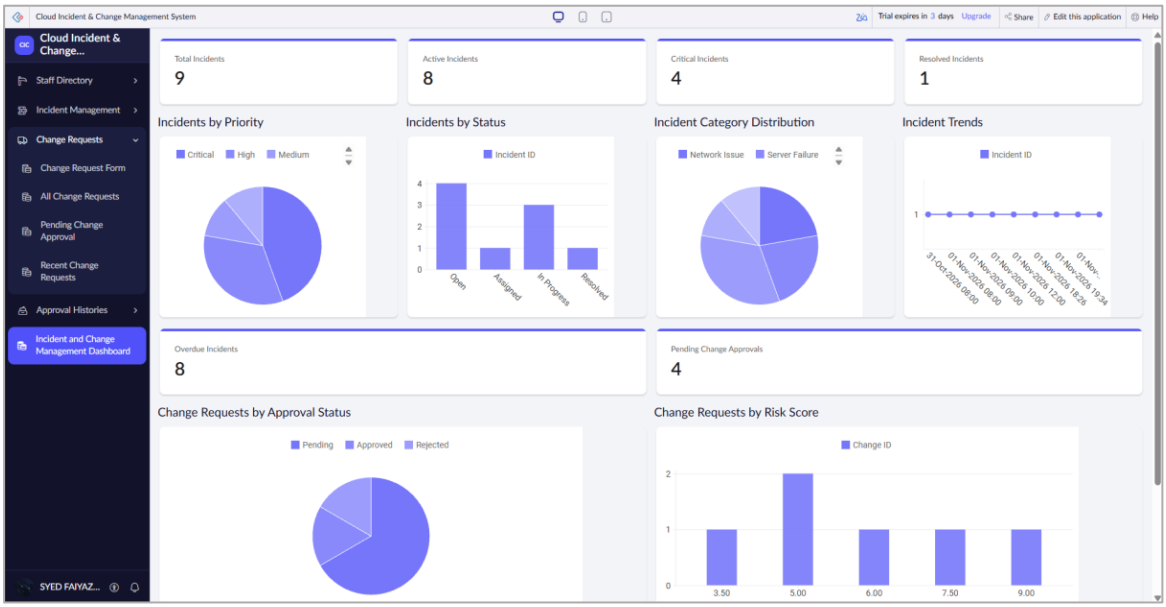


Figure 5.22: AI-Assisted Dashboard and Analytics in Zoho Creator

5.10 Pro-Code Extension

The proposed application was developed primarily using the low-code capabilities of Zoho Creator. Deluge scripting was used to implement custom business logic, including automatic incident priority calculation, SLA deadline generation, engineer assignment, email notifications, approval history creation and Zoho Desk ticket integration.

Additional pro-code extensions such as Zoho Catalyst, JavaScript widgets and custom REST APIs were not separately implemented. The existing low-code features and Deluge scripts were sufficient to meet the functional requirements of the project.

In future enhancements, Zoho Catalyst could be used to develop advanced backend services using programming languages such as Java, Node.js or Python. Custom REST APIs could enable external applications to securely exchange incident and change request information with the system. JavaScript widgets could also be introduced to provide customized dashboard components and interactive user interfaces.

These extensions could improve the application's scalability, integration capabilities and flexibility as the system grows.

5.11 Mobile and Portal Access

The application was primarily developed and tested using the Zoho Creator web interface. A separate customer portal and mobile-specific configuration were not implemented as part of the current project scope.

Zoho Creator provides mobile application support, allowing users to access application forms and reports through mobile devices. In future enhancements, the application can be made more accessible by configuring mobile-specific layouts and enabling suitable mobile features. A customer portal could also be introduced to allow external users to submit incidents and track their requests securely.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

A systematic testing strategy was followed to verify the functionality, reliability and correctness of the Intelligent Cloud Incident & Change Management System. Testing was organized into different levels to identify errors in individual components and ensure that the integrated application operates as expected.

1. Unit Testing

Individual application components were tested independently to verify their functionality. The Incident Management Form was checked for correct data entry, priority calculation and SLA deadline generation. The Change Request Form was tested for proper submission of change details and approval status updates. Deluge scripts were examined to verify automatic engineer assignment, notifications and approval history creation.

2. Integration Testing

Integration testing was carried out to verify the interaction between forms, workflows and external services. Incident submission was tested along with engineer assignment and notification workflows. Change request approval was checked to ensure that the corresponding approval history record was created. The Zoho Desk integration was tested by submitting an incident and verifying the creation of a corresponding support ticket in Zoho Desk.

3. Role-Based Access Testing

The application roles and permission profiles were configured for Employees, IT Support Staff, Cloud Engineers and Approving Managers. Access permissions were defined according to their responsibilities. Role-based testing is used to verify whether each user can access the permitted forms and reports while restricted operations remain unavailable.

4. User Acceptance Testing

User acceptance testing focuses on verifying whether the application meets the requirements of its intended users. Sample incident reports and change requests are submitted to evaluate the ease of use, record tracking, approval process and notification

features. The application interface and generated reports are also reviewed to ensure that the system supports the intended IT service management activities.

The results from these testing activities are used to identify errors, make necessary corrections and improve the overall reliability of the application.

6.2 Test Cases

Test cases were designed to verify the functional and non-functional requirements of the Intelligent Cloud Incident & Change Management System. Both positive and negative test scenarios were considered to evaluate the correctness of form submissions, automated processing, approval activities, notifications, access permissions and external integration.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Submit incident with valid details	Enter all mandatory incident details and submit the form.	Incident record should be created successfully.	To be recorded after testing.	Pending
TC-02	FR-01	Submit incident with missing mandatory fields	Leave a mandatory field empty and submit.	Validation message should be displayed and submission prevented.	To be recorded after testing.	Pending
TC-03	FR-02	Automatic priority calculation	Submit incidents with different impact and urgency values.	Priority should be calculated according to the defined rules.	To be recorded after testing.	Pending
TC-04	FR-02	Automatic SLA deadline generation	Submit a valid incident and check its SLA deadline.	SLA deadline should be generated based on the calculated priority.	To be recorded after testing.	Pending
TC-05	FR-03	Automatic engineer assignment	Submit an incident when active engineers are available.	Incident should be assigned according to the engineer assignment criteria.	To be recorded after testing.	Pending

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-06	FR-03	Engineer availability validation	Submit an incident when no eligible active engineer is available.	System should handle the absence of an eligible engineer without incorrect assignment.	To be recorded after testing.	Pending
TC-07	FR-04	Incident assignment notification	Submit an incident and check the notification received by the concerned user.	Assignment notification should be generated successfully.	To be recorded after testing.	Pending
TC-08	FR-04	Incident status notification	Update an incident status and check the notification.	Concerned users should receive a status update notification.	To be recorded after testing.	Pending
TC-09	FR-05	Incident status transition control	Attempt to change an incident from one status to another.	Status transitions should follow the defined Blueprint rules.	Blueprint was not implemented in the application.	Not Implemented
TC-10	FR-06	Submit change request with valid details	Enter the change description, risk score, implementation plan and rollback plan.	Change request should be created successfully.	To be recorded after testing.	Pending
TC-11	FR-06	Submit change request with missing details	Leave a mandatory change request field empty and submit.	Validation message should be displayed.	To be recorded after testing.	Pending
TC-12	FR-07	Approve a change request	Open a pending change request and select Approved.	Approval status should be updated and an approval history record should be created.	Approval history creation verified successfully.	Pass
TC-13	FR-07	Reject a change request	Open a pending change request and select Rejected.	Rejection status should be updated and the decision should be recorded.	To be recorded after testing.	Pending

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-14	FR-07	Change decision notification	Approve or reject a change request and check the notification.	Requester should receive the decision notification.	To be recorded after testing.	Pending
TC-15	FR-08	View dashboard and reports	Open the application dashboard and relevant reports.	Incident and change request information should be displayed correctly.	To be recorded after testing.	Pending
TC-16	FR-09, NFR-02	Unauthorized access test	Log in as an Employee and attempt to access restricted modules.	Access to restricted modules should be denied.	To be recorded after testing.	Pending
TC-17	FR-10	Search and filter records	Search for incidents and change requests using available fields and status.	Matching records should be displayed correctly.	To be recorded after testing.	Pending
TC-18	FR-02, FR-04	SLA overdue notification	Execute the scheduled workflow for an incident whose SLA deadline has passed.	Overdue notification should be generated successfully.	Scheduled SLA notification tested successfully.	Pass
TC-19	FR-07	Zoho Desk ticket integration	Submit a new incident and check the Zoho Desk Tickets module.	A corresponding support ticket should be created in Zoho Desk.	Ticket creation verified successfully.	Pass
TC-20	NFR-01	Application performance test	Open forms, reports and dashboards under normal testing conditions.	Components should load within the specified five-second target.	To be recorded after testing.	Pending
TC-21	NFR-05, NFR-06	Workflow modification and record history check	Modify a workflow and inspect available record history or audit information.	Changes should be manageable and important record activities should be traceable.	To be recorded after testing.	Pending

The test cases cover the major application functions, including incident registration, automated priority and SLA calculation, engineer assignment, notification generation, change request processing, approval history creation and external ticket integration. Negative test cases are included to verify mandatory field validation and access restrictions.

The test results are recorded based on the observed application behaviour. Features that were not implemented or tested are identified separately to distinguish completed functionality from pending verification.

6.3 Debugging Deluge Scripts

Debugging was performed during the development of the application to identify and correct errors in Deluge scripts and automated workflows. The debugging process involved using the info statement to display variable values and execution messages, examining workflow execution logs and checking error messages generated during form submission.

The info statement was used to display intermediate results and verify whether individual script operations were executed successfully. Workflow execution logs helped identify errors in automated processes such as incident assignment, SLA notifications, approval history creation and external ticket integration. Error messages displayed during form submission were also examined to identify incorrect field references and validation issues.

Example of a Debugging Issue – Zoho Desk Integration

During the implementation of automatic ticket creation in Zoho Desk, an error occurred because an incorrect connection link name was specified in the Deluge script. The script initially used desk_connection, whereas the actual connection link name configured in Zoho Creator was zoho_desk_connection.

The connection reference was corrected in the zoho.desk.create() task. Additional info statements were included to display the response received from Zoho Desk and verify whether the ticket was created successfully. After correcting the connection name, the integration was tested again by submitting a new incident. The corresponding support ticket was successfully created in Zoho Desk.

This debugging process helped identify configuration errors, verify script execution and improve the reliability of automated workflows.

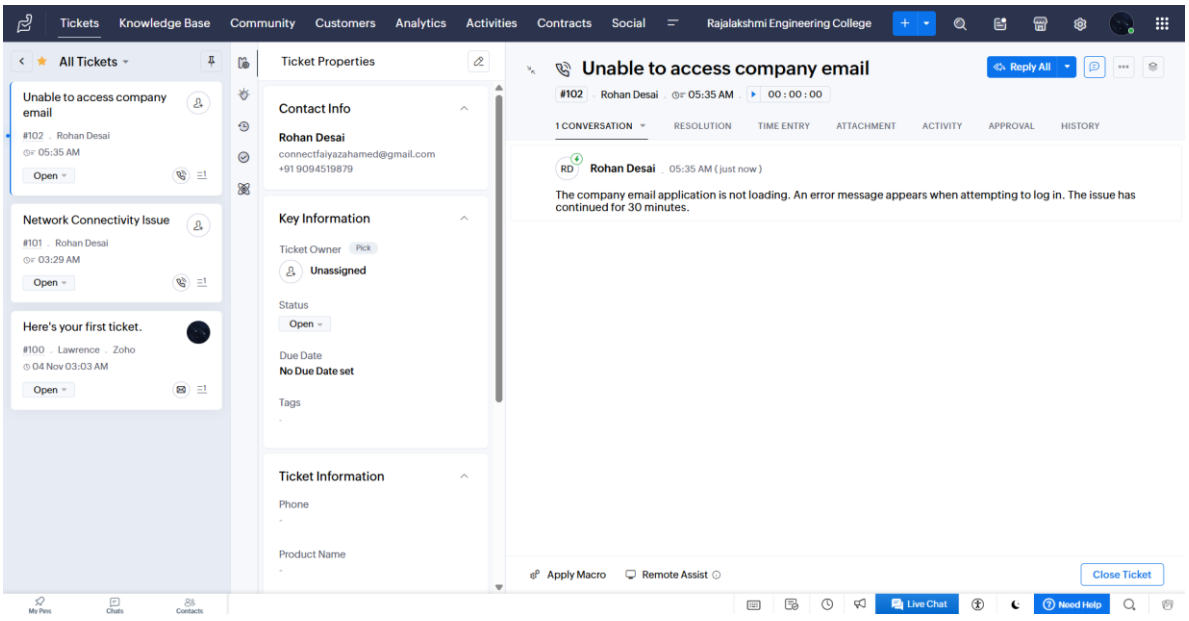


Figure 6.1: Deluge Debug Output Showing Successful Zoho Desk Integration

6.4 Defect Log

A defect log was maintained to record errors identified during application development and testing. It includes the description of each defect, its root cause, the corrective action taken and its final status. This helps track debugging activities and verify that identified issues have been resolved.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Zoho Desk ticket creation failed during incident submission.	Incorrect connection link name (`desk_connection`) was used in the Deluge script instead of the configured connection name.	Replaced the incorrect connection reference with `zoho_desk_connection` and tested the integration again.	Closed
BUG-02	Zoho Desk integration could not proceed initially due to connection configuration issues.	Required connection and integration details were not configured correctly.	Configured the Zoho Desk OAuth connection and verified the connection settings.	Closed

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-03	Approval history creation required verification after a change request decision.	The approval history automation needed to be checked against the correct approval status and form lookup field.	Verified the WF-07 script, corrected the configuration where necessary and tested the creation of approval history records.	Closed

6.5 User Acceptance Testing

User Acceptance Testing (UAT) was considered to evaluate the usability and suitability of the Intelligent Cloud Incident & Change Management System from the end-user perspective. The testing focuses on whether users can perform their assigned activities easily and whether the application meets the expected requirements.

Two users were selected to evaluate the application using different roles. They were asked to perform basic operations such as submitting incidents, viewing assigned records and submitting change requests. Their feedback was collected to identify areas for improvement.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken

The feedback collected during user acceptance testing helps assess the application's ease of use, navigation and suitability for its intended purpose. Suggestions provided by the users can be considered for further improvements.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Deployment strategy defines the process of developing, testing and releasing an application for end users. Zoho Creator provides three environments: Development, Stage and Production. Each environment serves a specific purpose in the application development lifecycle.

The three environments are described below:

1. **Development Environment:** Used to create application forms, reports, dashboards and Deluge workflows. Developers can make modifications and perform initial testing during the development process.
2. **Stage Environment:** Used to test the application before its final release. Forms, workflows, user permissions and integrations are verified to identify errors and ensure that the application functions correctly.
3. **Production Environment:** Represents the live application accessed by authorized end users. Tested and approved application changes are made available in this environment.

Deployment Strategy

The recommended deployment process follows the sequence Development → Stage → Production. Application modifications are first completed in Development and then published to Stage for testing. After successful verification, the approved version is published to Production.

In this project, the Intelligent Cloud Incident & Change Management System was developed using Zoho Creator. The forms, reports, dashboards, Deluge workflows, user permissions and Zoho Desk integration were configured and tested within the application before it was made available to users.

Version Notes and Rollback Plan

Version notes are maintained to record application modifications, including new features, workflow corrections and bug fixes. Each update can be documented with a version number and a brief description of the changes.

If an issue occurs after an update, the affected functionality must be investigated and corrected. Important application data should be backed up before major modifications. The recovery plan involves restoring the required application configuration and data using the available backup and recovery options, followed by testing to ensure normal operation.

The deployment strategy helps maintain application reliability, control modifications and reduce the possibility of errors affecting end users.

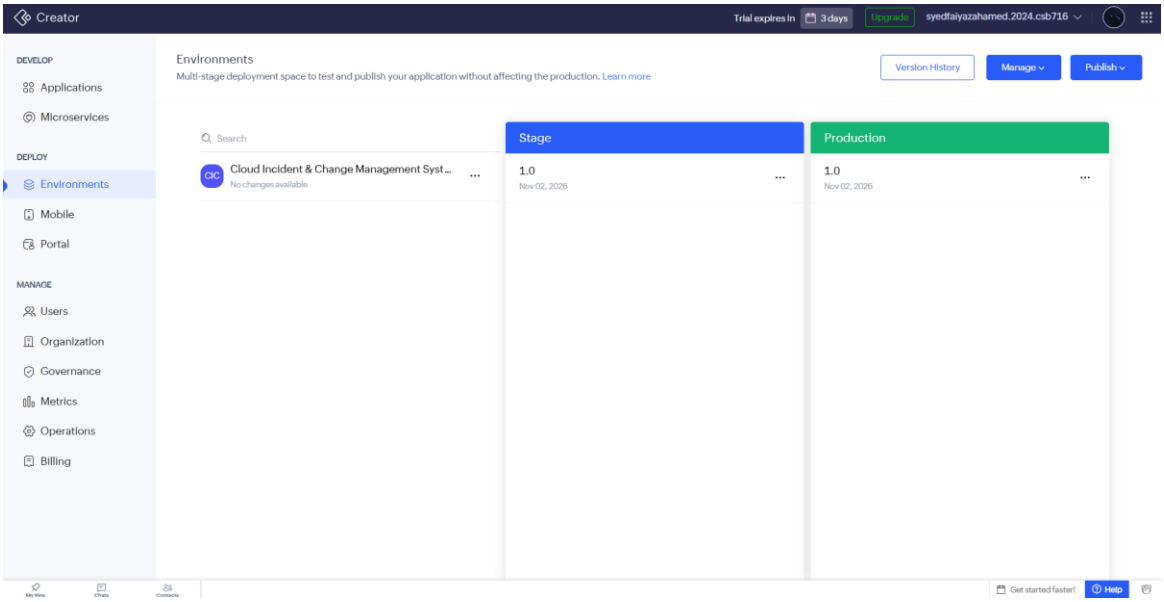


Figure 7.1: Environments and Application Publishing in Zoho Creator

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	[ ]
3	Roles, permissions and users configured in Production	✓
4	Connections authorised in Production	✓
5	Schedules and workflows enabled	✓
6	Application published and URL verified	N/A
7	Version / release notes recorded	[ ]

7.2 Performance Optimisation

Performance optimisation is important to ensure that the application operates efficiently and provides a smooth experience for users. In the Intelligent Cloud Incident & Change Management System, optimisation techniques were considered during the configuration of forms, reports, dashboards and Deluge workflows.

The following techniques were used or considered to improve application performance:

1. **Filtered Reports:** Reports such as Assigned Incidents and Pending Change Approval display relevant records based on specific conditions. This helps users access required information without browsing unrelated records.
2. **Efficient Record Fetching:** Deluge fetch operations can use specific criteria to retrieve only the required records instead of fetching the entire dataset. This reduces unnecessary data processing.
3. **Optimised Workflow Logic:** Deluge workflows are structured to perform the required calculations and actions without unnecessary operations. Conditional statements are used to control workflow execution.
4. **Relevant Report Columns:** Reports can be configured to display only the important fields required by users, improving readability and reducing unnecessary information.
5. **Appropriate Use of Lookups:** Lookup fields are used to connect related records such as staff details, incidents and change requests. This avoids unnecessary duplication of information.
6. **Scheduled Operations:** SLA monitoring is performed through a scheduled workflow rather than requiring continuous manual checking.

These techniques help improve data retrieval, report usability and workflow efficiency. The application was tested using sample records to verify that its major functions operated correctly. A separate numerical performance benchmark was not conducted.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Filtered report using relevant criteria	Assigned Incidents Report	Users may need to browse multiple incident records.	Displays incidents assigned to the relevant engineer.	Easier record identification and reduced unnecessary browsing.

Optimisation Applied	Component	Before	After	Improvement
Filtered report for pending records	Pending Change Approval Report	Users may need to check all change requests.	Displays requests awaiting approval.	Faster identification of pending approvals.
Use of lookup fields to connect related records	Staff Directory and Incident Management Forms	Repeated staff information may need to be entered.	Staff details are selected through lookup fields.	Reduces duplicate data entry and improves consistency.
Scheduled workflow execution	SLA Monitoring Workflow	Manual checking of SLA deadlines may be required.	Scheduled workflow checks overdue incidents.	Reduces repetitive manual monitoring.
Criteria-based record retrieval where required	Deluge Workflows	Fetching unnecessary records can increase processing.	Specific criteria retrieve relevant records.	Reduces unnecessary record processing.

7.3 Security Features

Security features are essential to protect application data and prevent unauthorized access. The Intelligent Cloud Incident & Change Management System uses the security and access control features available in Zoho Creator to ensure that users can access application components according to their assigned responsibilities.

The following security measures were configured in the application:

1. **Role-Based Access Control:** Four application roles were created: Employee, IT Support Staff, Cloud Engineer and Approving Manager. Each role is assigned responsibilities based on its function.
2. **Permission Profiles:** Separate permission profiles were created for each role to control access to forms, reports and application features.
3. **Restricted Module Access:** Users are provided access only to the application components required for their responsibilities. For example, employees can

access incident and change request forms, while approving managers can access change requests and approval history.

4. **Delete Permission Restriction:** Delete permissions were disabled in the configured permission profiles to reduce the possibility of accidental or unauthorized record deletion.
5. **Peer Data Sharing Restriction:** The Share Data with Peers option was disabled in the role configuration to restrict automatic data sharing between users at the same hierarchy level.
6. **OAuth Connection:** The Zoho Desk integration uses an authorized OAuth connection named zoho_desk_connection. This allows the application to communicate with Zoho Desk without directly storing authentication credentials in the Deluge script.

These security measures help maintain controlled access to application records and reduce the risk of unauthorized data access or modification. Additional security controls can be introduced based on future application requirements.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role-based permission profiles	Employee, IT Support Staff, Cloud Engineer and Approving Manager profiles	Unauthorized module access
Restricted form and report access	Incident Management, Change Request and Approval History modules	Unauthorized data access
Delete permission disabled	Configured permission profiles	Accidental or unauthorized deletion
Peer data sharing disabled	Application role configuration	Unintended sharing of records between peers
OAuth connection	Zoho Desk integration using zoho_desk_connection	Exposure of authentication credentials
Separate user roles	Manage Users and Roles	Unauthorized access to administrative functions

7.4 Governance and Administration

Governance and administration ensure the proper management, monitoring and maintenance of the application. In the proposed system, administrative activities are handled through Zoho Creator's application management features.

The following practices support application administration:

1. **Audit Trail:** Maintains records of user activities, including record creation, modification and deletion.
2. **Change History:** Helps administrators track changes made to application records and identify the users responsible.
3. **Naming Conventions:** Consistent names are used for forms, reports, workflows and fields to simplify application maintenance.
4. **Backup and Export:** Application data can be backed up and exported periodically to reduce the risk of data loss.
5. **User Management:** Administrators manage users, assign roles and configure permission profiles according to their responsibilities.
6. **Application Maintenance:** Forms, reports and Deluge workflows are reviewed and updated to correct errors and maintain application functionality.

These practices improve accountability, support data management and ensure the long-term maintainability of the application.

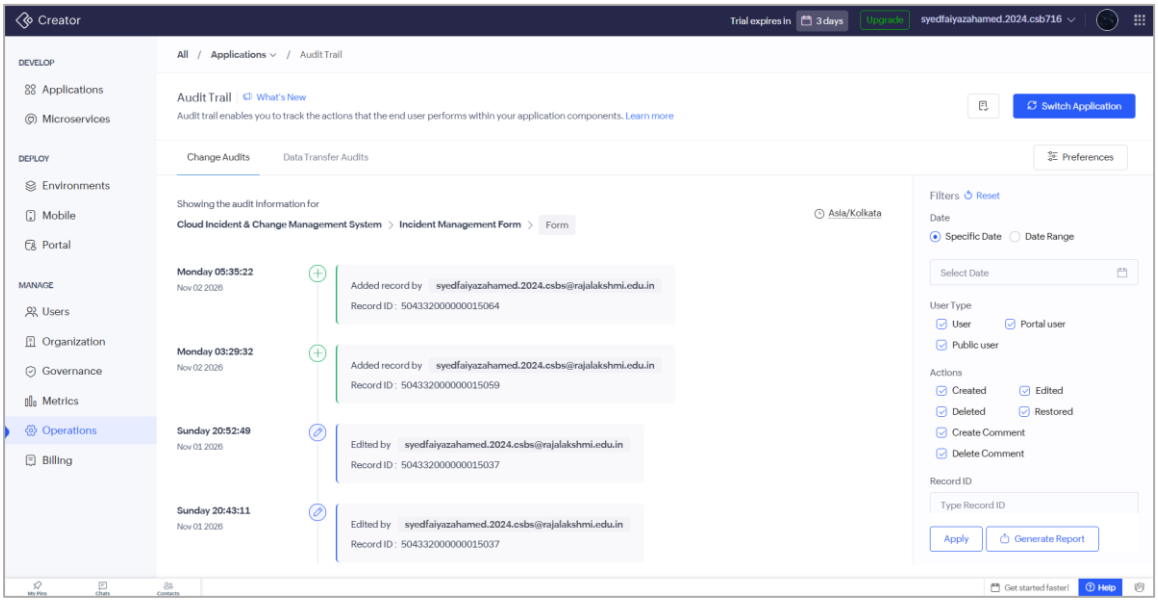


Figure 7.2: Audit Trail and Application Administration in Zoho Creator

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The Intelligent Cloud Incident & Change Management System was tested using a sample IT incident scenario to demonstrate its end-to-end functionality. The scenario involves an employee reporting an IT issue, the system processing the incident and the support team handling its resolution.

Scenario: Unable to Access Company Email

An employee reports an issue with the company email service. The application processes the incident, assigns it to an engineer and creates a corresponding ticket in Zoho Desk.

Step 1: Incident Submission

The employee logs into the application and submits an incident through the Incident Management Form by providing the incident title, description, category, affected service, impact and urgency. The submitted record is stored in the application.

Step 2: Automatic Processing and Assignment

After submission, the system calculates the incident priority and SLA deadline based on the defined rules. The incident is assigned to an eligible engineer through the automatic assignment workflow. A notification is also triggered to inform the concerned user.

Step 3: Incident Tracking and External Ticket Creation

The assigned engineer can access the incident and update its status and resolution details. The application also creates a corresponding ticket in Zoho Desk through the configured integration. The incident information can be monitored through the available reports and dashboard.

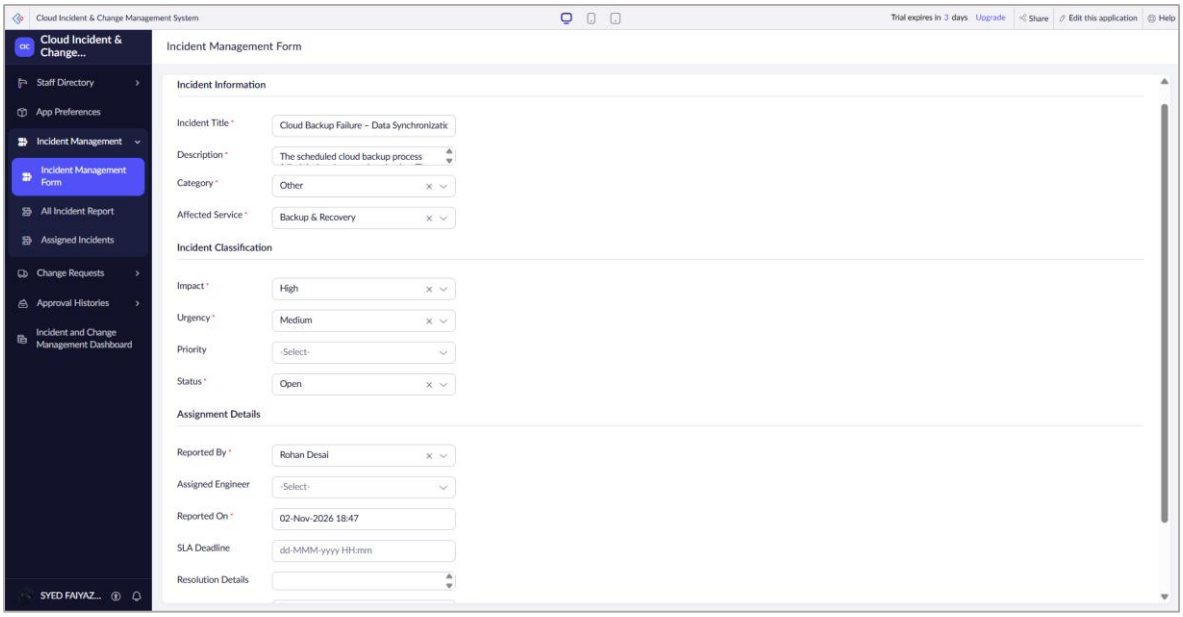


Figure 8.1: Scenario Step 1 – Incident Submission

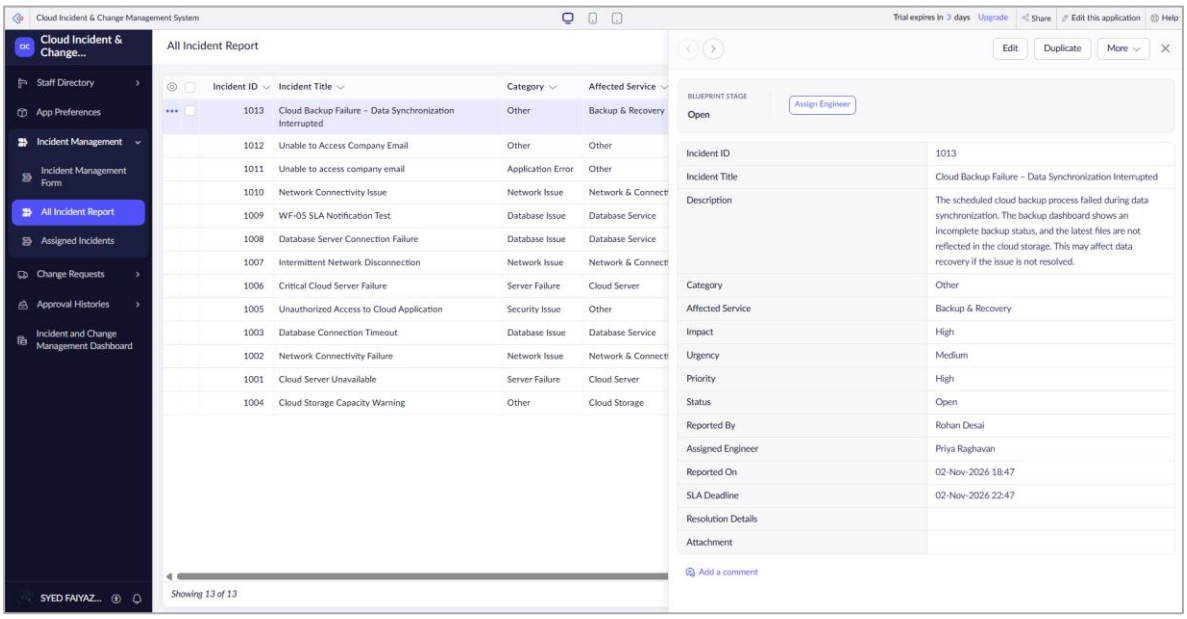


Figure 8.2: Scenario Step 2 – Automatic Priority Calculation and Engineer Assignment

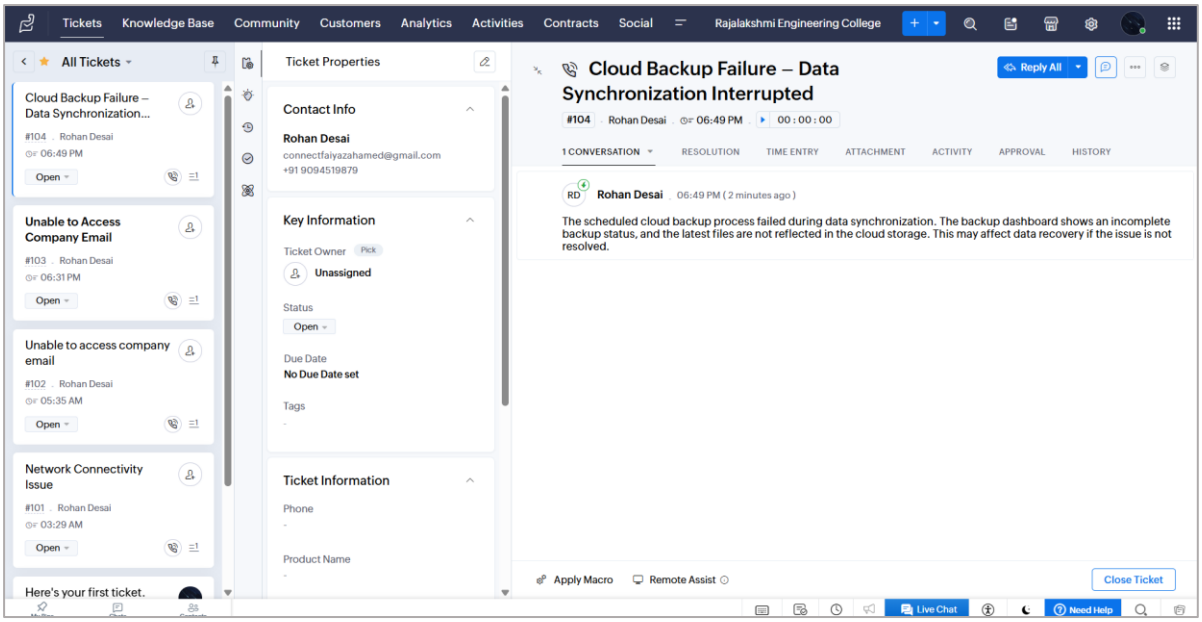


Figure 8.3: Scenario Step 3 – Zoho Desk Ticket Creation

8.2 Achievement of Objectives

The proposed objectives of the Intelligent Cloud Incident & Change Management System were addressed through the development of forms, reports, workflows, integrations and access control features in Zoho Creator. The achievement of each objective is summarized below.

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Design a structured data model for managing staff, incidents, change requests and approvals.	Staff Directory Form, Incident Management Form, Change Request Form and Change Approval History.	Centralized storage and management of project records.
Automate incident priority, SLA monitoring, engineer assignment, notifications and approval tracking.	Deluge workflows WF-01 to WF-08 and scheduled SLA workflow.	Automated priority calculation, engineer assignment, notifications and approval history creation.
Develop centralized dashboards and reports for monitoring incidents and change requests.	Zia-assisted dashboard, All Incident Report, Assigned Incidents, All Change Requests and Pending Change Approval reports.	Improved visibility of incident and change request activities.

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Implement a structured change management process with managerial review and approval.	Change Request Form, Approval Status field, manager lookup and Change Approval History automation.	Change requests can be submitted, reviewed and their approval or rejection decisions recorded.
Integrate Zoho Creator with suitable Zoho services to improve incident management.	Zoho Desk integration through WF-09 using the configured OAuth connection.	A corresponding Zoho Desk ticket is created automatically when an incident is submitted.
Configure role-based access, data validation and administrative controls.	Four application roles, separate permission profiles, form validations and restricted access settings.	Access to application components is controlled according to configured user roles.

The implementation addresses the major objectives of the project by providing centralized incident and change request management, workflow automation, approval tracking, external ticket creation and role-based access configuration. The developed application demonstrates how Zoho Creator and Deluge scripting can be used to simplify IT service management activities and reduce manual effort.

8.3 Comparison with the Existing System

The existing approach to IT incident and change management often relies on emails, phone calls, spreadsheets and manual follow-ups. These methods may result in delayed responses, scattered information and additional administrative effort. The proposed application provides a centralized platform with automated workflows, reports and role-based access to improve these activities.

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Depends on manual communication and follow-ups.	Automated priority calculation, assignment and notifications reduce manual processing.	Faster and more consistent request handling.

Parameter	Existing System	Proposed Application	Improvement
Manual steps	Requires manual recording, assignment and status updates.	Deluge workflows automate several repetitive activities.	Reduced manual effort.
Report generation	Information is collected and compiled manually from different sources.	Reports and dashboards display centralized incident and change request records.	Easier and quicker access to reports.
Data accessibility	Records may be scattered across emails, spreadsheets and communication channels.	Records are maintained centrally in Zoho Creator.	Improved data organization and accessibility.
Incident assignment	Support personnel manually identify and assign incidents.	Workflow automatically assigns incidents to eligible engineers.	More systematic incident allocation.
Change approval	Requests and decisions are communicated through emails or other manual channels.	Change requests and approval decisions are maintained in dedicated forms and reports.	Better tracking of approval activities.
Notifications	Requires manual follow-ups and communication.	Automated notifications are triggered by configured workflows.	Reduced dependency on manual communication.
External ticket management	Separate ticket creation may be required.	Incident submission automatically creates a corresponding Zoho Desk ticket.	Improved integration with external IT service management.

The comparison shows that the proposed application improves the organization of IT service management activities by combining incident registration, automated

assignment, SLA monitoring, change approval tracking and external ticket creation in one platform. However, exact processing-time and efficiency improvements would require measurement through a separate performance study.

8.4 Limitations of the Low-Code Approach

Although Zoho Creator simplifies application development through visual tools and Deluge scripting, the low-code approach has certain limitations. During the development of the Intelligent Cloud Incident & Change Management System, some practical challenges were identified.

1. UI Customisation Limitations

Zoho Creator provides predefined form, report and dashboard components. However, extensive control over page layouts, animations and advanced interface elements may be limited compared with a fully custom-coded application.

Workaround: The application uses the available form, report and dashboard components, with suitable field arrangements and navigation sections to maintain a consistent user interface.

2. Deluge Scripting Constraints

Deluge supports workflow automation and integration, but its syntax and execution environment differ from general-purpose programming languages. Complex business logic may require additional scripting and debugging. Zoho Creator also applies execution limits based on the subscription plan.

Workaround: The application uses separate workflows for priority calculation, SLA monitoring, engineer assignment, notifications and approval history. This keeps the scripts focused and easier to maintain.

3. API and Integration Limits

External service integrations depend on valid OAuth connections, correct identifiers and the API usage limits of the platform. Incorrect connection names or configuration details can cause integration failures. Zoho Creator also applies daily and rate-based API limits.

Workaround: The Zoho Desk integration uses a configured OAuth connection named zoho_desk_connection. The connection reference was corrected after an initial error, and the ticket creation workflow was tested successfully.

4. Handling Complex Business Logic

As application requirements become more complex, managing multiple conditions, workflows and dependencies through a low-code environment can become difficult. Debugging may also require checking field link names, variable declarations and workflow execution logs.

Workaround: The project separates incident management and change management into dedicated forms and workflows. Deluge info statements and execution logs are used to identify and correct scripting errors.

5. Vendor Dependency and Platform Lock-in

Applications developed using Zoho Creator depend on its platform, Deluge scripting environment, data structures and subscription plans. Moving the complete application to another platform may require redesigning forms, rewriting scripts and recreating integrations.

Workaround: Appropriate naming conventions and structured form design are followed to make the application easier to maintain. Future migration would require planning for data export and redevelopment of platform-specific logic.

The low-code approach provides faster development and reduces the need for extensive programming. However, limitations in UI customisation, scripting, API usage, complex logic and platform dependency must be considered. In this project, these limitations were addressed through modular workflows, debugging, proper connection configuration and the use of built-in Zoho Creator features.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The development of the Intelligent Cloud Incident & Change Management System provided practical experience with no-code, low-code and pro-code approaches.

1. No-Code Configuration: Forms, reports, dashboards, lookup fields, user roles and permission profiles were configured using Zoho Creator's visual interface without writing code.

2. Low-Code Development: Deluge scripting was used for automatic priority calculation, SLA monitoring, engineer assignment, notifications, approval history creation and Zoho Desk ticket integration.

3. Pro-Code Development: Advanced features such as custom web interfaces, complex analytics, external service integrations and specialised algorithms may require programming languages, custom APIs or external development tools.

Reflection

The project demonstrated that no-code tools are suitable for basic application configuration, while low-code scripting is useful for implementing business logic and workflow automation. Pro-code development would be considered if future requirements exceed Zoho Creator's customisation, scripting or integration capabilities. Therefore, selecting the appropriate approach depends on application complexity, flexibility, scalability and maintenance requirements.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Intelligent Cloud Incident & Change Management System was successfully developed using Zoho Creator to simplify IT incident and change management activities. The application provides centralized forms for staff details, incident registration, change requests and approval records. Deluge workflows automate incident priority calculation, SLA monitoring, engineer assignment, notifications and approval tracking.

The integration with Zoho Desk enables automatic ticket creation for submitted incidents, while dashboards, reports and role-based permission profiles support effective monitoring and controlled access. The project demonstrates how low-code application development can reduce manual effort, improve process coordination and provide a centralized solution for IT service management.

9.2 Future Enhancements

The following enhancements can be implemented in future versions of the application to improve its functionality and scalability:

- **AI-Based Predictive Analytics:** Use Zia and AI capabilities to predict recurring incidents, identify potential SLA violations and recommend suitable resolutions.
- **Advanced Cloud Monitoring Integration:** Integrate cloud monitoring services to automatically detect infrastructure issues and generate incident records.
- **Catalyst-Based Microservices:** Develop Zoho Catalyst microservices to handle complex processing, advanced analytics and tasks beyond the capabilities of standard Deluge workflows.
- **Mobile Application Support:** Provide improved mobile access for employees and support engineers to submit, update and monitor requests.
- **Enhanced Security and Auditing:** Introduce advanced authentication, record-level access restrictions and detailed activity monitoring.

These enhancements can improve the intelligence, scalability, security and overall effectiveness of the proposed application.

9.3 Learning Outcomes and Career Relevance

Developing the Intelligent Cloud Incident & Change Management System provided practical experience in low-code application development using Zoho Creator. The project helped develop skills in form design, report generation, dashboard configuration, Deluge scripting, workflow automation, API integration and role-based access management. It also provided an understanding of debugging, testing and integrating external services such as Zoho Desk.

These skills are relevant to several career pathways, including:

- **Citizen Developer:** Developing business applications using low-code and no-code platforms.
- **Low-Code Developer:** Building applications, automating workflows and implementing business logic using platforms such as Zoho Creator.
- **Solutions Consultant:** Understanding business requirements and designing suitable application solutions.
- **Automation Engineer:** Developing automated workflows and integrating different software services.

The project demonstrates the growing importance of low-code platforms in modern software development. The knowledge gained provides a foundation for exploring advanced automation, cloud services and pro-code development in future career opportunities.

REFERENCES

[1] Zoho Corporation, “Features of a Low-Code Application Development Platform – Zoho Creator.” [Online]. Available: https://www.zoho.com/creator/features/

[2] Zoho Corporation, “Zoho Creator – Starter Guide and FAQs.” [Online]. Available: https://help.zoho.com/portal/en/kb/creator/faqs/getting-started/articles/zoho-creator-starterguide

[3] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?” *Software and Systems Modeling*, vol. 21, pp. 437–446, 2022. doi: 10.1007/s10270-021-00970-2.

[4] D. Liu et al., “What's Wrong With Low-Code Development Platforms? An Empirical Study of Low-Code Development Platform Bugs,” *IEEE Transactions on Reliability*, vol. 73, no. 1, pp. 695–709, 2024. doi: 10.1109/TR.2023.3295009.

[5] Zoho Corporation, “Creating and Managing Form Workflows.” [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/create-and-manage-form-workflows/articles/create-new-form-workflow

[6] ServiceNow, “Incident Management.” [Online]. Available: https://www.servicenow.com/docs/r/it-service-management/incident-management/c_IncidentManagement.html

[7] Atlassian, “What is Change Management? – Jira Service Management.” [Online]. Available: https://support.atlassian.com/jira-service-management-cloud/docs/what-is-change-management/

[8] Zoho Corporation, “Deluge Documentation.” [Online]. Available: https://www.zoho.com/deluge/

[9] Zoho Corporation, “Understanding Schedules.” [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/schedule-workflows/articles/understand-schedules

[10] Zoho Corporation, “Zoho Desk API Documentation.” [Online]. Available: https://desk.zoho.com/DeskAPIDocument

[11] Zoho Corporation, “Understanding Workflows – Zoho Creator Help.” [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/understand-workflows/articles/understand-workflows

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Application home; menu sections Dashboards, Incidents, Changes, Masters	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Incident, Change_Request, Cloud_Service, Employee (20 field types)	5.2	Fig. 5.2, 5.3; Table 4.7
II	Lookup field (relationship between forms)	Y	Incident.Affected_Service, Reported_By; Employee.Department	4.2, 5.2	Fig. 4.2; Table 4.1
II	Subform (parent–child data)	Y	Incident.Activity_Log; Change_Request.Change_Tasks	4.2, 5.2	Fig. 5.4
II	Field validations / layout rules	Y	Description length, unique email, hidden SLA fields, conditional rollback plan	5.2	Tables 4.7, 4.8
II	List report with filter, grouping, custom action	Y	All_Incidents (Escalate button)	5.3	Fig. 5.5; Table 5.1
II	Kanban / Calendar / Timeline / Map report	Y	Incident_Board, Change_Calendar, Change_Timeline	5.3	Fig. 5.6, 5.7
II	Pivot table / chart report	Y	Incident_Priority_Pivot, Incidents_by_Status_Chart, Monthly_Incident_Trend	5.3	Fig. 5.8
II	Page / dashboard built with UI Builder	Y	Admin_Dashboard, Support_Workbench, Change_Dashboard	5.4	Fig. 5.9, 5.10
II	Deluge – form events (validate / success / field actions)	Y	DS-01, DS-02, DS-03, DS-05, DS-13, DS-14	5.5	Code 5.1, 5.2
II	Deluge – functions, fetch, loops, collections	Y	DS-04, DS-10, DS-11, DS-12; DS-02 Map and loops	5.5	Code 5.2, 5.4
II	Email / in-app notifications	Y	sendmail in DS-03, DS-06, DS-07, DS-08, DS-13	5.5, 5.6	Table 5.2
II	Form workflow	Y	Incident_Defaults, Incident_Validation, Incident_Assignment_Notify, Incident_Status_Notify, Change_Validation	5.6	Table 5.3; Fig. 5.13
II	Scheduled workflow	Y	SLA_Monitor, Pending_Approval_Reminder	5.6	Code 5.3; Fig. 5.13
II	Approval process	Y	Change_Approval	5.6	Fig. 5.12

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Blueprint	Y	Incident_Lifecycle	5.6	Fig. 5.11
II	Batch / report workflow	Y	Auto_Close_Resolved (batch); Escalate_Incident (report)	5.6	Fig. 5.13
II	Testing & debugging evidence	Y	Test cases, defect log, Deluge log	6	Tables 6.1, 6.2; Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Mail (sendmail); Zoho Cliq	5.7	Table 5.4
III	Connection / REST API call (invokeurl)	Y	cliq_conn; DS-11, DS-12	5.7	Code 5.4; Fig. 5.14
III	Webhook or Creator REST API	Y	Creator REST API v2 tested in Postman	5.7	Fig. 5.14
III	Users, roles and permission profiles	Y	Five permission profiles	5.8	Fig. 5.15; Table 4.10
III	Record-level access criteria / customer portal	Y	Added_User == zoho.loginuser (portal not used)	5.8	Fig. 5.15
III	Zia / AI feature	Y	Keyword Extraction field; suggestResolution	5.9	Fig. 5.16
III	Analytics (charts / Zoho Analytics)	Y	Native charts, pivot and KPI tiles	5.9	Fig. 5.8, 5.9
IV	Performance optimisation applied & measured	Y	DS-02, DS-07, DS-08, All_Incidents	7.2	Table 7.2
IV	Deployment across environments	Y	Development → Stage → Production	7.1	Fig. 7.1; Table 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Profiles, record criteria, OAuth connection	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Audit trail, naming conventions, backup	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used; justified in Section 5.10	5.10	—
V	Low-code limitations & spectrum analysis	Y	Discussion of limits and spectrum	8.4, 8.5	—
—	Mobile app / portal access	Y	Zoho Creator mobile app	5.11	Fig. 5.17

Total features implemented: 31 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Automatic Priority and SLA Calculation

```
// Script ID : DS-01
// Location  : Incident Management Form → Workflow "Automatic Priority
and SLA" → On Add, On Validate
// Purpose   : Calculate incident priority and SLA deadline from
Impact and Urgency

// AUTOMATIC INCIDENT PRIORITY AND SLA CALCULATION
if(input.Impact == "High" && input.Urgency == "High")
{
    input.Priority = "Critical";
    input.SLA_Deadline = input.Reported_On.addHour(2);
}
else if((input.Impact == "High" && input.Urgency == "Medium") ||
(input.Impact == "Medium" && input.Urgency == "High"))
{
    input.Priority = "High";
    input.SLA_Deadline = input.Reported_On.addHour(4);
}
else if((input.Impact == "High" && input.Urgency == "Low") ||
(input.Impact == "Medium" && input.Urgency == "Medium") ||
(input.Impact == "Low" && input.Urgency == "High"))
{
    input.Priority = "Medium";
    input.SLA_Deadline = input.Reported_On.addHour(8);
}
else
{
    input.Priority = "Low";
    input.SLA_Deadline = input.Reported_On.addHour(24);
}
```

C.2 Automatic Engineer Assignment

```
// Script ID : DS-02
// Location  : Incident Management Form → Workflow "Automatic Engineer
Assignment" → On Add, On Validate
// Purpose   : Assign the incident to the active Cloud Engineer with
the fewest unresolved incidents

// AUTOMATIC ENGINEER ASSIGNMENT
// Fetch all active Cloud Engineers
eligibleEngineers = Staff_Directory_Form[Role == "Cloud Engineer" &&
Active_Status == true];
// Initialize workload variables
selectedEngineerID = 0;
lowestLoad = 999999;
// Find the engineer with the lowest active incident count
for each engineer in eligibleEngineers
{
    currentLoad = Incident_Management_Form[Assigned_Engineer ==
engineer.ID && Status != "Resolved" && Status != "Closed"].count();
    if(currentLoad < lowestLoad)
    {
        lowestLoad = currentLoad;
        selectedEngineerID = engineer.ID;
    }
```

```
}
// Assign the selected engineer
if(selectedEngineerID != 0)
{
    input.Assigned_Engineer = selectedEngineerID;
}
```

C.3 Incident Assignment Notification

```
// Script ID : DS-03
// Location  : Incident Management Form → Workflow "Incident
Assignment Notification" → On Add, On Success
// Purpose   : Email the assigned engineer the details of the new
incident

// WF-03: INCIDENT ASSIGNMENT NOTIFICATION
// Check whether an engineer is assigned
if(input.Assigned_Engineer != null)
{
    // Fetch the assigned engineer's staff record
    engineer = Staff_Directory_Form[ID == input.Assigned_Engineer];
    // Check whether the engineer's email exists
    if(engineer != null && engineer.Email_Address != null)
    {
        sendmail
        [
            from :zoho.adminuserid
            to :engineer.Email_Address
            subject :"New Incident Assigned - " + input.Incident_ID
            message :"Hello " + engineer.Full_Name + ",\n\n" + "A new
incident has been assigned to you.\n\n" + "Incident ID: " +
input.Incident_ID + "\n" + "Incident Title: " + input.Incident_Title +
"\n" + "Category: " + input.Category + "\n" + "Affected Service: " +
input.Affected_Service + "\n" + "Priority: " + input.Priority + "\n" +
"SLA Deadline: " + input.SLA_Deadline + "\n\n" + "Please investigate
and resolve the incident within the specified SLA deadline.\n\n" +
"Regards,\nIT Incident Management System"
        ]
    }
}
```

C.4 Incident Status Notification

```
// Script ID : DS-04
// Location  : Incident Management Form → Workflow "Incident Status
Notification" → On Edit, update of Status
// Purpose   : Notify the reporter when the incident status changes

// WF-04: INCIDENT STATUS NOTIFICATION
if(input.Reported_By != null)
{
    reporterEmail = Staff_Directory_Form[ID ==
input.Reported_By].Email_Address;
    reporterName = Staff_Directory_Form[ID ==
input.Reported_By].Full_Name;
    if(reporterEmail != null && reporterEmail != "")
    {
        emailBody = "Hello " + reporterName + ",\n\n";
        emailBody = emailBody + "The status of your reported incident
has been updated.\n\n";
        emailBody = emailBody + "INCIDENT DETAILS\n";
```

```
        emailBody = emailBody + "Incident ID: " + input.Incident_ID +
"\n";
        emailBody = emailBody + "Incident Title: " +
input.Incident_Title + "\n";
        emailBody = emailBody + "Category: " + input.Category + "\n";
        emailBody = emailBody + "Priority: " + input.Priority + "\n";
        emailBody = emailBody + "Current Status: " + input.Status +
"\n";
        emailBody = emailBody + "SLA Deadline: " + input.SLA_Deadline
+ "\n\n";
        emailBody = emailBody + "Please check the latest incident
updates.\n\n";
        emailBody = emailBody + "Regards,\nIT Incident Management
System";
        sendmail
        [
            from :zoho.adminuserid
            to :reporterEmail
            subject :"Incident Status Updated - " + input.Incident_ID
            message :emailBody
        ]
    }
}
```

C.5 SLA Overdue Notification

```
// Script ID : DS-05
// Location  : Incident Management Form → Schedule "SLA Overdue
Notification" → starts at SLA_Deadline
// Purpose   : Email the assigned engineer when an unresolved incident
passes its SLA deadline

// WF-05: SLA OVERDUE NOTIFICATION
// Fetch all unresolved, assigned incidents whose SLA deadline has
passed
overdueIncidents = Incident_Management_Form[Status != "Resolved" &&
Status != "Closed" && Assigned_Engineer != null && SLA_Deadline <
zoho.currenttime];
for each incident in overdueIncidents
{
    engineer = Staff_Directory_Form[ID == incident.Assigned_Engineer];
    if(engineer != null && engineer.Email_Address != null &&
engineer.Email_Address != "")
    {
        emailBody = "Hello " + engineer.Full_Name + ",\n\n";
        emailBody = emailBody + "ALERT: INCIDENT SLA DEADLINE
EXCEEDED\n\n";
        emailBody = emailBody + "The following incident has exceeded
its SLA deadline and is still unresolved.\n\n";
        emailBody = emailBody + "INCIDENT DETAILS\n";
        emailBody = emailBody + "------------------------------\n";
        emailBody = emailBody + "Incident ID: " + incident.Incident_ID
+ "\n";
        emailBody = emailBody + "Incident Title: " +
incident.Incident_Title + "\n";
        emailBody = emailBody + "Category: " + incident.Category +
"\n";
        emailBody = emailBody + "Affected Service: " +
incident.Affected_Service + "\n";
        emailBody = emailBody + "Priority: " + incident.Priority +
"\n";
```

```
        emailBody = emailBody + "Current Status: " + incident.Status +
"\n";
        emailBody = emailBody + "SLA Deadline: " +
incident.SLA_Deadline + "\n\n";
        emailBody = emailBody + "ACTION REQUIRED\n";
        emailBody = emailBody + "Please investigate this incident
immediately and update its status.\n\n";
        emailBody = emailBody + "Regards,\n";
        emailBody = emailBody + "IT Incident Management System";
        sendmail
        [
            from :zoho.adminuserid
            to :engineer.Email_Address
            subject :"SLA Breached - Incident " + incident.Incident_ID
            message :emailBody
        ]
    }
}
```

C.6 Change Request Submission Notification

```
// Script ID : DS-06
// Location  : Change Request Form → Workflow "Change Request
Submission Notification" → On Add, On Success (criteria:
Approval_Status == "Pending")
// Purpose   : Notify all active Approving Managers of a new pending
change request

// WF-06: CHANGE REQUEST SUBMISSION NOTIFICATION
// Fetch all active approving managers
managers = Staff_Directory_Form[Role == "Approving Manager" &&
Active_Status == true];
// Fetch requester name
requesterName = "Not specified";
if(input.Requested_By != null)
{
    requesterName = Staff_Directory_Form[ID ==
input.Requested_By].Full_Name;
}
// Fetch change request details
changeID = input.Change_ID.toString();
changeTitle = input.Change_Title;
riskScore = input.Risk_Score.toString();
plannedDate = input.Planned_Date.toString();
// Send notification to active managers
for each manager in managers
{
    if(manager.Email_Address != null && manager.Email_Address != "")
    {
        // Prepare email message
        emailBody = "Hello " + manager.Full_Name + ",\n\n";
        emailBody = emailBody + "A new infrastructure change request
has been submitted and requires your review.\n\n";
        emailBody = emailBody + "CHANGE REQUEST DETAILS\n";
        emailBody = emailBody + "------------------------------\n";
        emailBody = emailBody + "Change ID: " + changeID + "\n";
        emailBody = emailBody + "Change Title: " + changeTitle + "\n";
        emailBody = emailBody + "Risk Score: " + riskScore + "\n";
        emailBody = emailBody + "Requested By: " + requesterName +
"\n";
        emailBody = emailBody + "Approval Status: " +
input.Approval_Status + "\n";
```

```
        emailBody = emailBody + "Planned Date: " + plannedDate +
"\n\n";
        emailBody = emailBody + "Please review the change request and
provide your approval decision.\n\n";
        emailBody = emailBody + "Regards,\n";
        emailBody = emailBody + "IT Change Management System";
        // Send email notification
        sendmail
        [
            from :zoho.adminuserid
            to :manager.Email_Address
            subject :"New Change Request - " + changeID
            message :emailBody
        ]
    }
}
```

C.7 Change Request Submission Notification

```
// Script ID : DS-07
// Location  : Change_Request_Form → Created → On Success
// Purpose   : Send email notifications to all active Approving
Managers when a new change request is submitted.

managers = Staff_Directory_Form[Role == "Approving Manager" &&
Active_Status == true];
// Fetch requester name
requesterName = "Not specified";
if(input.Requested_By != null)
{
      requesterName = Staff_Directory_Form[ID ==
input.Requested_By].Full_Name;
}
// Fetch change request details
changeID = input.Change_ID.toString();
changeTitle = input.Change_Title;
riskScore = input.Risk_Score.toString();
plannedDate = input.Planned_Date.toString();
// Send notification to active managers
for each  manager in managers
{
      if(manager.Email_Address != null && manager.Email_Address !=
"")
      {
            // Prepare email message
            emailBody = "Hello " + manager.Full_Name + ",\n\n";
            emailBody = emailBody + "A new infrastructure change
request has been submitted and requires your review.\n\n";
            emailBody = emailBody + "CHANGE REQUEST DETAILS\n";
            emailBody = emailBody + "------------------------------
\n";
            emailBody = emailBody + "Change ID: " + changeID + "\n";
            emailBody = emailBody + "Change Title: " + changeTitle +
"\n";
            emailBody = emailBody + "Risk Score: " + riskScore +
"\n";
            emailBody = emailBody + "Requested By: " + requesterName
+ "\n";
            emailBody = emailBody + "Approval Status: " +
input.Approval_Status + "\n";
            emailBody = emailBody + "Planned Date: " + plannedDate +
"\n\n";
```

```
            emailBody = emailBody + "Please review the change request
and provide your approval decision.\n\n";
            emailBody = emailBody + "Regards,\n";
            emailBody = emailBody + "IT Change Management System";
            // Send email notification
            sendmail
            [
                  from :zoho.adminuserid
                  to :manager.Email_Address
                  subject :"New Change Request - " + changeID
                  message :emailBody
            ]
      }
}
```

C.8 Change Approval History Automation

```
// Script ID : DS-08
// Location  : Change_Request_Form → Edited → On Success
// Purpose   : Automatically create a Change Approval History record
//             when a change request is approved or rejected.

if(input.Approval_Status == "Approved" || input.Approval_Status ==
"Rejected")
{
      // Check whether approving manager is selected
      if(input.Approved_By != null)
      {
            // Automatically create approval history record
            newApproval = insert into Change_Approval_History
            [
                  Change_Request=input.ID
                  Reviewed_By=input.Approved_By
                  Decision=input.Approval_Status
                  Decision_Date=zoho.currenttime
                  Added_User=zoho.loginuser
            ];

            // Display created record ID in execution logs
            info "Approval History Created Successfully";
            info newApproval;
      }
      else
      {
            info "Please select an Approving Manager.";
      }
}
```

C.9 Automatic Zoho Desk Ticket Creation

```
// Script ID : DS-09
// Location  : Incident_Management_Form → Created → On Success
// Purpose   : Automatically create a corresponding ticket in Zoho
Desk
whenever a new incident is submitted.

orgID = 60094239038;
// Replace with your Organization ID
```

```
departmentID = "282718000000010772";
// Replace with your Department ID
contactID = "282718000000364031";
// Replace with your created Contact ID
ticketData =
{"subject":input.Incident_Title,"description":input.Description,
"departmentId":departmentID,"contactId":contactID};
// Debugging information
info "Starting Zoho Desk Ticket Creation";
info "Incident Title: " + input.Incident_Title;
// Create ticket in Zoho Desk
deskResponse =
zoho.desk.create(orgID,"tickets",ticketData,"zoho_desk_connection");
// Check ticket creation response
if(deskResponse.containKey("id"))
{
      info "Zoho Desk Ticket Created Successfully";
      info "Ticket ID: " + deskResponse.get("id");
      info "Ticket Number: " + deskResponse.get("ticketNumber");
}
else
{
      info "Zoho Desk Ticket Creation Failed";
      info deskResponse;
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

A. Access through a Web Browser

- Open Google Chrome or Microsoft Edge.
- Navigate to the Zoho Creator application link shared by the administrator.
- Enter the registered Zoho account email address and password.
- Click **Sign In** to access the application.
- The application dashboard and available menu sections will be displayed according to the user's assigned role.
- Select the required form or report from the application menu.

B. Access through a Mobile Device

- Open a web browser on the mobile phone or tablet.
- Enter the application link shared by the administrator.
- Sign in using the registered Zoho account credentials.
- Access the available forms and reports based on the assigned permissions.
- Users can optionally add the application to their device home screen if the browser supports this feature.

D.2 Instructions for Employee

- Log in to the application using the registered account.
- Open **Incident Management Form** to report an IT issue.
- Enter the incident title, description, category, affected service, impact and urgency.
- Select the reporting employee in the **Reported_By** field, if required.
- Attach a supporting file or screenshot if available.
- Submit the form.
- The system calculates the priority and SLA deadline and assigns an engineer through the configured workflow.

- Open **All Incident Report** to view available incident records.
- To request an infrastructure change, open **Change Request Form**.
- Enter the change title, description, risk score, implementation plan and rollback plan, then submit the request.
- View the request status through **All Change Requests**.

D.3 Instructions for IT Support Staff

- Log in using the assigned account.
- Open **All Incident Report** to view incident records.
- Review the incident title, description, priority, affected service and SLA deadline.
- Check the assigned engineer and current incident status.
- Investigate the reported issue and coordinate with the assigned Cloud Engineer when required.
- Monitor progress through the incident report.
- Use the available status update option to track incident progress, according to the assigned permissions.

D.4 Instructions for Cloud Engineer

- Log in using the assigned account.
- Open **Assigned Incidents** to view the incidents assigned to you.
- Select an incident and review its description, category, priority and SLA deadline.
- Investigate the technical issue.
- Update the incident status as work progresses.
- Enter the resolution details after resolving the issue.
- Save the updated record.
- Verify the incident status in the assigned incident report.

D.5 Instructions for Approving Manager

- Log in using the assigned account.
- Open **Pending Change Approval** to view requests awaiting a decision.
- Select a change request and review its description and risk score.
- Examine the implementation plan and rollback plan.
- Select the appropriate approval decision.
- Update the **Approval_Status** field as Approved or Rejected.
- Enter comments explaining the decision, wherever applicable.
- Save the record.
- Open **Change Approval History Report** to review recorded approval decisions.

D.6 Instructions for Administrator

- Log in using the administrator account.
- Open the application dashboard to access the available sections and reports.
- Open **Staff Directory Form** to add or update employee details.
- Maintain employee roles and active status.
- Open the application user-management settings to manage users and assign the appropriate permission profiles.
- Review incident records through **All Incident Report**.
- Review change requests through **All Change Requests**.
- Monitor approval records through **Change Approval History Report**.
- Verify the functioning of configured workflows and notifications.
- Manage application settings and user access as required.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	SYED FAIYAZ AHAMED	**Register No.**	241401113
Project Title	Intelligent Cloud Incident & Change Management System	**Date**	03 November 2026

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	Syed Faiyaz Ahamed	**Register No.**	241401113
Department / Section	B.Tech CSBS 'B' Sec	**Date of Viva**	03 November 2026
Application URL Verified	Yes / No	**Demo Duration**	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____